RETRO DEV KIT APPLE 1 — ПЕРВОРОДНЫЙ ГРЕХ

Полное руководство

Эмулятор, кассетный интерфейс ACI, цветная карта GEN2, отладчик — с полными руководствами по Integer BASIC и ассемблеру 6502



ВЕРСИЯ 1.0 НОЯБРЬ 2026 РУССКОЕ ИЗДАНИЕ

Содержание

ЭМУЛЯТОР

1 Об этом руководстве 8

1.1 Как устроено руководство ... 8
1.2 Условные обозначения ... 8
1.3 Что понадобится ... 9
1.4 Основные пути ... 10

2 Окно и панель инструментов 11

2.1 Общий вид ... 11
2.2 Панель инструментов, кнопка за кнопкой ... 12
2.3 Строка состояния ... 15
2.4 Клавиатура Apple-1 ... 15
2.5 Состояние машины ... 16
2.6 Программатор EPROM ... 17

3 Экран: масштаб, люминофор и винтажный эффект 19

3.1 Оригинальный экран ... 19
3.2 Строка отображения ... 19
3.3 Винтажный экран ... 20

4 Библиотека 23

4.1 Папки ... 23
4.2 Запуск программы ... 24
4.3 Параметры программы ... 25
4.4 Распознанные кассеты ... 26
4.5 Проекты GitHub ... 27
4.6 Папка «Art ASCII» ... 28

5 Кассета и карта ACI 29

5.1 Форматы файлов ... 29
5.2 Виртуальный магнитофон ... 30
5.3 Звук и осциллограф ... 30
5.4 Запись с эмулируемого Apple-1 ... 31
5.5 Как работает ACI ... 31
5.6 Команды ACI ... 32
5.7 На настоящий Apple-1 ... 32
5.8 Какой нужен уровень? Модели входа ... 33
5.9 Оцифровка настоящей кассеты или записи Apple-1 ... 34

6 Графические карты: GEN2 и P-LAB 35

6.1 Подключение карты ... 35
6.2 Программные переключатели ... 36
6.3 Режимы и память ... 36
6.4 Тестовые программы ... 37
6.5 Что приближённо ... 38
6.6 Графическая карта P-LAB ... 38

7 Редактор 42
7.1 Вкладки и файлы ... 42
7.2 Поиск, замена и меню правого щелчка ... 43
7.3 Программа на BASIC ... 44
7.4 Исходник 6502 ... 45

8 Отладчик, дизассемблер и дамп 47
8.1 Панель отладчика ... 47
8.2 Точки останова ... 47
8.3 Отладка программы на BASIC ... 48
8.4 Отладка программы для 6502 ... 49
8.5 Дамп ... 49
8.6 В командной строке ... 50

9 Фото → ASCII 51
9.1 Открытие изображения ... 51
9.2 Настройки ... 52
9.3 Получаемая программа ... 52
9.4 В командной строке ... 53

10 Расположение панелей 54
10.1 Перемещение панели ... 54
10.2 Исходное расположение ... 54
10.3 Помощник при запуске ... 54
10.4 Размер текста каждого окна ... 55
10.5 Что запоминается между запусками ... 55
10.6 Когда экран больше своей панели ... 55
10.7 Открепление панели ... 56

11 Настройки 57

12 Командная строка 60
12.1 Эмулятор в терминале ... 60
12.2 Преобразования ... 62
12.3 BASIC и ассемблер ... 63
12.4 ASCII-арт ... 63
12.5 ППЗУ ... 63
12.6 Графический интерфейс ... 64

13 Неполадки и частые вопросы 65
13.1 Окно не открывается ... 65
13.2 Клавиши не доходят до Apple-1 ... 65
13.3 «Integer BASIC не найден» ... 65
13.4 Программа на BASIC останавливается с STOPPED AT ... 65
13.5 Программа не запускается после загрузки ... 65
13.6 Кассета не загружается на настоящем Apple-1 ... 65
13.7 Экран заполнен инверсными @ ... 66
13.8 Символы выводятся медленно ... 66
13.9 Звук кассеты не слышен ... 66
13.10 Проект GitHub не собирается ... 66
13.11 Чего эмулятор не делает ... 66

14 Установка ПЗУ или оригинального Integer BASIC **67**
14.1 Что уже встроено ... 67
14.2 Оригинальный Integer BASIC ... 67

INTEGER BASIC

15 Первые шаги в BASIC **69**
15.1 Два BASIC: RDK G2 Basic и оригинал ... 69
15.2 Запуск BASIC ... 70
15.3 Прямой режим и режим программы ... 70
15.4 Исправление строки ... 71
15.5 Первая программа ... 71
15.6 Команды ... 71
15.7 Три программы для набора ... 72

16 Справочник операторов и функций **74**
16.1 Операторы ... 74
16.2 Функции ... 75
16.3 Операции ... 76
16.4 Расширения RDK G2 Basic ... 76
16.5 Ключевые слова, которые ничего не делают ... 77

17 Переменные, строки, массивы и память **78**
17.1 Числа ... 78
17.2 Имена переменных ... 78
17.3 Строки ... 78
17.4 Массивы ... 79
17.5 Память BASIC ... 80
17.6 Смешивание BASIC и машинного языка ... 82

18 Сообщения об ошибках и ограничения **83**
18.1 Сообщения ... 83
18.2 Ограничения ... 84
18.3 Экономия памяти ... 84

19 Известные ошибки и способы их обхода **85**
19.1 Выход из цикла FOR через GOTO ... 85
19.2 Операция ^ вешает машину ... 86
19.3 Сообщение «PPED AT» ... 86
19.4 IF ... THEN управляет только первым оператором ... 86
19.5 −32768 ... 86
19.6 ASC(не работает ... 87
19.7 Графические ключевые слова, которые ничего не делают ... 87
19.8 Ошибка 251 символа Apple II ... 88
19.9 STOPPED AT 65535 ... 88
19.10 USR роняет машину ... 88
19.11 HIMEM= не может опуститься ... 88
19.12 Двоеточие отвергается в прямом режиме ... 88
19.13 INPUT всегда добавляет «?» ... 89
19.14 Цикл FOR выполняется хотя бы раз ... 89
19.15 Нажатие клавиши останавливает программу ... 89
19.16 Сводка ... 90

20 Перенос программы с Applesoft или Microsoft BASIC **91**
20.1 Соответствия ... 92
20.2 Метод ... 92
20.3 Пример переноса ... 93
20.4 Уроки переносов ... 94

АССЕМБЛЕР 6502

21 6502 вкратце **96**
21.1 Регистры ... 96
21.2 Режимы адресации ... 97
21.3 Инструкции по группам ... 97
21.4 NMOS 6502, а не 65C02 ... 98
21.5 Карта памяти с первого взгляда ... 98

22 Ассемблер devkit **99**
22.1 Рабочий цикл ... 99
22.2 Листинг и символы ... 101
22.3 Командная строка ... 101
22.4 Синтаксис ... 102
22.5 Директивы ... 103
22.6 Макросы, .repeat и условное ассемблирование ... 104
22.7 Сообщения об ошибках ... 104
22.8 Совместимость с ca65 и xa ... 106

23 Клавиатура, дисплей и Woz Monitor **107**
23.1 Четыре регистра PIA ... 107
23.2 Чтение клавиши ... 108
23.3 Вывод символа ... 108
23.4 Подпрограммы Woz Monitor ... 109
23.5 Запуск и завершение программы ... 109
23.6 PIA, кассета и цветная карта ... 110

24 Примеры с комментариями **111**
24.1 `hello.s`: вывод сообщения ... 112
24.2 `clavier.s`: эхо клавиш ... 113
24.3 `hexa.s`: дамп в стиле монитора ... 113
24.4 `decimal.s`: вывод числа в десятичном виде ... 114
24.5 `jeu.s`: угадай цифру ... 116
24.6 `macros.s`: пусть работает ассемблер ... 118
24.7 `gen2-lores.s`: цвет в низком разрешении ... 119
24.8 `gen2-hires.s`: высокое разрешение ... 121

25 Советы, ловушки и ограничения **124**
25.1 Где разместить программу ... 124
25.2 Жизнь в пределах дальности перехода ... 124
25.3 Строки и бит 7 ... 124
25.4 Небольшая пауза ... 125
25.5 Эффективная отладка ... 125
25.6 Переход на настоящий Apple-1 ... 126
25.7 Ограничения встроенного ассемблера ... 126

ПРИЛОЖЕНИЯ

A	Карта памяти	128
B	Команды Woz Monitor	130
C	Временные параметры ACI	131
D	Набор символов 2513	132
E	Сочетания клавиш	133
F	Ловушки и особенности поведения	134
G	Лицензия и благодарности	136

ЧАСТЬ I

Эмулятор

Окно, экран, библиотека, кассетный интерфейс ACI, графические карты GEN2 и P-LAB, отладчик и командная строка

ГЛАВА 1

Об этом руководстве

Retro Dev Kit Apple 1 — *«Первородный грех» — воспроизводит Apple-1 1976 года такт в такт: 6502, видеотерминал, клавиатуру, кассетный интерфейс ACI и, по желанию, цветную карту GEN2 Uncle Bernie и графическую карту P-LAB на TMS9918A. В комплекте идёт собственный свободный BASIC —* **RDK G2 Basic**. *Вокруг машины он добавляет полноценную мастерскую: библиотеку программ, редактор, ассемблер, отладчик и экспорт на настоящий Apple-1.*

Это руководство описывает весь эмулятор, версию 1.0 (первую публичную версию; номера 7.x обозначали тестовые версии). Оно дополняет *Краткое руководство*, которого достаточно, чтобы установить программу и запустить первую кассету. Здесь объяснены каждая панель, каждая кнопка и каждая команда, с примерами, которые можно набирать как есть.

1.1 Как устроено руководство

Часть I — Эмулятор

Окно и его панели, винтажный экран, библиотека, кассета и ACI, графические карты GEN2 и P-LAB, редактор, отладчик, ASCII-арт, настройки, командная строка и решение проблем.

Часть II — Integer BASIC

BASIC Стива Возняка и его свободный клон RDK G2 Basic: все операторы, функции и операции, память, сообщения об ошибках и известные ошибки с обходными путями.

Часть III — Ассемблер 6502

6502 с самых основ, встроенный ассемблер и его директивы, подпрограммы Woz Monitor, полные примеры и ловушки, которых стоит избегать.

Приложения

Карта памяти, команды Woz Monitor, формат ACI, набор символов 2513, сочетания клавиш и указатель команд.

1.2 Условные обозначения

Клавиши записаны так: <kbd>Ctrl</kbd>+<kbd>R</kbd>, <kbd>RETURN</kbd>. Кнопки интерфейса выделены жирным: ▶ **Запустить**. Адреса шестнадцатеричные, с префиксом `$`, как в документах того времени: `$FFEF`. Сам Woz Monitor пишет их без `$`: `FFEF`.

Сеансы на экране Apple-1 показаны на тёмном фоне. То, что набираете **вы**, — белым, ответ машины — зелёным:

```
300R

0300: A2
HELLO, APPLE-1!
```
WOZ MONITOR

По ходу текста встречаются четыре вида врезок:

> ✦ СОВЕТ
> Сокращённый путь или более удачный способ что-то сделать.

> ! ВНИМАНИЕ
> Ловушка, которая стоит времени или данных.

> ▣ НА НАСТОЯЩЕЙ МАШИНЕ
> Что происходит на настоящем Apple-1 и чем это отличается от эмулятора.

> ИЗВЕСТНАЯ ОШИБКА
> Дефект оригинального ПЗУ и способ его обойти. У каждой ошибки есть статус: ПРОВЕРЕНО (воспроизведено в эмуляторе с настоящим Integer BASIC) или НЕ ПРОВЕРЕНО (взято из источника, не опробовано).

Каждый пример этого руководства опробован в эмуляторе. Программы 6502 из части III лежат в папке `docs/exemples/en/asm/` дистрибутива (с английскими комментариями). Встроенный ассемблер и ca65 дают для них в точности одинаковые байты.

1.3 Что понадобится

Эмулятор работает в Linux и ничего не требует устанавливать. Woz Monitor и ПЗУ ACI встроены, как и **RDK G2 Basic** — свободный BASIC, совместимый с Integer BASIC (глава 14): кнопка **BASIC** работает сразу, на свежей установке, без поиска файлов. **Integer BASIC** от Apple (1976) в комплект **не** входит, так как он всё ещё защищён авторским правом. Если хотите оригинал, используйте свою копию (например, кассету `BASIC.wav` из коллекции «Apple1Cassettes» на archive.org) и выберите её в ⚙ **Настройки → Дополнительно → BASIC**.

> ! ПЗУ И КАССЕТЫ ОСТАЮТСЯ У ВАС
> Эмулятор не содержит ни Integer BASIC от Apple, ни коммерческих кассет. Его таблица распознавания хранит только размеры, отпечатки и параметры. Держите свои копии в личной библиотеке.

ЛИЦЕНЗИЯ

Retro Dev Kit Apple 1 — свободное программное обеспечение под лицензией **GNU GPL 3.0** (без гарантий). Некоторые части сохраняют собственную лицензию MIT: RDK G2 Basic, а также тесты и глифы GEN2 от Uncle Bernie. См. `LICENSE` и `NOTICE` в архиве и приложение «Лицензия и благодарности».

1.4 Основные пути

Четыре способа работы с эмулятором встречаются постоянно. Каждый описан дальше со своей схемой:

Я хочу…	Путь	См.
запустить программу того времени	библиотека → ▶	глава 4
написать программу на BASIC	📝 **Редактор** → ▶ **Запустить** → 📼 **Экспорт WAV**	часть II
написать программу для 6502	📝 **Редактор** (`.s`) → 📃 **Листинг** → 🐛 **Отладчик**	часть III
отправить программу на настоящий Apple-1	📼 **Экспорт WAV** (`.aiff`) → смартфон → ACI	глава 5

ГЛАВА 2

Окно и панель инструментов

2.1 Общий вид

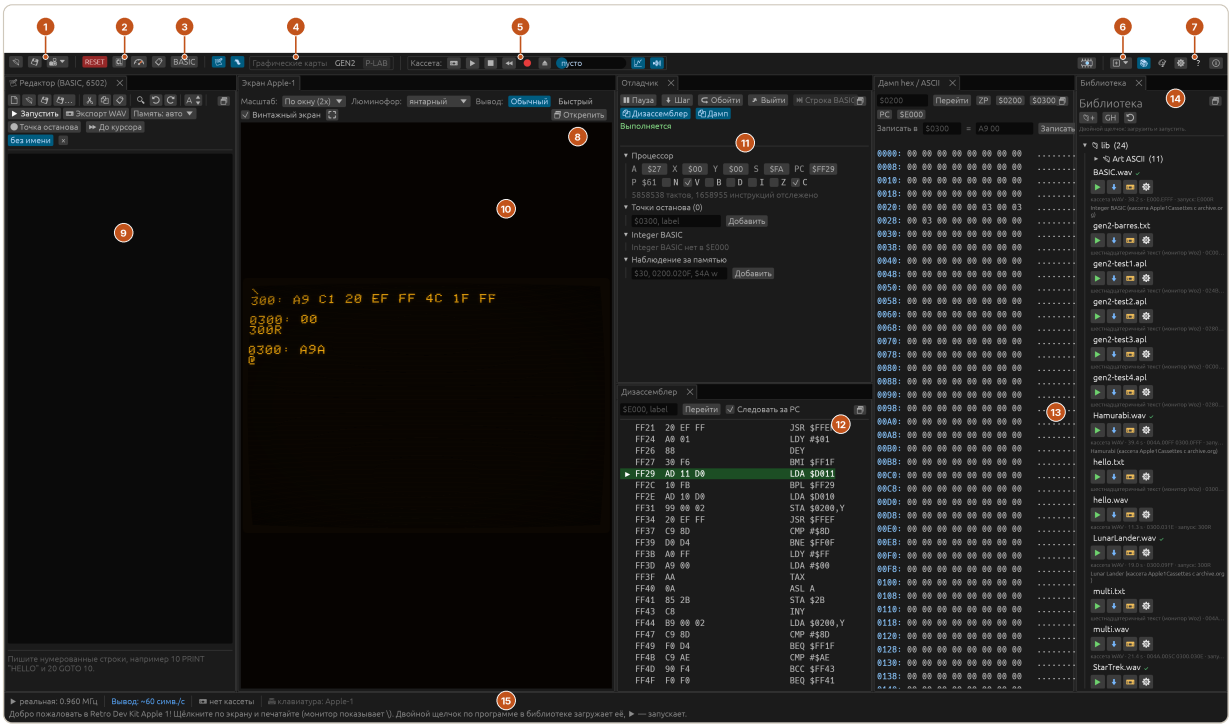


Рисунок 1 — Расположение по умолчанию: все панели открыты и закреплены.

1. **Загрузить, Сохранить, Состояние**: шестнадцатеричные дампы `.txt`, двоичные файлы `.bin` и `.prg`; «Сохранить» также записывает кассеты `.wav` / `.aiff`; «Состояние» сохраняет и восстанавливает машину.
2. **RESET, Очистить экран, Скорость (шкала), Вставить**: кнопки машины (Ctrl+R, Ctrl+L, F3, Ctrl+V).
3. **BASIC**: загружает встроенный RDK G2 Basic в `$E000` (или ваш Integer BASIC, если вы его выбрали).
4. **Окна**: меню ⊞ **Расположение ▾**, затем по кнопке на каждую панель (Монохромный экран, Цветной экран, Редактор, Отладчик, Дизассемблер, Дамп, Библиотека, ASCII).
5. **Графические карты**: **GEN2** и **P-LAB** подключают и отключают карты.
6. **Кассета**: 📼 **Вставить** кассету, затем воспроизведение, стоп, перемотка, запись, извлечение; осциллограф и звук.
7. **EPROM**, ⚙ **Настройки**, **?** (подробные подсказки, также F1) и **ℹ** (о программе, сочетания клавиш).
8. **Строка отображения экрана**: масштаб (от 1x до 8x или «По окну»), люминофор, скорость вывода **Обычный** или **Быстрый**, винтажный экран, полный экран, **Открепить**.
9. **Редактор**: программы на BASIC и исходники 6502 во вкладках.
10. **Экран Apple-1**: 40 × 24 символа; щёлкните по нему, чтобы печатать; правый щелчок — Копировать, Вставить, Очистить экран и Сброс.
11. **Отладчик**: пауза, пошаговое выполнение, регистры, точки останова, переменные.

12. **Дизассемблер**: код вокруг счётчика команд (PC).
13. **Дамп**: память в шестнадцатеричном виде и ASCII, с возможностью правки.
14. **Библиотека**: ваши программы, у каждой — маленькие значки ▶ ⬇ 📼 ⚙.
15. **Строка состояния**: скорость, темп вывода, кассета, куда идёт клавиатура; под ней — последнее сообщение.

Каждая панель (экран, редактор, отладчик, библиотека...) — это вкладка, которую можно переместить, сгруппировать с другими или вынести в отдельное окно: об этом глава 10. При первом запуске окно открывается развёрнутым, и окно **Ваше рабочее пространство** предлагает пять расположений (глава 10); размер и положение окна и расстановка его панелей запоминаются и при следующем запуске находятся такими, какими были.

2.2 Панель инструментов, кнопка за кнопкой

Панель инструментов — одна строка: машина и её инструменты слева, магнитофон посередине, панели (группа «Окна»), карты и настройки справа. Когда окно слишком узкое, у кнопок остаются только значки (текст остаётся во всплывающей подсказке) или, если снять в настройках флажок **Компактная панель**, панель переносится на несколько строк.

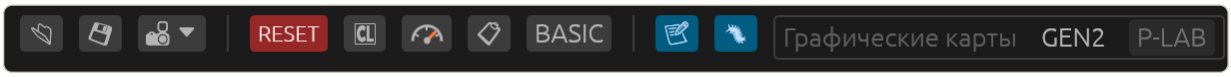


Рисунок 2 — Панель инструментов до группы «Графические карты»: файлы, машина, окна и карты. Далее идёт участок «Кассета» (глава 5), затем настройки.

Кнопка	Что делает
Загрузить	Открывает программу и помещает её в память: `.txt` (дамп Woz Monitor) или `.hex`; для `.bin` спрашивается адрес. Кассеты (`.wav`, `.aiff`) идут не сюда: их вставляют кнопкой 📼 **Вставить** на участке «Кассета».
Сохранить	Спрашивает одну или несколько областей (`0300.03FF`, `004A.00FF 0300.0FFF` …), затем файл. Формат выбирается расширением: `.wav`, `.aiff`, `.txt` или `.bin`.
RESET (красная кнопка)	Кнопка RESET Apple-1 (Ctrl+R). 6502 и PIA перезапускаются; память и экран сохраняются.
🆑 **Очистить экран**	Кнопка CLEAR SCREEN (Ctrl+L).
Шкала (Реальная скорость / Быстро)	Переключает между исходными 1 МГц и максимальной скоростью (F3). Циферблат спидометра: подсвечен синим при быстрой скорости.
📋 **Вставить**	Печатает буфер обмена на клавиатуре Apple-1 в темпе клавиатуры (Ctrl+V). Идеально для шестнадцатеричного дампа.
BASIC	Загружает встроенный **RDK G2 Basic** в `$E000` (7 933 байта, затем `E000R` его запускает). Если в ⚙ **Настройки → Дополнительно → BASIC** вы выбрали оригинальный Integer BASIC, загружается он: кассета, дамп `.txt` или `.bin` размером 4096 байт. Он ищется в библиотеке (`BASIC.wav`, `integer_basic.bin`, любое имя со словом «basic»), иначе спрашивается. Правый щелчок: выбрать другой файл.
Окна (группа: ⊞ **Расположение** ▾ и по кнопке на каждую панель)	Показывает или скрывает каждую панель: **Монохромный экран** (терминал Apple-1), **Цветной экран** (экран подключённой графической карты, GEN2 или P-LAB: одна и та же кнопка служит для обеих; если карта не подключена, она открывает выбор, какую подключить, и **P-LAB** тогда показывает свой экран), 📝 **Редактор** (глава 7), 🐛 **Отладчик**, **Дизассемблер**, **Дамп** (глава 8), 📚 **Библиотека** и 🎨 **ASCII** (глава 9). Подсвеченная кнопка означает, что панель видна; уже показанную панель можно закрыть и крестиком на её вкладке.
Графические карты: GEN2 и P-LAB	Именованная группа. **GEN2** подключает или отключает цветную карту Uncle Bernie (глава 6); **P-LAB** делает то же для карты P-LAB на TMS9918A (раздел 6.6). Кнопка подсвечена, пока её карта подключена; экран карты появляется в отдельной панели, которую показывает кнопка **Цветной экран**.
📸 **Состояние** ▾	Одна кнопка с меню: **Сохранить состояние…** (Ctrl+Shift+S), **Загрузить состояние…** (Ctrl+Shift+O), затем последние состояния, каждое с названием и датой (раздел 2.5).
EPROM (микросхема с УФ-окном, справа на панели, перед ⊞ **Расположение**)	Открывает программатор EPROM: чтение, запись или проверка микросхемы программатором `minipro` (раздел 2.6).
Кассета: 📼 **Вставить** ▶ ■ ⏪ ● ⏏	Магнитофон, собранный на одном участке. 📼 **Вставить** выбирает `.wav` или `.aiff` и помещает его в магнитофон как виртуальную кассету; затем воспроизведение, стоп, перемотка, запись, извлечение. Участок показывает имя, позицию и длительность.
📈 и 🔊	Осциллограф кассеты и звук сигнала (глава 5).
⊞ **Расположение** ▾ (первая кнопка группы «Окна»)	Меню: **По умолчанию** (все панели, как при первом запуске), **Игра** (экран Apple-1 и библиотека), **BASIC** (редактор и экран), **Ассемблер** (редактор, экран и отладчик), **Отладка** (редактор, отладчик, дизассемблер и дамп); затем **Сохранить текущее расположение…** (спрашивает имя) и ваши сохранённые расположения по алфавиту (правый щелчок: Переименовать, Удалить). Галочка отмечает последний выбор; всё хранится в `gui.json`.

Кнопка	Что делает
⚙	Настройки (глава 11).
?	Режим справки (F1): подробные подсказки к каждому элементу на 7 языках интерфейса.
i	«О программе» со списком сочетаний клавиш.

Окна выбора файлов. Открытие, «Сохранить как» и экспорт используют собственное окно выбора файлов эмулятора. Синяя кнопка 📁+ **Новая папка** вверху слева создаёт папку внутри показанной папки; у неё есть подсказка. Введите имя, нажмите Enter — и новая папка готова принять ваш файл.

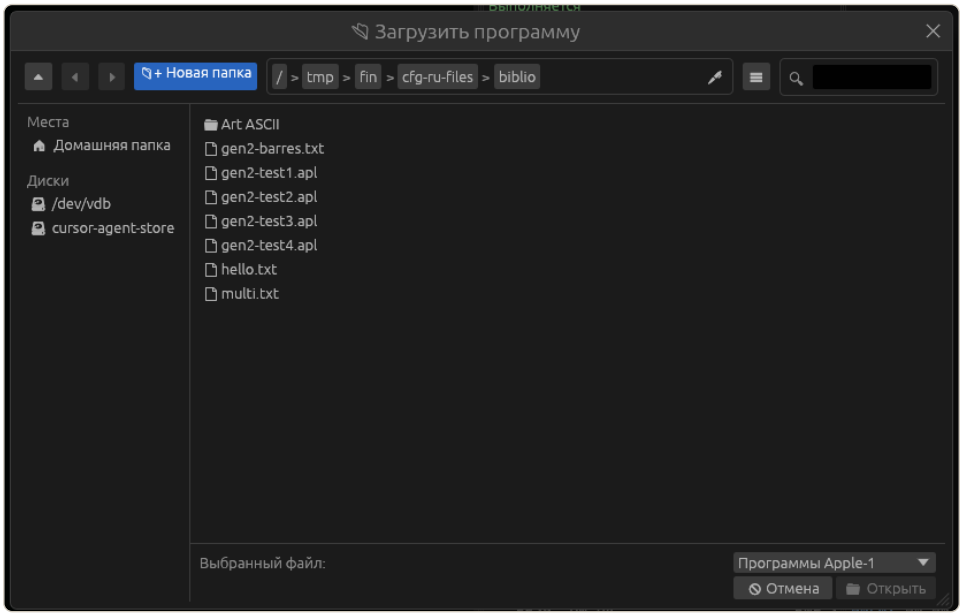


Рисунок 3 — Окно «Загрузить» с кнопкой 📁+ Новая папка вверху слева.

Настроек отображения на панели инструментов нет: у каждого экрана своя **строка отображения** вверху его панели (раздел 3.2).



Рисунок 4 — Строка отображения экрана Apple-1: масштаб, люминофор, скорость вывода, винтажный экран и полный экран; «Открепить» — в правом конце строки.

> ✦ **РЕЖИМ СПРАВКИ**
>
> Когда осваиваете панель, включите **?**: каждая кнопка объясняет, что она делает, при наведении. В выключенном режиме остаются только обычные короткие подсказки (их можно отключить в ⚙ → **Интерфейс**). Настройка запоминается.

2.3 Строка состояния



Рисунок 5 — Строка состояния во время воспроизведения BASIC.wav: скорость, ограничение вывода, кассета, клавиатура и последнее сообщение внизу.

- **Измеренная скорость**: около 0,960 МГц на реальной скорости. Это эффективная частота Apple-1, который останавливает 6502 на четыре такта из шестидесяти пяти для регенерации динамической памяти.
- **Вывод ~60 симв./с**: терминал Apple-1 выводит только один символ за кадр, около 60 в секунду. Щелчок переключает между этим ограничением и мгновенным выводом, как кнопки **Обычный** / **Быстрый** строк отображения.
- **Кассета**: имя, позиция и длительность или «нет кассеты».
- **Клавиатура**: куда идут нажатия (в Apple-1 или в текстовое поле интерфейса).

2.4 Клавиатура Apple-1

Щёлкните по экрану: нажатия пойдут в Apple-1. На оригинальной клавиатуре есть только заглавные буквы, и эмулятор преобразует строчные. Кириллица Apple-1 недоступна: переключите раскладку на латиницу.

Клавиша ПК	Для Apple-1
буквы, цифры, знаки	символ, заглавный
Enter	RETURN (код `$8D`)
Backspace	`_`: Woz Monitor и BASIC стирают предыдущий символ (он остаётся на экране, а за ним `_`)
Esc	ESC: отменяет текущую строку в Woz Monitor
Ctrl+буква	управляющий символ (`Ctrl+D` = `$84` …)
Ctrl+R / Ctrl+L	кнопки RESET и CLEAR SCREEN

На любом экране можно **выделять текст мышью**: нажмите и проведите по символам, выделение подсвечивается. **Правый щелчок** по любому экрану (Apple-1 или монитору GEN2) открывает небольшое меню: **Копировать** помещает в буфер обмена выделенный текст (или весь текст экрана, если ничего не выделено), **Вставить (напечатать текст)** печатает буфер обмена на клавиатуре Apple-1, как если бы вы набирали его сами, далее **Очистить экран** и **Сброс**.

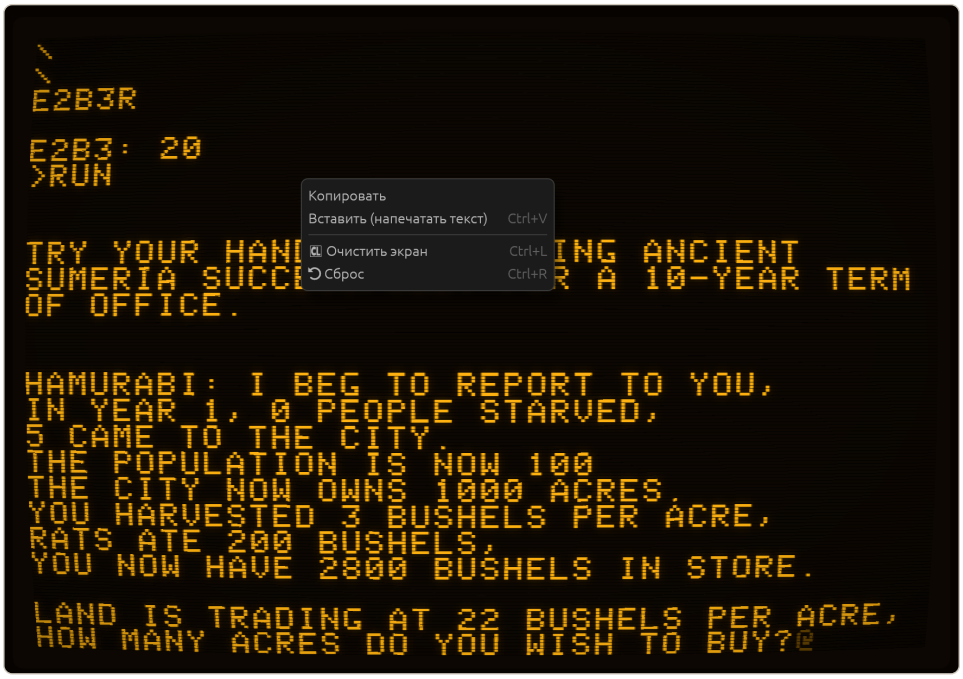


Рисунок 6 — Меню правого щелчка экрана Apple-1 во время игры в Hamurabi.

> ▣ **ЭКРАН APPLE-1 НИЧЕГО НЕ СТИРАЕТ**
>
> Терминал умеет только выводить символ и переходить на следующую строку. Нет ни подвижного курсора, ни способа стереть символ: клавиша стирания печатает `_`, а символ остаётся на экране. Это нормально.

2.5 Состояние машины

Кнопка 📸 **Состояние** на панели инструментов делает полный снимок машины и позже возвращает её точно в то же состояние: можно остановить игру или работающую программу, закрыть эмулятор и продолжить на следующий день. Кнопка открывает меню:

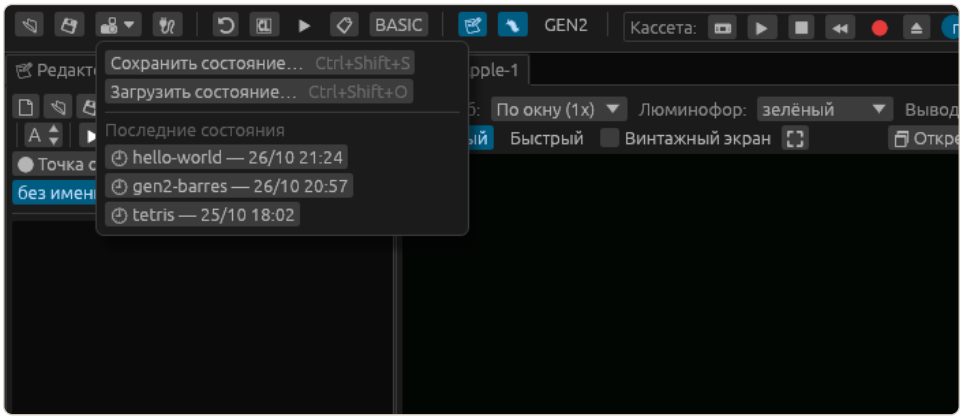


Рисунок 7 — Меню «Состояние»: сохранить, загрузить, затем последние состояния с датой.

- **Сохранить состояние…** (Ctrl + Shift + S) спрашивает имя файла и записывает файл `.a1state`. Предлагаемая папка — `etats`, рядом с папкой библиотеки.

- **Загрузить состояние…** (Ctrl+Shift+O) заменяет машину выбранным состоянием. Строка состояния показывает файл и значение PC.
- **Последние состояния** перечисляет восемь последних с названием и датой сохранения; щелчок сразу загружает состояние. Если файл перемещён или удалён, появится сообщение.

Состояние содержит память, регистры 6502, PIA, экран, карту GEN2 (если она подключена) и виртуальную кассету с её позицией. Настроек и библиотеки в нём нет. Файл, который не читается (повреждён или не является состоянием), отклоняется с сообщением, а машина остаётся нетронутой.

> ✦ ТИПИЧНОЕ ПРИМЕНЕНИЕ
>
> Сохраните состояние перед трудным местом игры или в конце долгого сеанса BASIC: состояние — ещё и удобный способ передать работающую программу другу с той же версией.

2.6 Программатор EPROM

Кнопка **EPROM** на панели инструментов открывает окно, в котором из эмулятора можно прочитать, записать и проверить микросхему EPROM или EEPROM, например чтобы прошить ПЗУ с BASIC, программу для расширения GEN2 или содержимое вашей собственной памяти.

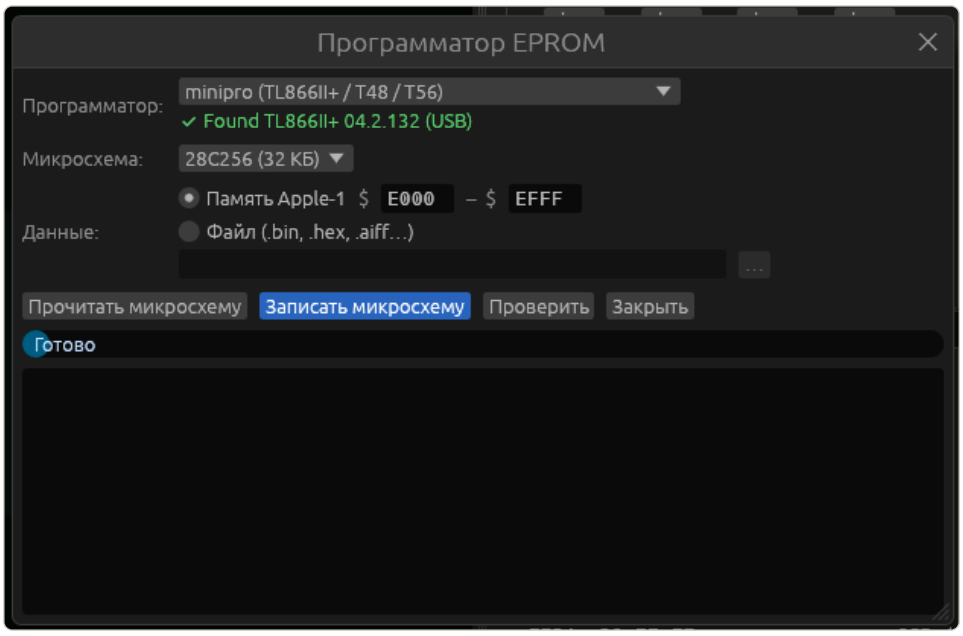


Рисунок 8 — Программатор EPROM: программатор, микросхема, источник данных и три операции.

Поддерживаются только программаторы, управляемые свободной утилитой `minipro`: XGecu **TL866II+**, **T48** и **T56**. В Debian, Ubuntu и Linux Mint установите её командой `sudo apt install minipro`, подключите программатор к USB-порту и нажмите **Найти**. Если `minipro` отсутствует, окно сообщит об этом и подскажет, что делать; со стороны эмулятора больше ничего не требуется.

Параметр	Назначение
Программатор	найденный программатор с версией прошивки (например, «Found TL866II+ 04.2.132 (USB)»)
Микросхема	распространённые микросхемы мира Apple-1: 2716, 2732, 2764, 27C64, 27C128, 27C256, 27C512, 28C64, 28C256, SST27SF256, SST27SF512 и масочное ПЗУ 23128 (только чтение)
Данные	**память Apple-1** (диапазон адресов, по умолчанию `$E000` – `$EFFF`) или **файл** (`.bin`, `.hex`, `.aiff`…)

Три кнопки действуют на микросхему, а индикатор выполнения и журнал показывают, что происходит:

- **Прочитать микросхему** читает всю микросхему. Если данными служат память или существующий файл, содержимое **сравнивается** и различия перечисляются: существующий файл никогда не перезаписывается. Если указано новое имя файла, содержимое сохраняется в него.
- **Записать микросхему** программирует микросхему выбранными данными. Микросхема должна быть чистой или стираемой, а в конце показывается подтверждение результата.
- **Проверить** побайтно сравнивает микросхему с выбранными данными и сообщает о каждом различии (`$E010: микросхема $4C, данные $A9`).

Кнопка **Закрыть** закрывает окно.

Окно никогда не изменяет эмулируемую память: запись и проверка её читают, а чтение лишь сравнивает её или заполняет файл.

ГЛАВА 3

Экран: масштаб, люминофор и винтажный эффект

3.1 Оригинальный экран

Apple-1 выводит 40 столбцов × 24 строки заглавных символов. Каждый символ — матрица 5 × 7 точек в ячейке 7 × 8, взятая из ПЗУ знакогенератора **Signetics 2513**: эмулятор использует содержимое этого ПЗУ точка в точку (приложение D). Изображение, таким образом, имеет 280 × 192 точки.

Курсор — мигающий `@`, которым управляет NE555: 2,18 Гц, виден примерно две трети времени. На некоторых платах допуск конденсатора даёт 1,7 Гц; параметр `--cursor-hz 1.7` воспроизводит это.

3.2 Строка отображения

У каждого экрана своя строка настроек вверху его панели: у экрана Apple-1 и у цветного монитора, когда подключена карта GEN2 (глава 6).

Настройка	Назначение
Масштаб	список: от `1x` до `8x` увеличивают изображение в целое число раз, так что точки остаются квадратными и чёткими; **По окну** выбирает наибольший множитель, который помещается в панель (выбранный множитель указан в скобках). С клавиатуры: Ctrl++, Ctrl+–, Ctrl+0 (по окну).
Люминофор	только экран Apple-1: зелёный, как у мониторов того времени, белый, как у телевизора, или янтарный.
Вывод: Обычный / Быстрый	**Обычный** выводит символы в темпе настоящего терминала, около 60 в секунду; **Быстрый** показывает всё сразу. Настройка общая: изменение на одном экране меняет её и на другом.
Винтажный экран	эффект кинескопа 1970-х (следующий раздел); у каждого экрана свой флажок.
⛶	полный экран (F11).
Открепить	выносит экран в отдельное окно (глава 10).

Каждый экран **центрируется** в своей панели по горизонтали и вертикали, а подпись под изображением тоже по центру. Если выбранный масштаб больше панели, экран показывается в наибольшем масштабе, который помещается, а не обрезается. Когда окно уже строки отображения, строка переносится на вторую линию, а **Открепить** остаётся у правого края последней линии.

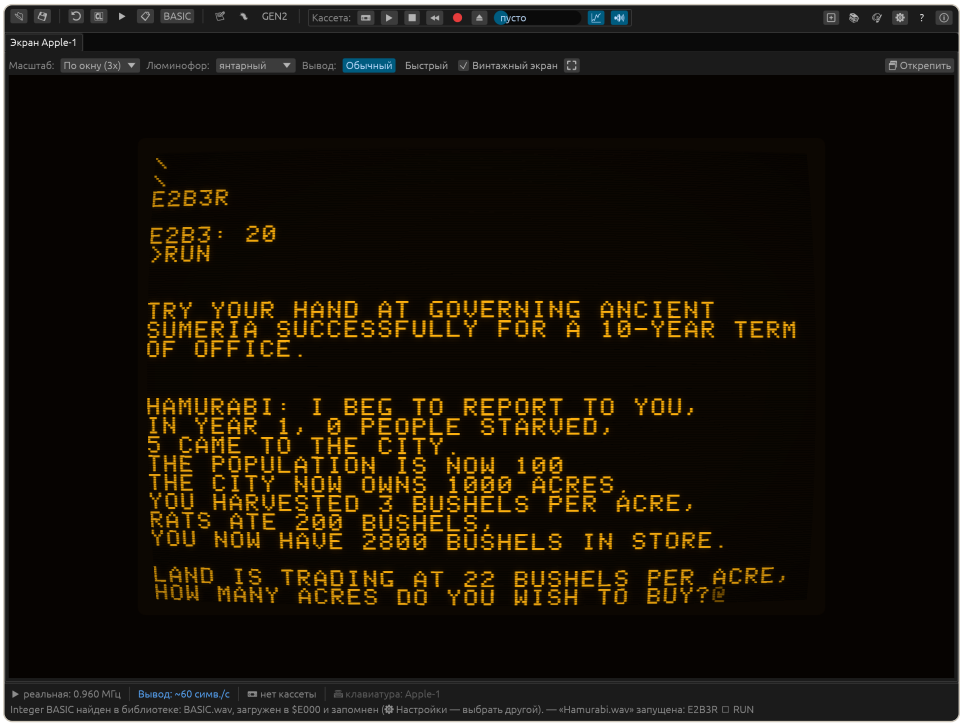


Рисунок 9 — Янтарный люминофор с винтажным экраном и лёгкой кривизной.

3.3 Винтажный экран

Флажок **Винтажный экран** в строке отображения имитирует ЭЛТ-монитор 1970-х. Главный эффект — тонкая тёмная линия между двумя строками развёртки: каждый ряд точек становится горизонтальной полосой, как на фотографии настоящего Apple-1. Эта линия сохраняет одинаковую толщину в пикселях вашего дисплея при любом масштабе: она остаётся тонкой и чёткой даже в полноэкранном режиме.

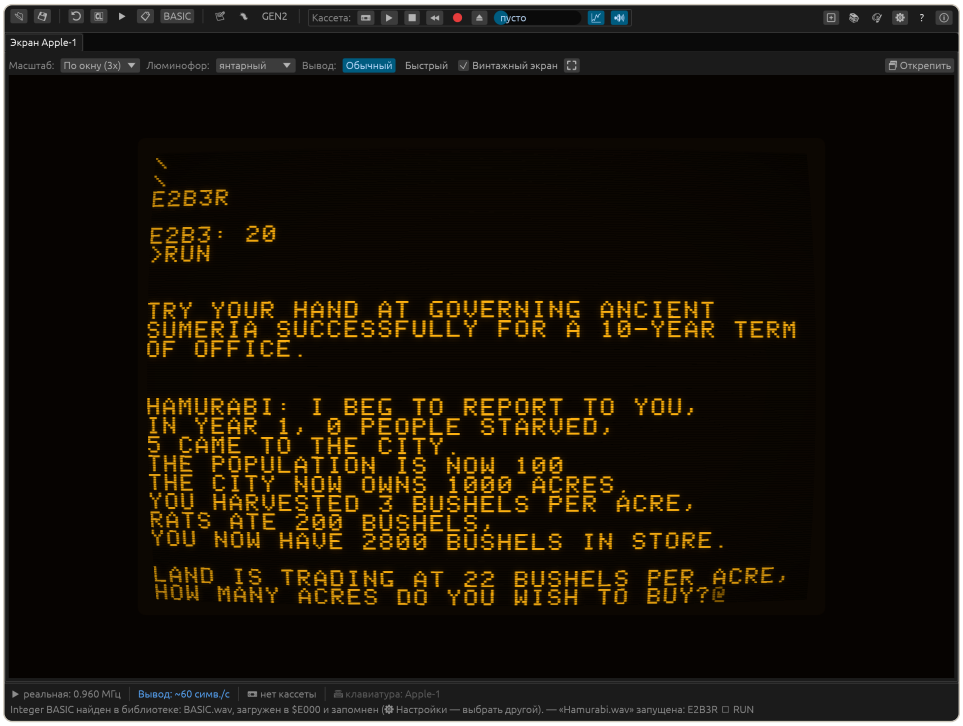


Рисунок 10 — Винтажный экран с лёгкой выпуклостью кинескопа (0,15) и тёмными углами, во время игры в Hamurabi.

В ⚙ **Настройки → Дополнительно → Винтажный экран: тонкие настройки** задаётся **Интенсивность** и выбираются эффекты; эти настройки общие для обоих экранов. Рядом с каждым ползунком показано его **числовое значение** (кривизна, интенсивность…):

Эффект	По умолчанию	Назначение
промежутки между строками развёртки	вкл.	тёмная линия между двумя рядами точек
смягчённые по горизонтали точки	вкл.	лёгкое размытие, только по горизонтали, как от луча
послесвечение люминофора	короткое	точки гаснут менее чем за десятую долю секунды (у янтарного чуть медленнее); крайнее левое положение — без него
свечение	вкл., слабое	ореол вокруг точек, которые остаются чёткими
выпуклый кинескоп и тёмные углы	выкл.	с ползунком **Кривизна**
мерцание	выкл.	очень лёгкое колебание яркости
RGB-маска люминофора	вкл.	только цветной экран карты GEN2: тонкие вертикальные красные, зелёные и синие полосы, разделённые узкими тёмными линиями, как на цветном телевизионном кинескопе. Видна начиная с масштаба 3, её сила следует за **интенсивностью**

Эффект можно применить и к цветному монитору карты GEN2 — флажком в его собственной строке отображения. Он вычисляется процессором в разрешении Apple-1 и только при изменении изображения: всё остаётся плавным даже без графического ускорения.

В командной строке: `--screening`, `--sans-screening`, `--gen2-screening` (монитор GEN2) или, например, `--screening-options intensite=0.7,remanence=0.5,courbure=0.4,sans-halo` (значения от 0 до 1; имена параметров французские).

> ✦ **ОТКРЕПЛЁННЫЙ ЭКРАН И ВТОРОЙ МОНИТОР**
>
> **Открепить** в конце строки отображения выносит экран Apple-1 в отдельное окно. Поместите его на второй монитор, в полноэкранный режим, с винтажным эффектом: клавиатура идёт в Apple-1, как только это окно получает фокус. С **По окну** изображение следует за размером окна — целыми шагами (**Целый масштаб**, в заголовке окна, самый чёткий вариант) или заполняя его.

ГЛАВА 4

Библиотека

Библиотека — отправная точка почти всего: она хранит ваши кассеты, дампы и исходники, узнаёт кассеты того времени и запускает программу одним щелчком, в том числе на BASIC.

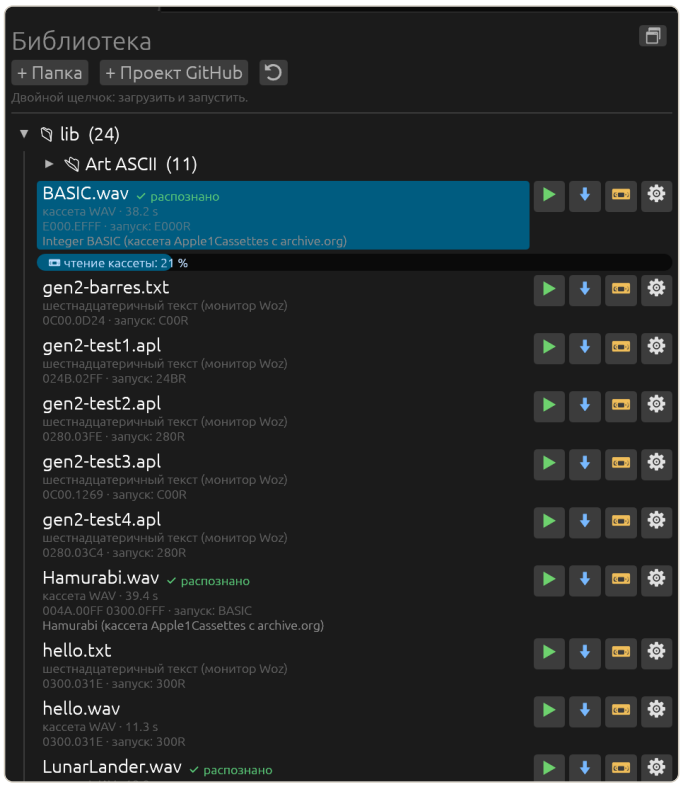


Рисунок 11 — Панель библиотеки: основная папка, папка «Art ASCII», распознанные кассеты и проект GitHub. Играет BASIC.wav.

4.1 Папки

Основная папка — `~/apple1/programmes`. Она создаётся при первом запуске, а примеры раскладываются по пяти папкам: **Art ASCII** (одиннадцать примеров ASCII-графики 40 × 24, глава 9), **BASIC** (программы `.bas` и `.hex.txt` для BASIC), **6502** (исходники `.s` / `.asm` с их `.bin` / `.hex`), **GEN2 COLOR** (тесты карты GEN2 от Bernie, `gen2-*.apl`, и цветные игры для G2 Basic) и **WAV** (все кассеты). На панели эти пять папок показаны прямо на верхнем уровне, в порядке **BASIC**, **6502**, **GEN2 COLOR**, **WAV**, **Art ASCII**: корневая папка (имя каталога на диске) над ними не отображается, а файлы, оставленные в ней россыпью, не перечисляются. Положите их в одну из пяти папок. Добавленная папка или проект GitHub сохраняют собственный заголовок. Уже существующая библиотека никогда не перестраивается, кроме

того, что её папки **Assembleur** и **Uncle Bernie** (названия версий с 7.8 по 7.9.1) переименовываются в **6502** и **GEN2 COLOR** с сохранением содержимого. Чтобы выбрать другую: ⚙ **Настройки → Папка библиотеки →** 📂 **Обзор…** (↺ **По умолчанию** возвращает `~/apple1/programmes`).

+ Папка вверху панели добавляет другие папки; основная папка остаётся всегда. Библиотека — это дерево: папки и подпапки сворачиваются, и их состояние запоминается. Каждая папка хранит свои настройки в файле `programs.json`. Правый щелчок по добавленной папке позволяет открыть её в файловом менеджере или убрать из списка (на диске ничего не удаляется).

Узкая панель. Библиотеку можно сделать сколь угодно узкой (до минимальной ширины, ниже которой разделитель останавливается). Каждая запись при этом подстраивается: имя занимает всю ширину и обрезается «…», если слишком длинное (полное имя — в подсказке), маленькие значки ▶ ⬇ 📼⚙ переходят на следующую строку, а описание ограничено одной-двумя строками, тоже с «…», полный текст — в подсказке. В заголовке **+ Папка** и **+ Проект GitHub** превращаются в значки с подсказкой.

Показываются файлы: `.wav .aiff .bin .hex .txt .apl .bas .s .asm .prg`. Список обновляется сам каждые две секунды: скопируйте файл в папку, и он появится.

Для каждой программы список показывает имя, тип, области адресов, длительность кассеты, команду запуска и ваши заметки.

Игры Claude IA

В папки **BASIC** и **6502** также попадают пять небольших игр, написанных Claude IA, каждая во французской и английской версиях: *Виселица* (`pendu.bas`, `hangman.bas`), *Спички* (`allumettes.bas`, `matches.bas`) и *Морской бой* (`bataille.bas`, `battleship.bas`) на BASIC; *Тунель* (`tunnel.s`, `tunnel-en.s`) и *Крестики-нолики* (`morpion.s`, `tictactoe.s`) на ассемблере 6502. Двойной щелчок по игре на BASIC запускает её на RDK G2 Basic (эмулятор запускает BASIC, загружает листинг и набирает `RUN`); исходники на ассемблере собираются встроенным ассемблером. Они подключают `apple1.inc`, который устанавливается вместе с ними. В папке **GEN2 COLOR** есть также *Четыре в ряд в цвете* (`puissance4-couleur.bas`) — игра для G2 Basic и карты GEN2 от Claude IA. Q и S двигают фишку над сеткой, пробел бросает её. Игры входят в проект и действуют на условиях его лицензии (приложение «Лицензия и благодарности»).

4.2 Запуск программы

У каждой программы справа четыре маленьких значка:

Значок	Действие
▶	**Загрузить и запустить**: при необходимости загружает программу, затем запускает её: RESET, затем её команда, набранная на клавиатуре (`300R` или `E2B3R` и `RUN` для программы на BASIC). Если программе нужен Integer BASIC, а его нет в `$E000`, эмулятор сначала загружает его. Листинг BASIC набирается в Integer BASIC, исходник 6502 ассемблируется.
⬇	**Загрузить**: помещает программу в память мгновенно, не запуская её.
📼	**Через кассету**: как на настоящей машине. Эмулятор нажимает RESET, набирает `C100R`, затем команду чтения ACI (например, `0300.031ER`), и кассета играет с реальной скоростью, со звуком и полосой прогресса. Файлы `.txt` и `.bin` сначала преобразуются в кассету. Неактивен для исходника (`.bas`, `.s`), у которого нет записи.
⚙	**Параметры**: области, адрес загрузки `.bin`, адрес и команда запуска, тип, нужен ли BASIC, заметки.

Загрузка **мгновенная**, если вы не выбрали кассету значком 📼. **Двойной щелчок** по программе загружает её и запускает, если у неё есть команда запуска (как ▶). Кассета используется, только когда вы об этом просите; тогда магнитофон на панели инструментов показывает кассету и её позицию.

Когда редактор открыт, ▶, ⬇ или двойной щелчок по листингу BASIC или исходнику 6502 к тому же **открывают его во вкладке редактора**, помеченной 📚; если он уже открыт, просто показывается его вкладка.

Правый щелчок по программе предлагает остальное: загрузить, загрузить и запустить, вставить как кассету без набора команд, параметры, переименовать, удалить (с подтверждением), открыть папку.

4.3 Параметры программы

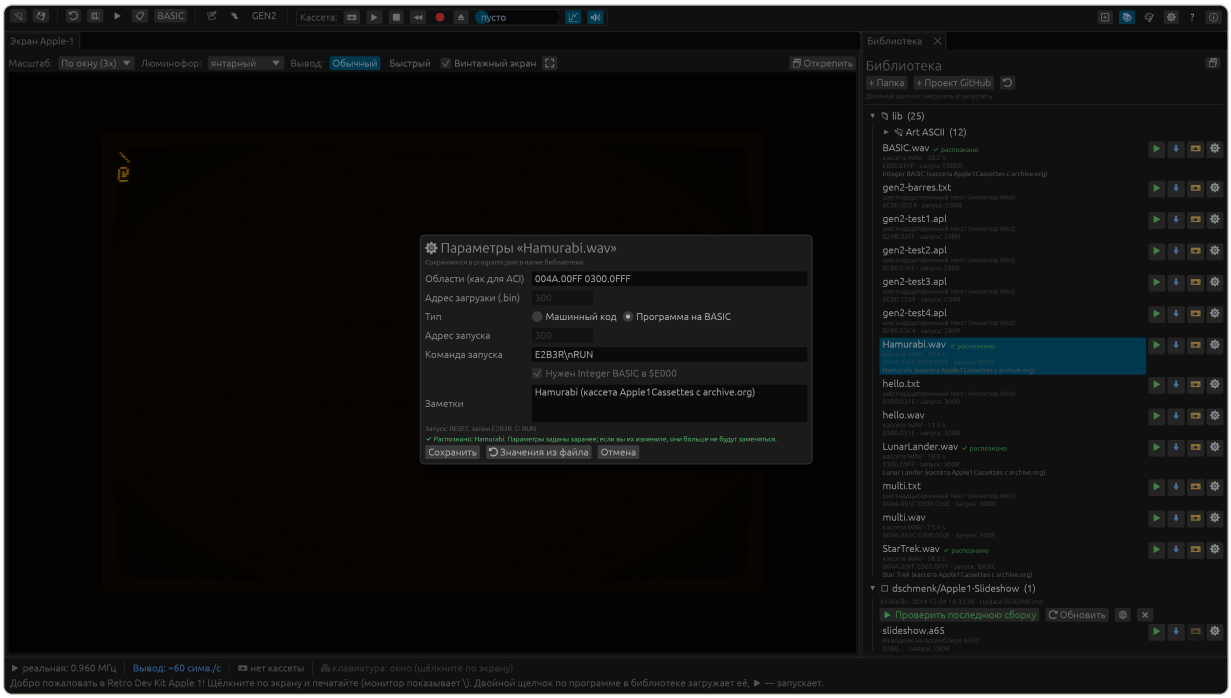


Рисунок 12 — Параметры ⚙ Hamurabi: две области (нулевая страница BASIC, затем программа), запуск через E2B3R, затем RUN.

Поле	Назначение
Области	Участки памяти, записанные как для ACI: `0300.031E` или `004A.00FF 0800.0FFF` для программы на BASIC. Каждая запись на кассете по порядку попадает в свою область.
Адрес загрузки	Для `.bin`, который не содержит адреса.
Запуск	Адрес и команда, которую набирает ▶. `\n` обозначает RETURN между двумя строками: `E2B3R\nRUN`.
Тип	**Машинный код**, **Программа G2 Basic** (выбор по умолчанию для листинга BASIC, так как эмулятор его содержит) или **Программа Integer BASIC** (для тех, кто предпочитает оригинал Apple: ▶ тогда загружает оригинальный Integer BASIC, выбранный в настройках). Тип, выбранный для программы, главнее общего параметра BASIC в настройках.
Нужен BASIC	▶ сначала загрузит BASIC того типа, который указан у программы.
Заметки	Свободный текст, показываемый в списке.

Значения заранее заполняются по файлу и проверяются при сохранении; ⟲ **Значения из файла** читает их заново.

> **! ВСЕГДА ЧЕТЫРЕ ЦИФРЫ: 004A.00FF, НИКОГДА 4A.FF**
>
> Параметры отвергают `4A.FF`. Это не каприз: ПЗУ ACI не очищает свой шестнадцатеричный накопитель на точке, так что цифры продолжают сдвигаться, и она прочитала бы `$004A-$4AFF` (см. раздел 5.6). Поэтому эмулятор всегда пишет адреса четырьмя цифрами.

4.4 Распознанные кассеты

Когда файл **распознан**, его параметры заполняются сами, а список показывает ↻ **распознано**. Эмулятор декодирует записи и сравнивает их размеры и отпечатки SHA-256 со встроенной таблицей: 24 кассеты коллекции «Apple1Cassettes» на archive.org. Имя файла тогда совершенно не важно. Иначе он узнаёт **имя** (без учёта регистра, пробелов и дефисов) при тех же размерах записей; подсказка тогда сообщает, что содержимое отличается.

Кассета	Области	Запуск
`BASIC.wav` (Integer BASIC)	`E000.EFFF`	`E000R`
`Hamurabi.wav`, `StarTrek.wav`, `Wumpus.wav`, `Slots.wav`, `Craps.wav`, `Bowling.wav`, `Buzzword.wav`, `WordCross.wav`, `AppleiEnhancedCheckers.wav`, `LunarlanderasciiGraphics.wav`	`004A.00FF 0300.0FFF`	`E2B3R` ⏎ `RUN`
`AceyDucey.wav`, `Blackjack.wav`, `HiLo.wav`	`004A.00FF 0800.0FFF`	`E2B3R` ⏎ `RUN`
`Football.wav`	`004A.00FF 0400.2FFF`	`E2B3R` ⏎ `RUN` (12 КБ)
`DealOrNoDeal.wav`, `StarTrek2003.wav`	`004A.00FF 0300.3FFF`	`E2B3R` ⏎ `RUN` (16 КБ)
`APPLE30TH.wav`	`0280.0FFF`	`280R`
`ASMmchess.wav`	`0300.0BFF`	`300R`
`Microchess.wav`	`1000.18FF`	`1000R`
`LunarLander.wav`	`0300.09FF`	`300R`
`Mastermind.wav`	`0300.03FF`	`300R`
`Pasart.wav`	`0300.05FF`	`300R`
`Life.wav`	`2000.21FF`	`2000R` (12 КБ)

Ваши собственные параметры никогда не перезаписываются: программа, которую вы настроили сами, сохраняет свои параметры, появляется лишь отметка «распознано». Программы, выходящие за базовые 8 КБ, при запуске автоматически получают нужное ОЗУ, как с платой расширения.

Неизвестная кассета. Эмулятор угадывает, что может: запись длиной 4096 байт, начинающаяся с `JMP`, — это Integer BASIC; две записи, первая из которых длиной 182 байта (`$4A-$FF`), — программа на BASIC, размещаемая по её указателям LOMEM и HIMEM. Список тогда показывает «(предположительно)»; проверьте через ⚙. Программа в машинных кодах без адресов не угадывается: укажите её область.

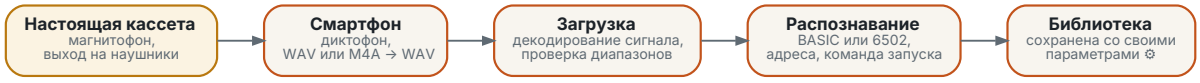


Рисунок 13 — Оцифровка настоящей кассеты: от магнитофона к файлу, затем в библиотеку.

4.5 Проекты GitHub

+ Проект GitHub принимает URL (`https://github.com/jefftranter/6502/tree/master/asm/jmon`) или просто `владелец/репозиторий`. Нужные файлы копируются в `~/.cache/apple1-emu/github/<владелец>/<репозиторий>/<коммит>/`, а проект становится узлом дерева, с коммитом, датой и сообщением. ⟳ **Обновить** получает последний коммит; 🌐 открывает страницу репозитория. Вся работа с сетью идёт в фоне.

Проверить последнюю сборку пробует по порядку:

1. файлы последнего **релиза** репозитория (`.wav` / `.aiff` , затем `.bin` , затем `.hex` / `.txt`);
2. последний **артефакт GitHub Actions** (нужен токен);
3. **сборку из исходников**: листинг BASIC набирается в Integer BASIC и запускается через `RUN` ; исходник 6502 ассемблируется **ca65/ld65** (пакет `cc65`), по умолчанию с `-D APPLE1` ; дамп Woz загружается.



Рисунок 14 — Проект GitHub: от адреса репозитория до работающей программы.

Файл `apple1.toml` (или `apple1.cfg`) в корне репозитория указывает, что собирать:

apple1.toml

```toml
file    = "src/game.s"    # программа для запуска
load    = $0300           # адрес загрузки (иначе .org исходника)
run     = $0300           # адрес запуска (иначе адрес загрузки)
ld65    = "apple1.cfg"    # конфигурация ld65 (иначе -t none)
defines = "APPLE1"        # символы -D для ca65
```

✦ ТОКЕН GITHUB НЕОБЯЗАТЕЛЕН

Публичным репозиториям он не нужен. Токен (переменная `GITHUB_TOKEN` или ⚙ **Настройки**) повышает лимит запросов и даёт доступ к вашим приватным репозиториям и к артефактам. Он никогда не показывается и не пишется в сообщения, а `gui.json` тогда становится доступен для чтения только вашей учётной записи.

4.6 Папка «Art ASCII»

В библиотеке есть и папка **Art ASCII**: рисунки в формате экрана (яблоко, плата Apple-1, микросхема 6502, кассета, ракета…) и портреты Стива Возняка и Стива Джобса, преобразованные из фотографий со свободной лицензией. ASCII-арт `.txt` при выделении показывает превью с кнопками → **BASIC**, → **6502** и 🎨 **Открыть в инструменте**; ▶ сразу запускает его версию для 6502. Инструмент описан в главе 9.

ГЛАВА 5

Кассета и карта ACI

У Apple-1 не было ни дисковода, ни BASIC в ПЗУ: всё проходило через звуковую кассету и карту ACI (Apple Cassette Interface). Эмулятор выполняет настоящее ПЗУ ACI, поэтому его файлы WAV и AIFF загружаются на настоящем Apple-1, а записи, сделанные на настоящем Apple-1, загружаются в эмуляторе.

5.1 Форматы файлов

Расширение	Содержимое	Чтение	Запись
`.wav`	кассета ACI, PCM от 8 до 32 бит или float, моно или стерео, любая частота дискретизации	да	по умолчанию 44,1 кГц моно 16 бит
`.aif` , `.aiff`	кассета ACI (AIFF и AIFF-C)	да	профиль для смартфона: 48 кГц, 2 одинаковых канала, 16 бит
`.txt` , `.hex` , `.apl`	дамп Woz Monitor: строки `0300: A9 C1 20 …`	да	да
`.bin`	«сырые» байты без адреса (`FILE.bin@300`)	да	да
`.bas`	листинг Integer BASIC (текст)	в редакторе	в редакторе
`.s` , `.asm` , `.a65`	исходник 6502 (ca65 или xa)	в редакторе	в редакторе

Кассеты, экспортированные эмулятором, несут свои адреса в блоке комментария файла (`APPLE1 ACI 0300.031E`), который аудиоплееры игнорируют. У кассеты из другого источника адресов нет: библиотека распознаёт или угадывает их (глава 4), а командная строка принимает `--addr` или `--range` .

Декодер терпит шум, низкий уровень, инвертированную полярность, ошибку скорости ленты ±10 % и несколько записей в одном файле. Файлы WAV, созданные утилитой **c2t** Игана Форда, тоже читаются. **Сетевой фон** (50 или 60 Гц, даже такой же сильный, как сам сигнал кассеты) отфильтровывается до поиска фронтов, поэтому длинная запись, сделанная плохо экранированным кабелем, всё равно читается правильно.

5.2 Виртуальный магнитофон

Участок **Кассета:** панели инструментов собирает все органы управления кассетой: 📼 **Вставить**, ▶ воспроизведение, ■ стоп, ⏪ перемотка, ● запись, ⏏ извлечение, затем имя кассеты и позиция, осциллограф 📈 и звук 🔊. Кассету вставляют кнопкой 📼 **Вставить** (`.wav` или `.aiff`) или правым щелчком по программе в библиотеке («Вставить как кассету»): тогда `C100R` и область вы набираете сами. Значок 📼 программы в библиотеке идёт на шаг дальше: он вставляет кассету и набирает команды за вас.



Рисунок 15 — Участок «Кассета:» во время чтения BASIC.wav.

Кассета идёт по **эмулируемому** времени: на быстрой скорости (F3) загрузка занимает гораздо меньше 10-секундного заголовка — и всё равно работает. Без кассеты загрузка из библиотеки мгновенная (глава 4).

Загрузка Integer BASIC «по-старому» выглядит так:

```
WOZ MONITOR
C100R
C100: A9*
E000.EFFFR
E000R

E000: 4C
>
```

`C100R` запускает программу ACI, которая отвечает приглашением `*`. Строка `E000.EFFFR` просит её **прочитать** кассету в `$E000-$EFFF`. Когда кассета кончается, ACI возвращается в монитор (`\`); `E000R` запускает BASIC, чьё приглашение — `>`.

5.3 Звук и осциллограф

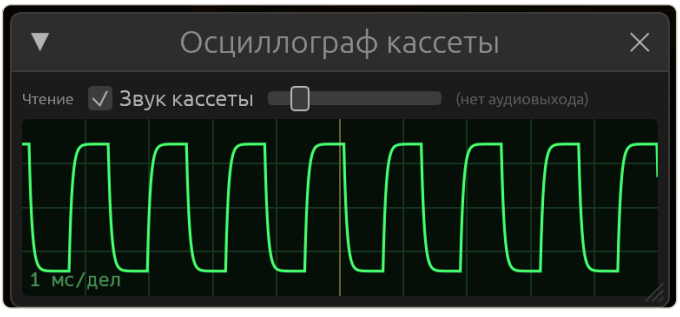


Рисунок 16 — Осциллограф во время воспроизведения BASIC.wav: флажок «Звук кассеты», громкость и сигнал при 1 мс на деление.

Во время воспроизведения или записи 🔊 позволяет слышать сигнал, как из динамика магнитофона, по умолчанию негромко. 📈 открывает **осциллограф**: зелёная линия на тёмной сетке, 1 мс на деление, показывающая 10 мс вокруг головки воспроизведения или последние 10 миллисекунд, записанные Apple-1. Можно различить заголовок (≈ 800 Гц), биты 0 (≈ 2 кГц) и биты 1 (≈ 1 кГц). На быстрой скорости звук отключается.

5.4 Запись с эмулируемого Apple-1

Чтобы сделать кассету как в 1976 году, приведите запись в готовность, затем попросите ACI записать:

1. Нажмите ● и выберите файл (`.wav` или `.aiff`).
2. Наберите `C100R` , затем область с `W` в конце, например `0300.03FFW` , затем RETURN .
3. ACI пишет 10-секундный заголовок, затем данные. Файл записывается через 1,5 с после окончания или по ■.

Проще всего всё же кнопка **Сохранить** на панели инструментов или 📼 **Экспорт WAV** в редакторе: сигнал тот же, такт в такт, и создаётся мгновенно.

5.5 Как работает ACI

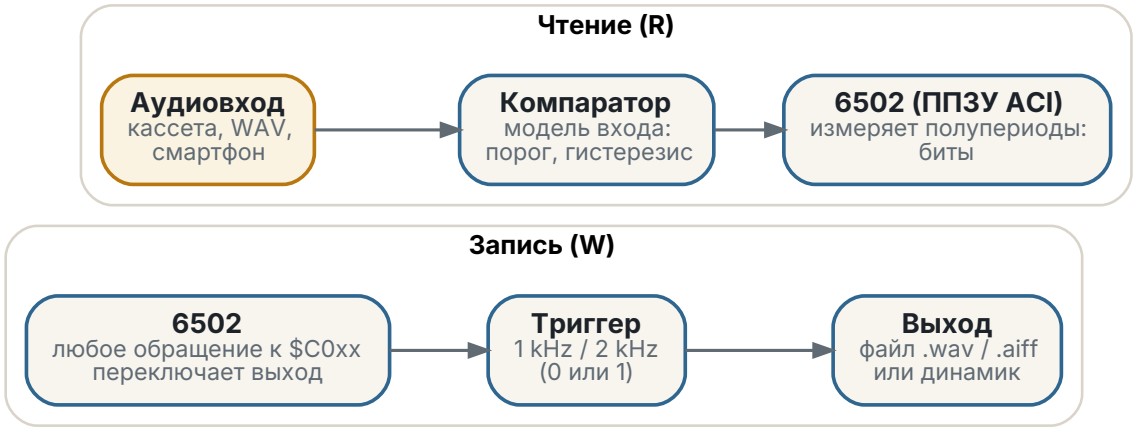


Рисунок 17 — Два направления ACI: при записи любое обращение к $C0xx переключает выход; при чтении компаратор управляет адресной линией A0 ППЗУ.

Карта удивительно проста: ППЗУ на 256 байт, видимое по адресам `$C000` и `$C100` , триггер и компаратор.

- **Запись.** Каждое обращение по адресу из `$C000-$C0FF` переключает выход (74LS74). Программа ПЗУ задаёт время между двумя обращениями: это время и кодирует биты.
- **Чтение.** Уровень на входе кассеты после компаратора LM311 управляет адресной линией A0 ППЗУ, когда A7 = 1. Одна и та же инструкция поэтому читает `$C080` или `$C081` в зависимости от уровня: ПЗУ измеряет полупериоды циклом в 12 тактов.

- **Кодирование.** «1» длится примерно вдвое дольше «0»: 1 кГц для единицы, 2 кГц для нуля. Байты выводятся старшим битом вперёд, без чётности и без контрольной суммы.

Точные времена в тактах и микросекундах приведены в приложении C.

5.6 Команды ACI

После `C100R` ACI ждёт строку в формате монитора, оканчивающуюся на `R` (чтение) или `W` (запись):

Строка	Действие
`0300.03FFR`	читает кассету в `$0300-$03FF`
`0300.03FFW`	записывает `$0300-$03FF` на кассету, с заголовком 10 с
`004A.00FFR 0300.0FFFR`	читает **две** записи подряд: нулевую страницу BASIC, затем программу
`004A.00FFW800.FFFW`	слитная форма из руководства по BASIC 1976 года, тоже принимается

> **! НУЛИ В 004A.00FF ОБЯЗАТЕЛЬНЫ**
>
> Разборщик ПЗУ не очищает свой шестнадцатеричный накопитель на точке («Doesn't clear HEX1!!!», говорит листинг). Цифры конца вдвигаются вслед за цифрами начала: `4A.FF` означает `$004A-$4AFF`, почти 19 КБ! `0300.0FFF` работает, потому что четыре цифры выталкивают тройку из `0300`. Всегда пишите оба адреса четырьмя цифрами. Эмулятор воспроизводит это поведение, так как выполняет настоящее ПЗУ.

Программа на BASIC сохраняется двумя записями: нулевая страница `$004A-$00FF` (там указатели LOMEM и HIMEM и конец программы), затем сама программа, которая всегда заканчивается на HIMEM. Чтобы загрузить её обратно, BASIC уже должен быть в памяти; затем в BASIC возвращаются «тёплым» стартом `E2B3R`, который сохраняет программу.

WOZ MONITOR

```
C100R
C100: A9*
004A.00FFR 0300.0FFFR
E2B3R

E2B3: 20
>RUN
```

5.7 На настоящий Apple-1

Именно ради этого и существует эмулятор: подготовить программу на ПК, затем воспроизвести её в ACI настоящей машины со смартфона или с выхода на наушники компьютера.

1. **Подготовьте файл.** 📼 **Экспорт WAV** в редакторе или `apple1-emu export prog.txt -o prog.aiff`. Для смартфона выбирайте `.aiff`. Запомните показанную строку, например `C100R`, затем `0300.031ER`.

2. **Настройте проигрыватель.** Громкость на максимум (примите предупреждение о «высокой громкости»), все звуковые эффекты выключены (эквалайзер, Dolby, «улучшение звука»), режим полёта, без Bluetooth. Подойдёт простой проигрыватель вроде VLC.
3. **На Apple-1** наберите `C100R` : монитор ответит `C100: A9*` .
4. Наберите строку чтения, например `0300.031ER` , но **пока не нажимайте RETURN**.
5. Запустите воспроизведение и нажмите RETURN **во время заголовка** (он длится около 10 с после секунды тишины).
6. В конце ACI возвращается в монитор (`\`). Запустите программу: `300R` или `E2B3R` , затем `RUN` для программы на BASIC.

Почему такой профиль AIFF? Несжатые 16 бит, без кодека с потерями: MP3 или AAC скруглили бы фронты. 48 кГц — родная частота Android, чтобы избежать любой передискретизации. Два строго одинаковых канала, потому что кабель, соединяющий левый и правый на FROM TAPE, никогда не должен получать два разных сигнала. Меандр на −1 dBFS: вход оригинальной ACI — это фильтр верхних частот (0,01 мкФ), который видит почти только фронты, и меандр с крутыми фронтами подходит ему лучше всего. Секунда тишины в начале и в конце, потому что телефон иногда обрезает начало звука.

> ▣ **НЕ ДОВЕРЯЙТЕ СВЕТОДИОДУ**
>
> Руководство 1976 года просит уровень, при котором светодиод ACI «едва горит». Но он загорается лишь примерно при 1,2 В — гораздо больше, чем нужно компаратору. Негорящий светодиод поэтому не доказывает, что сигнал слишком слаб. Если загрузка никак не кончается, ПЗУ всё ещё ждёт фронтов: нажмите RESET и прибавьте громкость.

> ! **НИКОГДА НЕ MP3 И НЕ AAC**
>
> Формат с потерями сдвигает фронты и делает кассету нечитаемой. Используйте WAV, AIFF или FLAC, если ваш проигрыватель его декодирует.

5.8 Какой нужен уровень? Модели входа

Команда `info --aci-input` моделирует аналоговый вход карты (разделительный конденсатор, смещение, гистерезис LM311) для ряда пиковых напряжений и сообщает, с какого уровня файл будет прочитан:

TERMINAL

```
apple1-emu info prog.aiff --aci-input stock     # карта 1976 года, 0,01 мкФ
apple1-emu info prog.aiff --aci-input mimeo     # 0,1 мкФ (совет руководства Mimeo)
apple1-emu info prog.aiff --aci-input gen2      # ACI Gen2 Uncle Bernie
apple1-emu info prog.aiff --aci-input stock --volts 0.5 --series 1200
```

Модель	Что представляет
`stock`	оригинальная ACI: 0,01 мкФ на компаратор со смещением 2,5 В, обратная связь 3k/47k
`mimeo`	доработка Майка Уиллегала: 0,1 мкФ, надёжнее
`gen2`	ACI Gen2 Uncle Bernie: согласованные 100 нФ, 300k вместо 47k (меньше гистерезис)

`--volts` задаёт пиковое напряжение на гнезде при полной цифровой шкале; `--series` добавляет последовательное сопротивление (например, 1,2 кОм кабеля смартфона). С моделью `stock` файлы эмулятора читаются **примерно от 0,25 В пикового значения**; выход на наушники смартфона на максимуме обычно даёт от 0,5 до 1 В. Модель упрощена: без шума и без дрейфа рабочей точки.

5.9 Оцифровка настоящей кассеты или записи Apple-1

Чтобы спасти программу, набранную на настоящем Apple-1, запишите выход ACI **TO TAPE** через микрофонный вход (телефон или звуковая карта ПК с Audacity на 44,1 или 48 кГц), выключив автоматическую регулировку усиления и шумоподавление. На Apple-1: `C100R`, затем `0300.03FFW` и RETURN. Остановите запись через несколько секунд после возвращения `\`, затем:

TERMINAL

```
apple1-emu import recording.aiff --addr 300 -o prog.txt
apple1-emu import startrek.aiff --range "004A.00FFR 0300.0FFFR" -o startrek.txt
```

> ▣ **TO TAPE — ВЫХОД МИКРОФОННОГО УРОВНЯ**
>
> Он даёт лишь около 50 мВ от пика до пика. Проверьте форму сигнала в Audacity: без ограничения, без скачков уровня во время заголовка. Импорт эмулятора терпит любое усиление и постоянное смещение.

ГЛАВА 6

Графические карты: GEN2 и P-LAB

В 2026 году Uncle Bernie разработал для Apple-1 цветную графическую карту «GEN2», которая приносит ему режимы отображения Apple II. Эмулятор моделирует её по желанию: её программные переключатели, память, видеоразвёртку и цветной экран. В конце главы описана и графическая карта P-LAB на TMS9918A.

6.1 Подключение карты

Карта **по умолчанию отключена**. Подключить её можно тремя способами:

- **GEN2** в группе «Графические карты» на панели инструментов: карта сразу подключается или отключается; работающая программа продолжает выполняться (нажмите RESET, если она использовала карту);
- ⚙ **Настройки → GEN2 → Цветная карта Uncle Bernie GEN2**, с выбором 60 или 50 Гц;
- в командной строке: `apple1-emu-gui --carte-couleur` (`--couleur-50hz`, `--sans-carte-couleur`).

На настоящей машине у карты свой видеовыход для второго монитора, цветного; терминал Apple-1 остаётся на своём исходном выходе и работает как прежде. Поэтому окно показывает **два экрана**: экран Apple-1 и **цветной монитор** под ним или рядом. Подпись показывает текущий режим. У цветного монитора своя строка отображения: масштаб, вывод **Обычный** / **Быстрый** (общий с экраном Apple-1), собственный флажок **Винтажный экран**, полный экран и **Открепить**, чтобы вынести его на второй монитор.

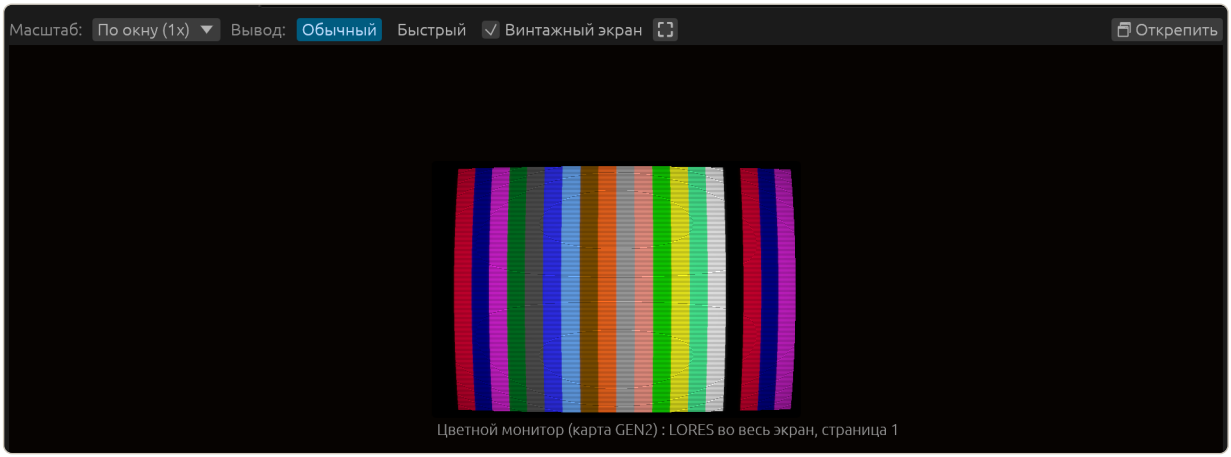


Рисунок 18 — Цветной монитор: полосы LORES примера gen2-lores.s и подпись режима.

> ▣ **КАРТА — ЭТО ЕЩЁ И РАСШИРЕНИЕ ПАМЯТИ**
>
> С GEN2 ОЗУ занимает `$0000-$BFFF` (плюс `$E000-$EFFF`). Графические страницы находятся в этой памяти: программа не должна туда загружаться.

6.2 Программные переключатели

Карта воспроизводит программные переключатели Apple II `$C050-$C057`, перенесённые в `$C250-$C257`. Они реагируют **только на чтение**: запись не действует. На ассемблере их поэтому включают через `BIT` или `LDA`, но никогда через `STA`.

Адрес	Имя (gen2.inc)	Действие
`$C250`	`SS_GR`	графический режим
`$C251`	`SS_TEXT`	текстовый режим
`$C252`	`SS_FULL`	полный экран
`$C253`	`SS_MIX`	смешанный: 4 строки текста внизу
`$C254`	`SS_PAGE1`	страница 1
`$C255`	`SS_PAGE2`	страница 2
`$C256`	`SS_LORES`	низкое разрешение
`$C257`	`SS_HIRES`	высокое разрешение

Бит 7 прочитанного значения возвращает **HSTo**: 1 во время горизонтального и вертикального обратного хода, 0 во время видимой части. Программа может использовать его для синхронизации с развёрткой (так делает TEST4). Биты с 0 по 6 «плавают». Карта также отвечает на своих зеркалах (`$C258`, `$C3F1`…) и оставляет `$C0xx` и `$C1xx` свободными для ACI. RESET не трогает переключатели; эмулятор стартует в текстовом режиме, страница 1.

В Integer BASIC тоже достаточно чтения: `X=PEEK(-15792)` читает `$C250` (−15792 = `$C250` − 65536).

6.3 Режимы и память

Режим	Разрешение	Страница 1	Страница 2
TEXT	40 × 24 символа	`$0400-$07FF`	`$0800-$0BFF`
LORES	40 × 48 блоков, 16 цветов	`$0400-$07FF`	`$0800-$0BFF`
HIRES	280 × 192 точки, 6 цветов	`$2000-$3FFF`	`$4000-$5FFF`
MIXED	графика, последние 4 строки — текст		

Адреса строк. Как на Apple II, строки в памяти идут не подряд. Текстовая строка *r* (от 0 до 23) начинается с:

```
$0400 + (r & 7) × $80 + (r >> 3) × $28
```

Каждый байт страницы LORES хранит **два** блока: младшая тетрада — верхний блок, старшая — нижний.

В HIRES каждая строка развёртки y (от 0 до 191) — это 40 байтов по 7 точек (бит 0 слева, бит 7 выбирает палитру), и она начинается с:

```
$2000 + (y & 7) × $400 + ((y >> 3) & 7) × $80 + (y >> 6) × $28
```

Цвета LORES (в порядке значений от 0 до 15):

0	1	2	3	4	5	6	7
чёрный	пурпурный	тёмно-синий	фиолетовый	тёмно-зелёный	серый	средне-синий	голубой
8	**9**	**10**	**11**	**12**	**13**	**14**	**15**
коричневый	оранжевый	серый	розовый	светло-зелёный	жёлтый	аквамарин	белый

Цвета HIRES (артефакты NTSC): при сброшенном бите 7 — фиолетовый на чётных столбцах и зелёный на нечётных; при установленном бите 7 — синий и оранжевый; две соседние горящие точки дают белый. Так, чередование `$55` / `$2A` даёт фиолетовый, `$2A` / `$55` — зелёный, `$D5` / `$AA` — синий.

Символы текстового режима: коды `$00-$3F` — инверсные, `$40-$7F` — мигающие, `$80-$FF` — обычные; `$E0-$FE` — строчные буквы, а `$FF` — шахматное поле.

6.4 Тестовые программы

Папка `exemples/` (и библиотека, когда она создаётся с нуля) содержит демонстрацию и четыре теста Uncle Bernie:

Файл	Запуск	Содержимое
`gen2-barres.txt`	`C00R`	16 цветов LORES в виде полос плюс строка текста в смешанном режиме
`gen2-test1.apl`	`24BR`	TEST1: проверка ОЗУ карты, с полосами HIRES
`gen2-test2.apl`	`280R`	TEST2: диагональ LORES, затем смешанный режим и «пишущая машинка» (клавиша — старт, ESC — выход)
`gen2-test3.apl`	`C00R`	TEST3: 256 кодов символов, затем полноэкранный редактор
`gen2-test4.apl`	`280R`	TEST4: измерение HST0, затем текстовое окно в 4 строки, прокручиваемое поверх LORES

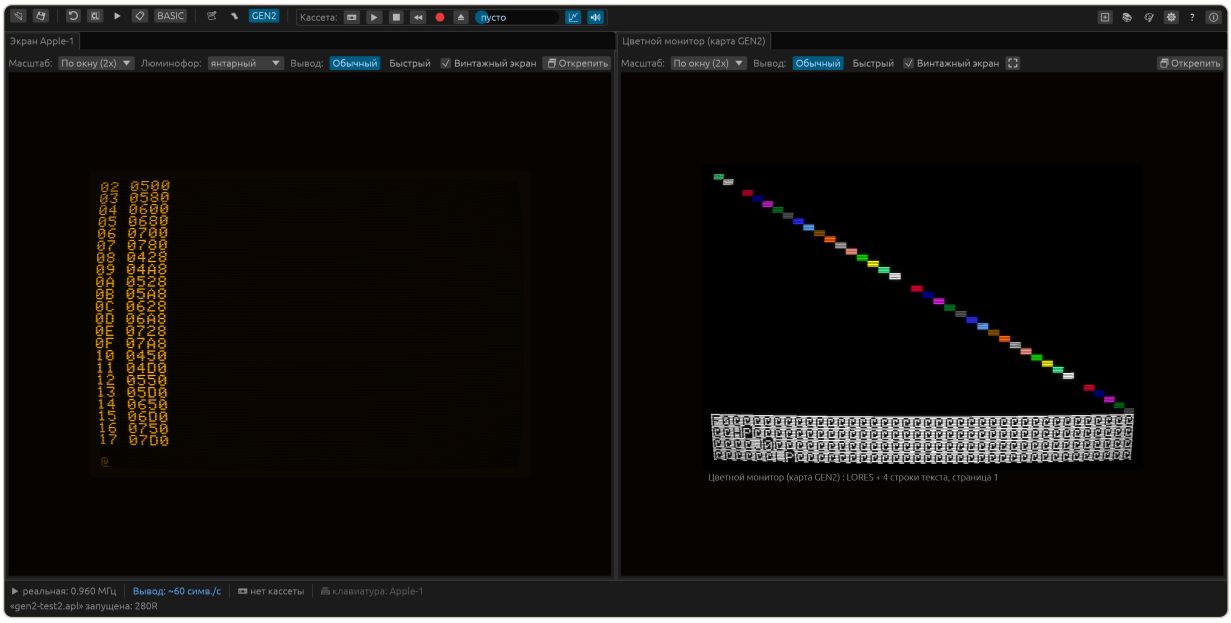


Рисунок 19 — TEST2 Uncle Bernie: диагональ LORES на цветном мониторе.

Исходники `gen2-lores.s` и `gen2-hires.s` из части III (глава 23) показывают, как самому писать в графические страницы.

6.5 Что приближённо

Точные оттенки зависят от NTSC, от монитора и от подстроечного конденсатора карты: эмулятор использует значения, близкие к Apple IIgs, в документированном порядке. Глифы строчных букв нарисованы вручную (ППЗУ карты не опубликовано). Сдвиг на полточки в HIRES при установленном бите 7 не учитывается. Смена режима действует со следующей строки развёртки. При включении память обнулена, что даёт экран из инверсных `@`, пока программа его не заполнит. Карта GEN1 2023 года (по адресу `$C050`) не эмулируется.

6.6 Графическая карта P-LAB

«Apple-1 Graphic Card» от P-LAB даёт Apple-1 настоящую видеомикросхему — **TMS9918A** *от Texas Instruments (ту же, что в ColecoVision, MSX и TI-99/4A): 256 × 192 точки, 16 цветов, 32 спрайта и 16 КБ видеопамяти. Эмулятор воспроизводит её по желанию, рядом с* GEN2.

Подключение карты

Карта **по умолчанию отключена**. Щёлкните **P-LAB** в группе «Графические карты» панели инструментов (кнопка загорится), или выберите **Подключить карту P-LAB** в кнопке **Цветной экран**, когда ни одна карта не подключена, или запустите `apple1-emu-gui --plab` (`--sans-plab`, чтобы убрать её). **Экран P-LAB** появляется как панель (кнопка **Цветной экран** группы «Окна» показывает или скрывает его); как любую панель, его можно закрепить, сделать вкладкой или вынести в отдельное окно.

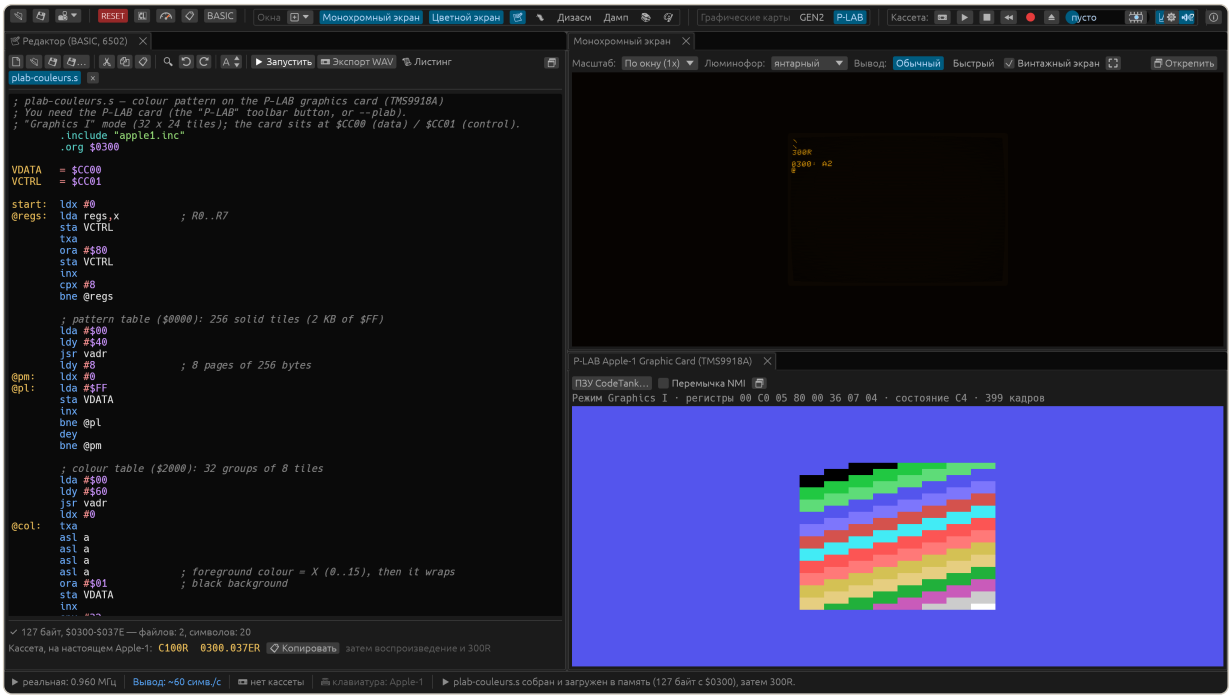


Рисунок 20 — Экран P-LAB под терминалом Apple-1 с цветным узором из примера plab-couleurs.s. Строка под заголовком панели показывает режим отображения, восемь регистров и состояние.

Терминал Apple-1 продолжает работать как обычно. Карта занимает адреса `$CC00-$CFFF`: `$CC00` — **порт данных** (чтение и запись видеопамяти), а `$CC01` — **порт управления** (видеоадрес и регистры при записи, состояние при чтении). Эмулятор декодирует в окне `$CC00-$CFFF` только адресную линию A0, так что пара повторяется по всему килобайту; руководство карты этого не оговаривает, это предположение (см. конец раздела).

У панели два элемента управления и одна информационная строка:

- **ПЗУ CodeTank...** вставляет образ 16 КБ в дочернюю плату CodeTank: он тогда читается по адресам `$4000-$7FFF`, только для чтения, поверх ОЗУ. Наберите `4000R` на терминале, чтобы запустить его. **Извлечь ПЗУ** убирает его;
- **Перемычка NMI** — это перемычка IRQ/NMI карты. Снятая (обычная настройка), прерывание конца кадра TMS9918A (60 раз в секунду) приходит на 6502 как IRQ; установленная — как NMI. Прерывание происходит, только если программа его разрешила (бит 5 регистра 1);
- **информационная строка** показывает режим (Graphics I, Graphics II, Text, Multicolor), регистры R0–R7 в шестнадцатеричном виде, регистр состояния и число нарисованных кадров.

Программирование

К TMS9918A обращаются только через два порта. Чтобы записать видеопамять, отправьте адрес в порт управления (младший байт, затем старший байт **ИЛИ** `$40`), затем записывайте байты в порт данных: адрес увеличивается сам. Чтобы записать регистр, отправьте в порт управления значение, затем номер регистра **ИЛИ** `$80`. Чтение регистра состояния (бит 7 = флаг кадра, бит 6 = пятый спрайт, бит 5 = столкновение) сбрасывает его.

Пример `plab-couleurs.s` (в библиотеке, папка **6502**; также в `docs/exemples/en/asm/`) включает режим Graphics I, заполняет таблицы шаблонов, цветов и имён и оставляет на экране карты цветной узор. Запустите его из редактора (▶ **Запустить**, затем `300R`, или ассемблируйте и запустите одним щелчком):

plab-couleurs.s (фрагмент) 6502

```
VDATA   = $CC00
VCTRL   = $CC01

start:  ldx #0
@regs:  lda regs,x          ; R0..R7
        sta VCTRL           ; значение
        txa
        ora #$80
        sta VCTRL           ; номер регистра + $80
        inx
        cpx #8
        bne @regs
```

Загрузка публичных демонстраций

Демонстрации проекта `nippur72/apple1-videocard-lib` (написанные на KickC) бывают в четырёх видах; эмулятор знает все:

Файл	Что это	Как загрузить
`*.bin`	сырая программа, начинающаяся с `$0280`	**Загрузить** (диалог адреса предлагает `280`), затем `280R` на терминале
`*.woz`	то же, в виде текста для Woz Monitor	**Загрузить**, затем `280R`
`*.prg`	начальный адрес из 2 байт (младший байт первым), затем код	**Загрузить**: адрес читается из файла
`*_jukebox.bin`	образ EPROM 16 КБ для CodeTank	**ПЗУ CodeTank...**, затем `4000R`

Эти демонстрации не распространяются с комплектом (их автор не опубликовал лицензию); скачайте их со страницы проекта и держите в своей библиотеке. Эмулятор проверен на четырёх из них (меню демонстраций с его экранами и подпрыгивающий воздушный шар, рука Amiga, просмотрщик фотографий и Tetris) в вариантах для ОЗУ и jukebox.

Что эмулируется, а что нет

▣ ЧТО ДЕЛАЕТ ЭМУЛЯТОР

Четыре режима отображения (Graphics I и II, Text, Multicolor); 16 КБ видеопамяти; 32 спрайта, 8 × 8 или 16 × 16, увеличенные или нет, с ограничением в пять спрайтов на строку и флагом столкновения; флаг кадра и прерывание конца кадра, IRQ или NMI; CodeTank по адресам `$4000-$7FFF`. Палитра TMS9918A — обычная для NTSC.

! ЧТО НЕ ЭМУЛИРУЕТСЯ

Регистр, изменённый посреди кадра, вступает в силу со следующего кадра (эффекты развёртки не воспроизводятся); режим DRAM 4 КБ; точная синхронизация спрайтов и обращений к видеопамяти; внешний видеовход; артефакты композитного видео. Состояние видеомикросхемы **не** сохраняется в состояниях машины.

ГДЕ ДОКУМЕНТАЦИЯ МОЛЧИТ

Руководство карты (ревизия 0.4) не говорит, как повторяется окно, какова синхронизация шины, что содержит видеопамять при включении питания, защищён ли CodeTank от записи и как именно подключена перемычка IRQ/NMI. По этим пунктам эмулятор делает разумный выбор (описанный выше); если у вас есть карта и вы видите различие, сообщите нам.

ГЛАВА 7

Редактор

Встроенный редактор позволяет писать программы на Integer BASIC и исходники 6502, проверяет их при каждом нажатии клавиши, запускает их на эмулируемом Apple-1 одним щелчком и экспортирует на кассеты для настоящей машины.

7.1 Вкладки и файлы

📝 **Редактор** открывает панель, закреплённую слева от экрана. Его панель инструментов компактна, чтобы оставить тексту как можно больше строк. Она начинается со значков, которые объясняются во всплывающих подсказках:

Значок	Действие
Новый (первый значок)	Новая вкладка: «Программа на BASIC» или «Исходник на ассемблере 6502»
📂 / 💾 / 💾…	Открыть (`.bas` , `.txt` , `.s` , `.asm` , `.a65` , `.inc`) / Сохранить (Ctrl+S) / Сохранить как…
✂, копировать, 📋	Вырезать, копировать, вставить
🔍	Найти и заменить (Ctrl+F, Ctrl+H)
⟲ ⟳	Отменить (Ctrl+Z), повторить (Ctrl+Shift+Z)
A▲▼	Размер текста: один компактный элемент, буква A с двумя маленькими треугольниками друг над другом; ▲ увеличивает, ▼ уменьшает

За ними, в той же строке (или в следующей, если панель узкая), идут действия вкладки: ▶ **Запустить**, 📼 **Экспорт WAV** и **Память** для BASIC или 📃 **Листинг**, • **Точка останова** и ⏩ **До курсора** для исходника 6502.

Каждый файл — это вкладка. Точка • отмечает изменённую вкладку; × закрывает её (если она не сохранена, нужен второй щелчок). Вкладка с пометкой 📚 открыта из библиотеки: открытие листинга или исходника из библиотеки (▶, ⬇ или двойной щелчок) открывает его вкладку, когда редактор виден, или показывает её, если она уже открыта. Вкладки, включая несохранённый текст, возвращаются при следующем запуске. Правый щелчок по вкладке предлагает **Сохранить как…** и **Закрыть**. В тексте **щелчок средней кнопкой** вставляет первичное выделение (Linux): курсор встаёт туда, куда вы щёлкнули, затем вставляется текст, выделенный последним в любом приложении.

Размер текста задаётся элементом **A▲▼**, Ctrl+колесо или Ctrl++ / Ctrl+– / Ctrl+0, когда фокус клавиатуры в тексте.

Открепить выносит редактор в отдельное окно, например на второй экран; **⊟ Закрепить** возвращает его. Клавиши, нажатые в этом окне, идут в редактор, а не в Apple-1.

7.2 Поиск, замена и меню правого щелчка

🔍 (или Ctrl+F) открывает над текстом строку поиска: Enter переходит к следующему совпадению, Shift+Enter — к предыдущему, **Aa** учитывает регистр, **Слово** ищет слова целиком, а счётчик показывает «3 из 7». Ctrl+H добавляет поле **Заменить на**, с кнопками **Заменить** (текущее совпадение) и **Заменить все**. Esc закрывает строку.

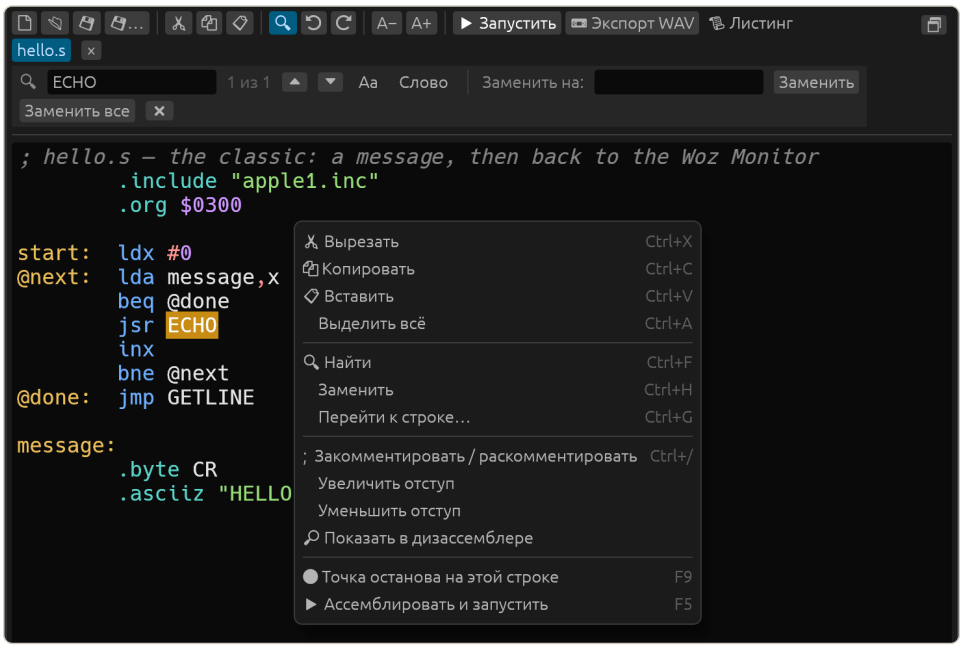


Рисунок 21 — Строка поиска, открытая на ECHO, и меню правого щелчка для исходника 6502.

Правый щелчок по тексту открывает меню со всеми командами и их сочетаниями клавиш. Меню действует на **выделение**, которое сохраняется при правом щелчке по нему: **Вырезать** и **Копировать** работают с выделенным текстом. Команды, зависящие от слова (переход к определению, справка), используют **слово под щелчком**, а не под курсором:

Команда	Клавиши
Вырезать, Копировать, Вставить, Выделить всё	Ctrl + X, C, V, A
Найти, Заменить	Ctrl + F, Ctrl + H
Перейти к строке…	Ctrl + G
Закомментировать / раскомментировать (6502), REM / убрать REM (BASIC)	Ctrl + /
Увеличить отступ, Уменьшить отступ	
Перейти к определению метки (6502) или номера строки (BASIC)	F12
Справка по *слову* (для ключевого слова, инструкции или директивы: открывает её статью в справочнике)	
Показать в дизассемблере (6502)	
Перенумеровать… (BASIC: `GOTO`, `GOSUB` и `THEN` следуют за номерами)	
Точка останова на этой строке	F9
Ассемблировать и запустить (6502), Отправить и запустить (BASIC)	F5

Цвета. Цвета текста следуют теме интерфейса. ⚙ **Настройки** → **Редактор** предлагает, отдельно для BASIC и для ассемблера 6502, схемы *Автоматически (по теме)*, *Классическая (тёмная)*, *Светлая*, *Янтарная*, *Высокий контраст* или ваши собственные цвета, по образцу на каждый вид слов (глава 11).

7.3 Программа на BASIC

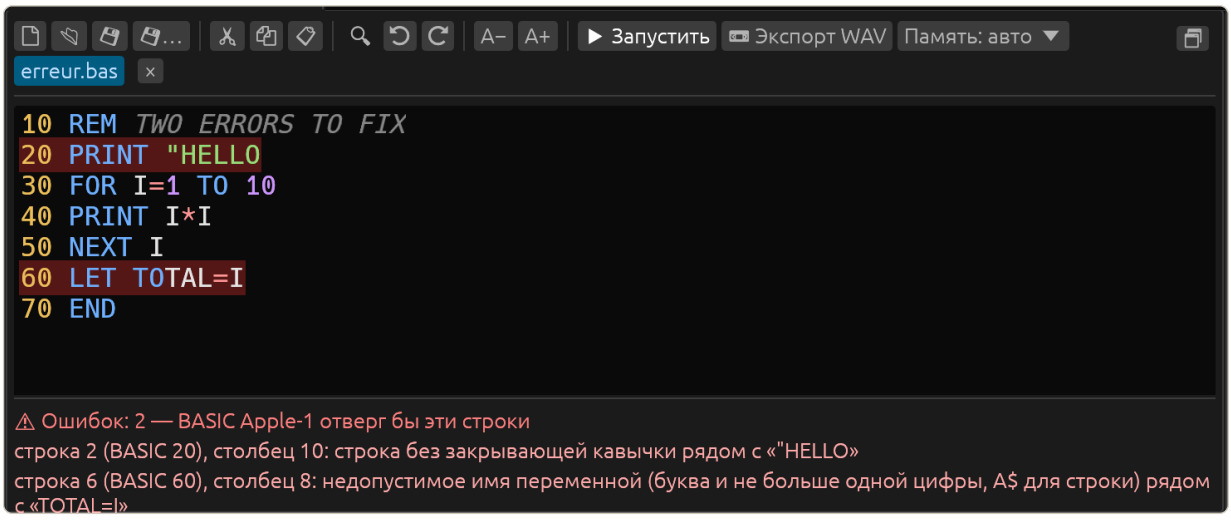


Рисунок 22 — Редактор BASIC: ошибки на красном фоне, с номером строки и столбца внизу панели.

Текст раскрашен: номера строк, ключевые слова, числа, строки, `REM`, операции. Каждая строка **токенизируется** так, как это сделал бы Integer BASIC, собственным токенизатором, сверенным байт в байт с настоящим. Строка, которую BASIC отверг бы, становится красной, а ошибка указывает строку текста, номер строки BASIC и столбец; щелчок ставит туда

курсор. Обнаруживаемые ошибки: синтаксис, недопустимое имя переменной, число больше 32767, команда прямого режима (`LIST` , `RUN` ...) внутри программы, незакрытая строка, строка длиннее 255 байт, переполнение памяти.

- **Память**: автоматически (8 КБ с LOMEM `$0800` или `$0300` , если программа большая, иначе 12, 16 или 32 КБ) или принудительно.
- ▶ Запустить: загружает Integer BASIC, если его нет (как кнопка BASIC), помещает токенизированную программу в память, при необходимости добавляет ОЗУ, затем набирает `E2B3R` и `RUN` . Это мгновенно: ничего не набирается построчно.
- 📼 Экспорт WAV: кассета (`.wav` или `.aiff` для смартфона) из двух записей, с параметрами в библиотеке. Внизу панели показано, что набрать на настоящем Apple-1, например `C100R` , затем `004A.00FFR 0C3B.0FFFR` , затем `E2B3R` и `RUN` ; 📋 **Копировать** помещает это в буфер обмена.

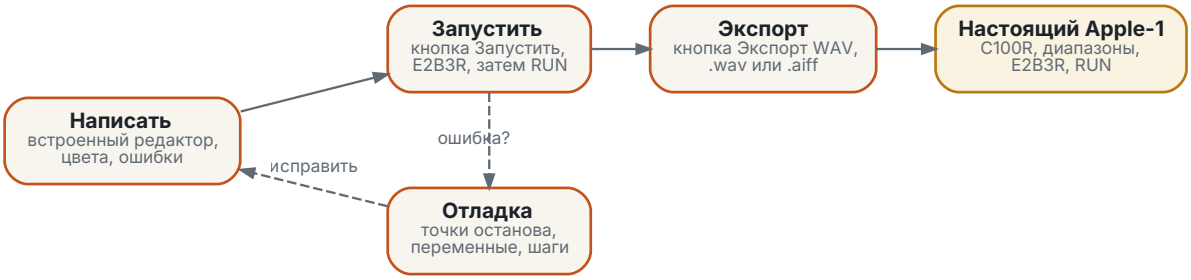


Рисунок 23 — От редактора к настоящему Apple-1: написать, запустить, исправить, экспортировать.

7.4 Исходник 6502

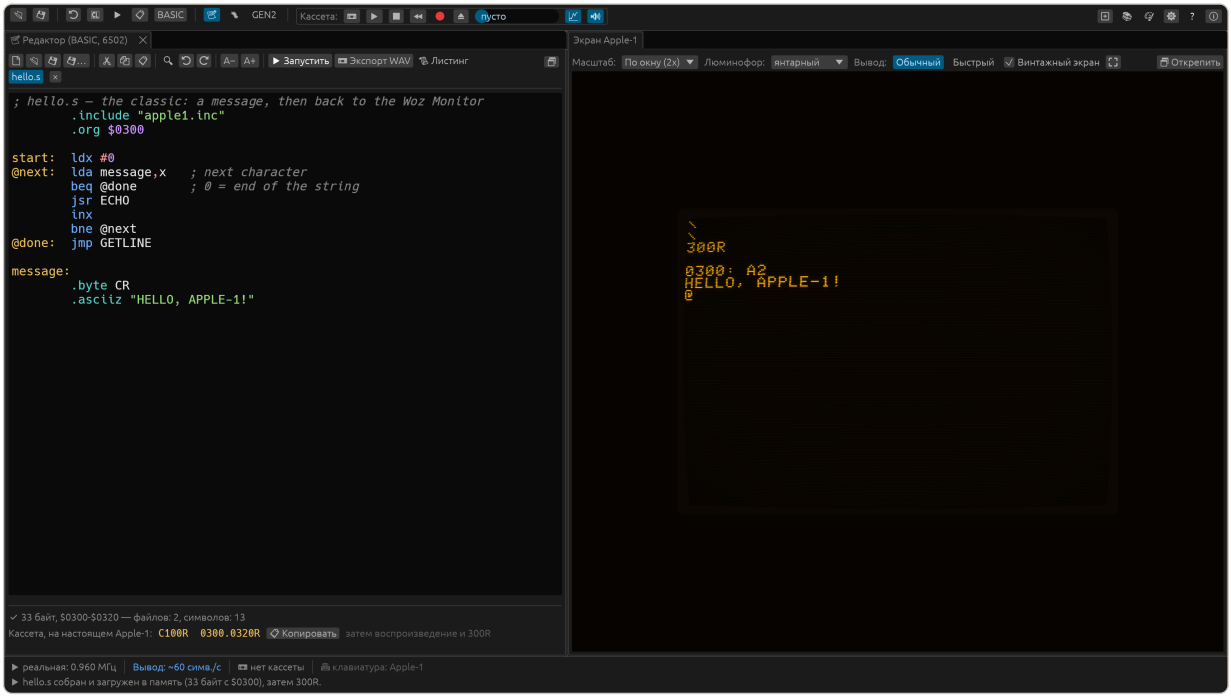


Рисунок 24 — Исходник 6502 в редакторе: цвета, результат ассемблирования и строка для настоящего Apple-1.

Исходник ассемблируется **при каждом изменении** встроенным двухпроходным ассемблером, который принимает синтаксис ca65 (и синтаксис xa, переводимый автоматически). Внизу панели показаны размер и адреса (`33 байт, $0300-$0320 — файлов: 2, символов: 13`), затем строка для настоящей машины (`C100R` , затем `0300.0320R` , затем `300R`).

- **Ошибки**: файл, строка и столбец; щелчок переводит туда, при необходимости открывая включаемый файл. Предупреждения (нулевая страница, закодированная как абсолютный адрес) — жёлтые.
- 📃 **Листинг**: адрес, байты и исходный текст каждой строки, с включаемыми файлами и развёрнутыми макросами, затем таблица символов; щелчок возвращает к строке.
- ▶ **Запустить**: помещает байты в память по начальному адресу (`.org` , по умолчанию `$0300`), при необходимости добавляет ОЗУ, затем набирает, например, `300R` .
- 📼 **Экспорт WAV**: кассета, как для BASIC.
- ● **Точка останова** и ⏩ **До курсора**: связь с отладчиком (глава 8).
- **Связь с отладчиком**: когда вы выделяете инструкцию в исходнике (или ставите курсор на её строку), порождённые ею байты подсвечиваются в **дизассемблере** отладчика и в **дампе hex / ASCII**, если эти панели видны; если байты лежат в другом месте памяти, панели прокручиваются к ним. Это возможно благодаря таблице строк исходника и адресов ассемблера.

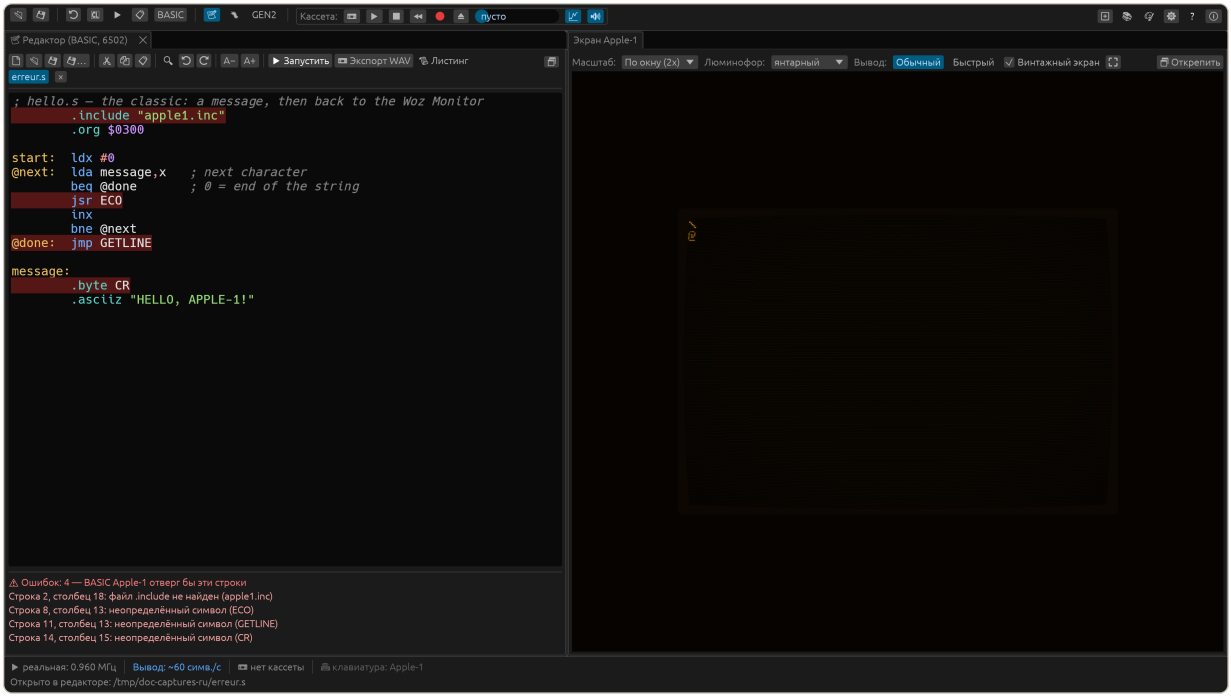


Рисунок 25 — Ошибка ассемблирования: ECO вместо ECHO, неизвестный символ, с указанием строки и столбца.

Часть III целиком посвящена ассемблеру: синтаксис, директивы, листинг, примеры.

ГЛАВА 8

Отладчик, дизассемблер и дамп

Отладчик останавливает Apple-1 и ведёт его пошагово — по одной инструкции в 6502 или по одной строке в BASIC, — показывая регистры, переменные и память.

8.1 Панель отладчика

🐛 **Отладчик** открывает панель, закреплённую справа или откреплямую, как редактор. Её команды:

Команда	Действие
⏸ **Пауза** / ▶ **Продолжить**	останавливает или возобновляет работу Apple-1
⬇ **Шаг**	выполняет одну инструкцию
↪ **Обойти**	выполняет `JSR` целиком и останавливается по возврату
⤴ **Выйти**	выполняет до `RTS` текущей подпрограммы
⏭ **Строка BASIC**	продвигается на одну строку программы на BASIC
⏩ **До курсора**	(в редакторе) выполняет до строки с курсором

Рамка **Процессор** показывает A, X, Y, S, PC и флаги N V B D I Z C; все они изменяются щелчком по значению. Ниже показано число выполненных тактов и инструкций.

8.2 Точки останова

Точку останова можно поставить четырьмя способами:

- ● **Точка останова** в редакторе (или `F9`, или меню правого щелчка) на строке с курсором: в исходнике `.s` (благодаря таблице адресов ассемблера) или в программе на BASIC (номер строки);
- щелчком на полях 📃 **Листинга**;
- щелчком по строке дизассемблера;
- вводом адреса или метки: `$0368`, `ASK`, `PUTS+3` или `L190` для строки BASIC 190. Метка, которой ещё нет, не отвергается: точка останова сохраняется и ставится, как только метка появится (когда вы ассемблируете исходник редактора или запускаете его), с сообщением об этом.

Точки останова, наблюдаемые переменные и наблюдения за памятью сохраняются от сеанса к сеансу, но загрузка другой программы стирает точки останова и метки предыдущей. Когда загружен встроенный BASIC, раздел BASIC в отладчике называется **RDK G2 Basic**; если в

`$E000` нет никакого BASIC, он говорит об этом нейтрально. Цвета текущей строки, адресов дампа и столбца ASCII меняются в ⚙ **Настройки → Интерфейс** («Цвета интерфейса»). В узких панелях у дизассемблера пропадает столбец байтов, а строка poke переносится вместо обрезания.

8.3 Отладка программы на BASIC

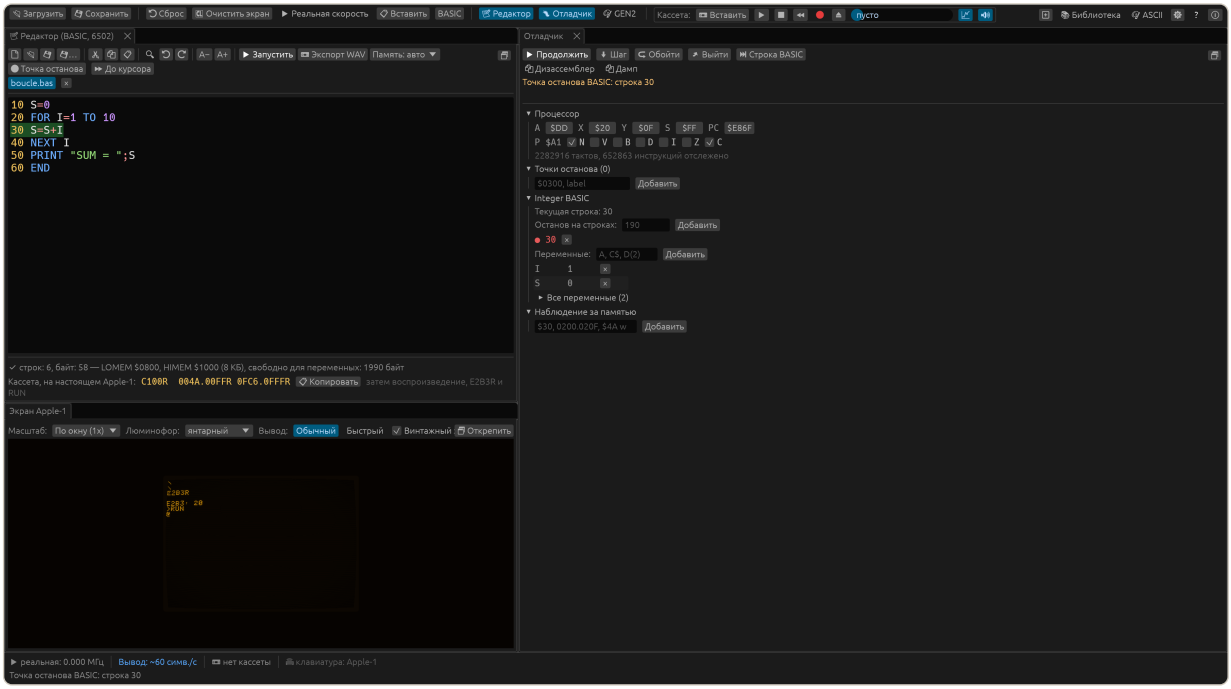


Рисунок 26 — boucle.bas, остановленная на точке останова в строке 30: текущая строка зелёным в редакторе, переменные I и S под наблюдением в отладчике.

Текущая строка подсвечена в редакторе зелёным, точки останова — фиолетовым. Рамка BASIC (**RDK G2 Basic** или **Integer BASIC** — в зависимости от загруженного) показывает текущую строку, точки останова на номерах строк и **наблюдаемые переменные**: целые (`A`, `B1`), элементы массивов (`D(2)`), массивы целиком (`D`), строки (`C$`), а также **Все переменные**.

Отладка работает с **RDK G2 Basic** — BASIC, встроенным в эмулятор, — так же, как с оригиналом: настоящий Integer BASIC больше не нужен. Отладчик замечает начало каждой строки (текущая строка, точки останова на строках, ⏭ **Строка BASIC**) и читает переменные из таблиц интерпретатора. Для оригинала он читает строку из указателей нулевой страницы `$DC` / `$DD`; G2 Basic хранит её в другом месте, и отладчик знает оба устройства.

> **! НАЖАТИЕ КЛАВИШИ ОСТАНАВЛИВАЕТ ПРОГРАММУ**
>
> Как и на настоящей машине, клавиша, нажатая во время работы программы на BASIC, останавливает её (`STOPPED AT`). Во время отладки держите фокус клавиатуры в панели отладчика, а не на экране Apple-1. Программа на G2 Basic может читать клавиатуру, не останавливаясь, с помощью `KEY(0)` (глава 15).

8.4 Отладка программы для 6502

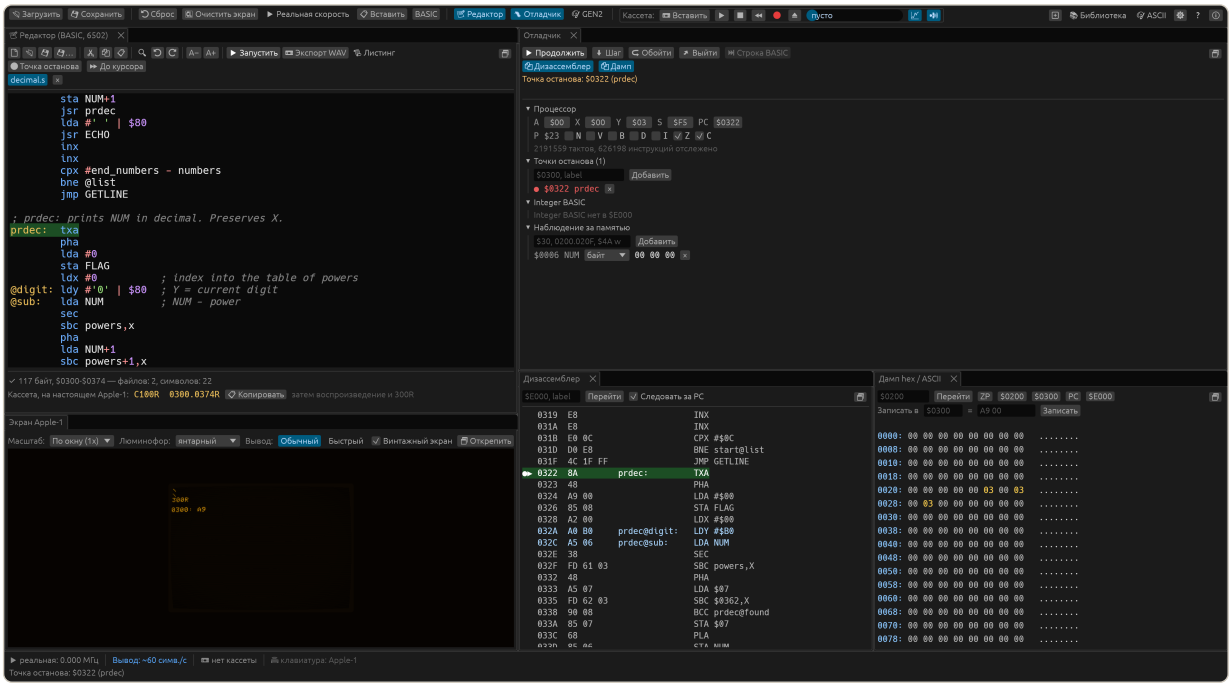


Рисунок 27 — decimal.s, остановленная на метке prdec: регистры, дизассемблер с метками программы, наблюдение за памятью $06.$08.

Дизассемблер показывает 24 строки вокруг PC, с метками ассемблированной программы в отдельном столбце. Он перецентрируется, только когда PC выходит из видимой области, поэтому изображение не прыгает на каждом шаге. **Перейти** закрепляет список на адресе или метке; **Следовать за PC** снова заставляет его следовать за выполнением.

Когда в исходнике 6502 редактора выделена инструкция, её байты **подсвечиваются** в дизассемблере и в дампе (глава 7), а обе панели прокручиваются к ним, если они вне поля зрения; дизассемблер при этом перестаёт следовать за PC, пока вы не нажмёте **Следовать за PC**.

Наблюдение за памятью показывает байт, слово или ASCII (`$30` , `$30 w` , `0200.020F` , метку), с возможностью правки, жёлтым — при изменении значения.

8.5 Дамп

Дамп показывает память в шестнадцатеричном виде и в ASCII, по 16 байт в строке (8, если панель узкая), с отброшенным битом 7 для ASCII. **Перейти** переходит к адресу или к ярлыкам ZP, `$0200` , `$0300` , PC, `$E000` . Изменённые байты — жёлтые, байт по адресу PC — на зелёном фоне. **Записать** записывает байты (`0300 = A9 00`); щелчок по байту заполняет строку записи.

Дизассемблер и дамп — самостоятельные панели: по умолчанию откреплённые, их можно закрепить рядом с отладчиком (глава 10).

8.6 В командной строке

Для демонстраций и автоматических тестов: `--debogueur` открывает панель, `--dbg-arret ASK,L190` ставит точки останова, `--dbg-surveiller 'A,C$,$30 w'` наблюдает за переменными и памятью, а `--dbg-commande 5:pas` посылает команду через 5 секунд (`pause` , `continuer` , `pas` , `dessus` , `sortir` , `basic`). Эти имена параметров и ключевые слова — французские.

ГЛАВА 9

Фото → ASCII

Инструмент «Фото → ASCII» превращает фотографию в картинку Apple-1 из 40 × 24 символов, а затем в программу на BASIC или 6502, готовую к запуску или к экспорту на кассету.

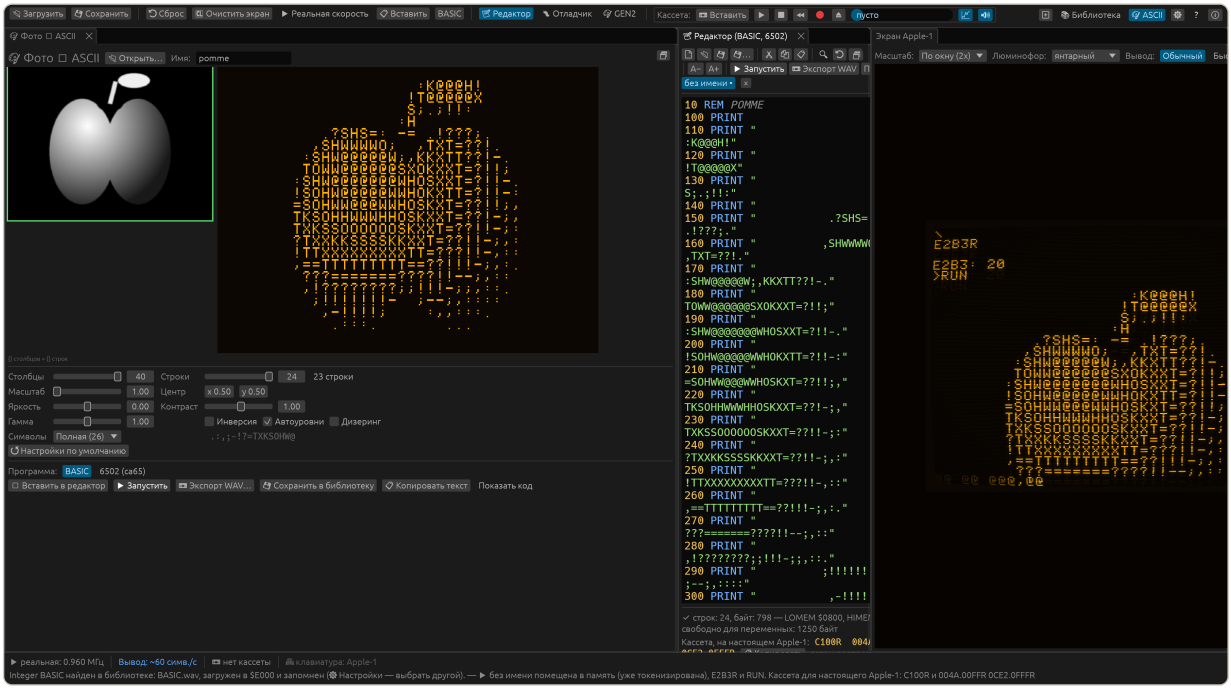


Рисунок 28 — Инструмент «Фото → ASCII»: фотография и рамка кадрирования, настройки и предпросмотр шрифтом 2513.

9.1 Открытие изображения

🎨 **ASCII** открывает инструмент — закрепляемую панель, как и остальные. **Перетащите** фотографию `.jpg` или `.png` в окно или воспользуйтесь 📂 **Открыть…** (фотография или готовый ASCII-арт в `.txt`).

Зелёная рамка на фотографии показывает преобразуемую часть: перетаскивайте её, чтобы сдвинуть, колесо мыши масштабирует.

9.2 Настройки

Настройка	Назначение
Столбцы, **Строки**	не больше 40 × 24. **23 строки** оставляют последнюю строку для приглашения, чтобы картинка не прокручивалась.
Масштаб, **Центр**	сохраняемая часть фотографии
Яркость, **Контраст**, **Гамма**	обычные настройки (гамма > 1 осветляет)
Инверсия	тёмное = много краски
Автоуровни	растягивает уровни серого (от 2 % до 98 %) до остальных настроек
Дизеринг	Флойд — Стейнберг: больше оттенков, больше шума
Символы	шкала **Полная** (26 уровней), **Простая** (10), из **Slideshow (Шменк)** или **Своя**

Возможны только символы 2513: заглавные буквы, цифры и знаки, без строчных, без `"` и без `_`. Они упорядочены по числу горящих точек в шрифте Apple-1: уровень серого задаёт именно это число, а не форма.

> ✦ ДЛЯ ПОРТРЕТА
>
> 40 × 24 символа — это немного: кадрируйте по лицу (масштаб от 1,3 до 1,6), немного поднимите контраст и для светлой фотографии попробуйте гамму меньше 1. Лучше всего получается однотонный тёмный фон.

9.3 Получаемая программа

- **BASIC**: по одной строке `PRINT "…"` на каждую строку картинки. Полная строка из 40 символов заканчивается на `;`, потому что экран сам переносит строку. Такая программа медленно вводится (около 2 с на строку).
- **6502**: исходник ca65 по адресу `$0300`, запускаемый командой `300R`. После очистки экрана цикл посылает в ECHO сжатую таблицу (серии одинаковых символов закодированы как «количество + символ»). Вывод мгновенный.

При 24 строках программа ждёт нажатия клавиши, прежде чем вернуться; при 23 приглашение оказывается на последней строке, и картинка остаётся целой.

Кнопки: ↰ **Вставить в редактор** (новая вкладка BASIC или 6502), ▶ **Запустить**, 📼 **Экспорт WAV...**, 💾 **Сохранить в библиотеку** (`.txt` в папке «Art ASCII»), 📋 **Копировать текст**, **Показать код**.



Рисунок 29 — От фотографии к настоящему Apple-1: преобразование, программа, кассета.

9.4 В командной строке

TERMINAL

```
apple1-emu ascii photo.jpg                      # только картинка, на экран
apple1-emu ascii woz.jpg --zoom 1.3 --cx 0.45 --cy 0.32 \
          --contrast 1.3 --gamma 0.45 --rows 23 -o woz.txt
apple1-emu ascii woz.jpg -o woz.bas             # программа на BASIC
apple1-emu ascii woz.jpg -o woz.s               # исходник 6502
apple1-emu ascii woz.jpg -o woz.wav             # кассета 6502, запуск 300R
apple1-emu ascii woz.jpg -o woz.wav --format basic
```

Все параметры приведены в главе 12.

▣ **СЛАЙД-ШОУ ДЭЙВА ШМЕНКА**

В 2014 году Дэйв Шменк написал для Apple-1 слайд-шоу из портретов. Библиотека добавляет его как проект GitHub (`dschmenk/Apple1-Slideshow`): он скачивается, а не поставляется, потому что у репозитория нет лицензии. ▶ **Проверить последнюю сборку** ассемблирует его (синтаксис xa, переводится автоматически; те же 3456 байт, что и у xa 2.4.1) и запускает командой `280R` .

ГЛАВА 10

Расположение панелей

Каждая панель — это вкладка, которую можно поставить куда угодно: рядом с другими, во вкладки или в отдельное окно на втором экране.

10.1 Перемещение панели

Редактор, экран Apple-1, цветной монитор GEN2, отладчик, дизассемблер, дамп, инструмент ASCII и библиотека — всё это **закрепляемые вкладки**. Перетащите вкладку за заголовок: появятся направляющие, и вы отпустите её **слева от**, **справа от**, **над** или **под** другой панелью или **в ту же группу вкладок**. Перетаскивайте разделители, чтобы менять размеры. Расположение запоминается в `gui.json`.

10.2 Исходное расположение

Редактор — слева; в центре — экран Apple-1, а под ним цветной монитор, когда карта GEN2 подключена; справа — отладчик с дизассемблером под ним, затем дамп, а библиотека — у самого правого края. При самом первом запуске выбор делается по ширине **монитора** (окно открывается небольшим и через мгновение разворачивается): если она меньше 1400 пикселей (ноутбук 1366 × 768, экран 1024 × 768), запускается **редактор и экран** — самая полезная пара; от 1400 пикселей — со всеми панелями. Другое расположение можно выбрать в меню в любой момент. Оно следует за шириной окна: каждая панель сохраняет свою долю — от ноутбука до большого монитора. Меню **⊞ Расположение ▾** на панели инструментов восстанавливает его в любой момент (**По умолчанию**, все панели открыты) и предлагает ещё четыре готовых расположения: **Игра** (экран Apple-1 и библиотека), **BASIC** (редактор и экран), **Ассемблер** (редактор, экран и отладчик) и **Отладка** (редактор, отладчик, дизассемблер и дамп). **Сохранить текущее расположение...** спрашивает имя и запоминает текущее расположение вместе с открепённые окнами и размером главного окна (при загрузке эти окна открываются там, где были); ваши расположения перечислены под разделителем по алфавиту, а правый щелчок позволяет **Переименовать** или **Удалить** их. Они хранятся в `gui.json`.

10.3 Помощник при запуске

При первом запуске помощник из трёх экранов заменяет прежнее окно «Ваше рабочее пространство». Он нарисован янтарём на чёрном, с эффектом трубки (строки развёртки, свечение, виньетка, кривизна 0,07) на рамке; мелкий текст и миниатюры остаются чёткими. Маленький × справа сверху его закрывает. **‹ Назад** и **Далее ›** переключают экраны; на последнем — **Готово**.

1. **Добро пожаловать**: для чего комплект.

2. **ПЗУ и BASIC**: Woz Monitor, ПЗУ кассеты и G2 Basic уже внутри. Больше ничего не нужно. Пуристы ставят свой оригинальный Integer BASIC из внешнего источника; кнопка открывает руководство («Установка ПЗУ»), в котором только ссылки, без файлов.
3. **Ваше рабочее пространство**: пять расположений (**По умолчанию**, **Игра**, **BASIC**, **Ассемблер**, **Отладка**). Каждая миниатюра нажимается (кнопки «Выбрать» нет). Прямоугольники сохраняют номера с 1 по 6, а цветной экран (GEN2 / P-LAB) — это **7**, своим цветом, в миниатюрах, которые его показывают, и в легенде.

Флажок **Показывать этот помощник при запуске** есть на всех трёх экранах и отмечен по умолчанию. Тот же флажок — в ⚙ **Настройки** → **Интерфейс**. ⊞ **Расположение** ▾ → **Помощник при запуске…** открывает его снова. Это обычное окно: оно не остаётся поверх других. Оно не открывается, когда командная строка задаёт размер (`--taille`), и при снимках.

10.4 Размер текста каждого окна

У редактора, листинга ассемблера, отладчика, дизассемблера, дампа и библиотеки в их строке есть компактный регулятор **A 13 ▲▼**: треугольники меняют размер, и он запоминается для этого окна. Ctrl+колёсико и Ctrl+плюс / Ctrl+минус действуют на окно под указателем мыши (над экраном Apple-1 Ctrl+плюс / Ctrl+минус меняют масштаб). **Сбросить настройки** возвращает все эти размеры к 13.

10.5 Что запоминается между запусками

Расположение сохраняется само при выходе и находится таким, каким было: положение и размер каждой панели, открепённые окна, масштаб и выбранное расположение. Окну возвращаются его размер и место, затем панели расставляются в прежних пропорциях, на любом экране: восстановление проверено при 2880 × 1620, 2560 × 1440, 1920 × 1080 и 1366 × 768, после перезапуска различалось менее 1 % изображения. Расположение, сохранённое под именем (**Сохранить текущее расположение…**), дополнительно хранит открепённые окна. Интерфейс никогда не масштабируется сам: размер кнопок меняет только параметр «Размер значков и текста».

10.6 Когда экран больше своей панели

При **фиксированном масштабе** (2x, 3x… или Ctrl+ +) экран Apple-1 сохраняет запрошенный размер, даже если панель меньше: он тогда **прокручивается** (колесом или полосами), а не уменьшается, и строка состояния сообщает об этом, советуя увеличить панель. Только **По окну** подбирает масштаб под панель. В узком окне (например 1024 × 768) панель инструментов прокручивается по горизонтали, а не обрезается: кнопка **»** показывает группу «Кассета», второй щелчок возвращает к началу. Кнопка **Запустить** редактора всегда остаётся видимой.

10.7 Открепление панели

У каждой панели есть кнопка **Открепить** справа вверху (а также в меню правого щелчка по вкладке), которая открывает её в **отдельном окне**; это окно можно перенести на второй экран, его размер и положение запоминаются. ⊟ **Закрепить** возвращает панель на исходное место.

- Экран Apple-1 и цветной монитор открепляются по отдельности. Клавиатура идёт в Apple-1 из любого из их окон, а также из главного окна. Закрытие окна экрана снова закрепляет его.
- Закрытие окна инструмента (редактора, отладчика…) закрывает инструмент; его кнопка на панели инструментов снова открывает его на том же месте.
- Диалоговые окна (Открыть, Сохранить как, Экспорт) всегда открываются в главном окне. В окнах файлов Esc закрывает окно (даже из поля имени), Ctrl+L позволяет править путь (его текст выделен: набор заменяет его); «Сохранить как» и «Экспорт» из вкладки начинаются в папке её файла.
- Сохранённые расположения хранят открепённые окна; главное окно сохраняет один и тот же размер от запуска к запуску, при любом масштабе.
- Осциллограф кассеты остаётся маленьким плавающим окном внутри главного окна.

Чтобы играть

Закройте редактор и отладчик, оставьте библиотеку. **По окну** увеличивает экран; F11 переходит в полноэкранный режим, с винтажным экраном.

Чтобы программировать

Редактор слева, экран в центре, отладчик и дамп справа. Открепите редактор на второй монитор, чтобы отдать ему всю высоту.

Для GEN2

Подключите карту: цветной монитор встанет под экраном Apple-1. Открепите его, чтобы получить большое изображение, как на втором мониторе.

Для выступления

Открепите экран Apple-1 на проектор, в полноэкранном режиме, а главное окно оставьте на ноутбуке.

ГЛАВА 11

Настройки

⚙ *Настройки собирают все сохраняемые параметры. Всё хранится в ~/.config/apple1-emu/gui.json (или в папке комплекта, в переносном режиме).*

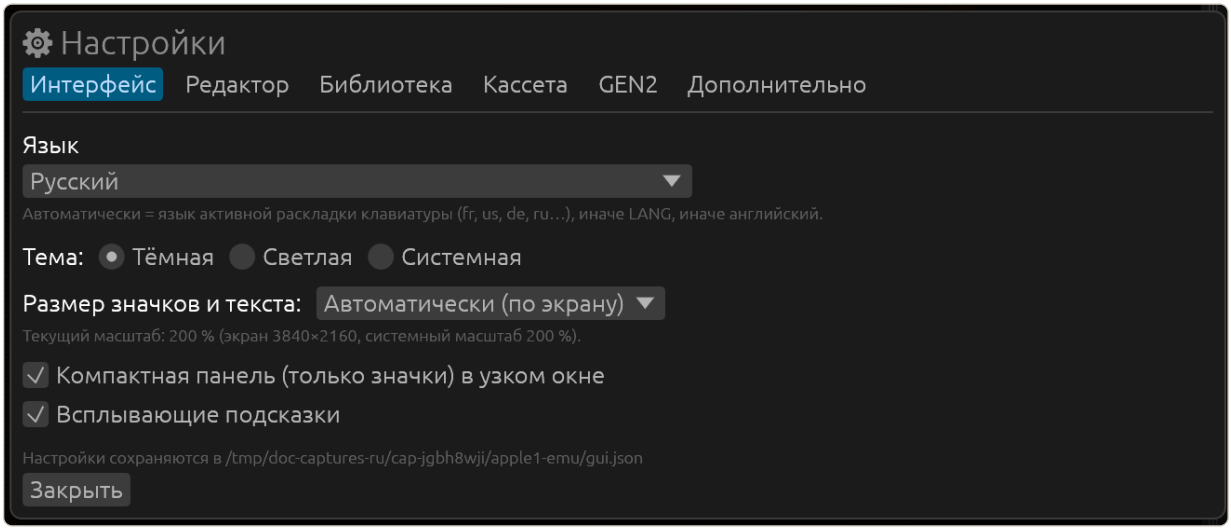


Рисунок 30 — Вкладка «Интерфейс» в настройках: язык, тема, размер значков и текста, компактная панель, подсказки, окно запуска и цвета интерфейса.

Параметры распределены по шести вкладкам:

Вкладка	Параметры
Интерфейс	язык: автоматически (по раскладке клавиатуры) или принудительно: французский, английский, немецкий, русский, испанский, итальянский, португальский; показаны определённый язык и его источник. Тема: тёмная, светлая или системная. Размер значков и текста: мелкий, обычный (по умолчанию), крупный, очень крупный. Компактная панель (только значки в узком окне). Всплывающие подсказки. **Показывать это окно при запуске** (окно «Ваше рабочее пространство») и кнопка, открывающая его сразу. **Цвета интерфейса**: цвета списков интерфейса, выбираемые **отдельно для тёмной и светлой темы** (активные кнопки, текущая строка отладчика, подсвеченная строка, выбранная ячейка, строка точки останова, адреса и столбец ASCII дампа, изменённые значения, метки); у каждого цвета есть образец с мгновенным просмотром, значения по умолчанию читаемы (контраст не ниже 4,5:1 к фону темы), а кнопки **По умолчанию** возвращают одну тему или обе.
Редактор	цвета синтаксиса, отдельно для BASIC и для ассемблера 6502 (глава 7)
Библиотека	основная папка (📂 **Обзор...**, ⟲ **По умолчанию**, **Открыть папку**), другие папки, проекты GitHub и необязательный токен GitHub (только для закрытых репозиториев и артефактов Actions; никогда не показывается, а переменная `GITHUB_TOKEN` имеет приоритет); положение библиотеки — слева или справа (по умолчанию справа)
Кассета	звук кассеты (выключен на быстрой скорости) и его громкость, осциллограф при воспроизведении или записи
GEN2	цветная карта Uncle Bernie, развёртка 60 или 50 Гц, палитра LORES (глава 6)
Дополнительно	экран и скорость: показывать библиотеку, вывод ограничен ~60 символами/с, как у настоящего терминала, быстрая скорость; **Винтажный экран: тонкие настройки** (общие для обоих экранов, глава 3); BASIC, загружаемый в `$E000`: встроенный **RDK G2 Basic** (по умолчанию) или **оригинальный Integer BASIC** (ваш собственный файл, 📂 **Выбрать...** / **Забыть**)

У каждого ползунка настроек (интенсивность винтажного экрана, послесвечение и кривизна, громкость кассеты) рядом показано его **числовое значение**. При первом запуске кривизна винтажного экрана равна **0,07** (лёгкая округлость трубки); существующая установка сохраняет своё значение. Предпросмотр на вкладке **Редактор** следует языку интерфейса.

Масштаба, люминофора и флажка винтажного экрана в настройках нет: они находятся в строке отображения каждого экрана (глава 3).

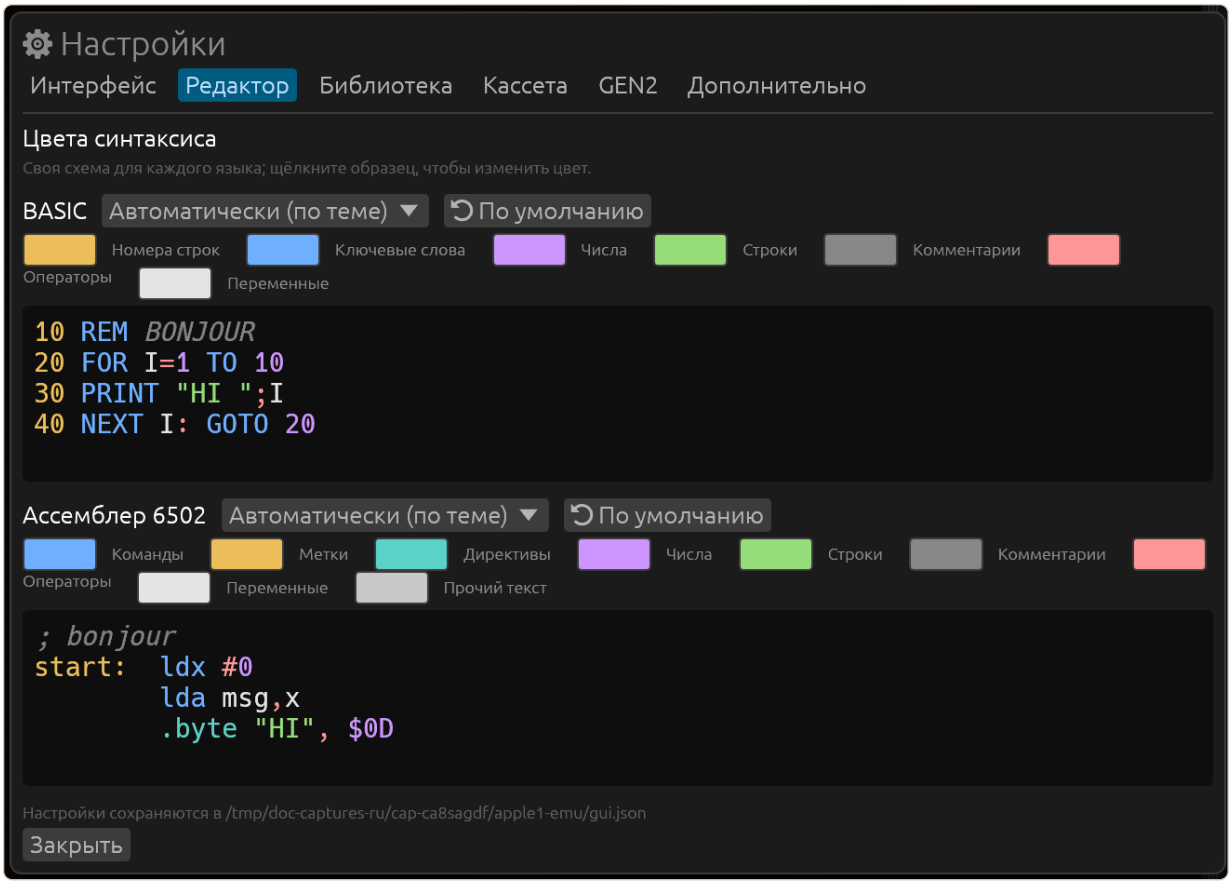


Рисунок 31 — Вкладка «Редактор»: цветовая схема для BASIC и для ассемблера 6502 или ваши собственные цвета.

Автоматический язык. Эмулятор следует активной раскладке клавиатуры: `fr` даёт французский, `us` или `gb` — английский, `de`, `ch` или `at` — немецкий, `ru` — русский, и так далее. Он опрашивает GNOME, `setxkbmap`, Cinnamon (случай Linux Mint), `localectl`, затем `/etc/default/keyboard` и, наконец, переменные `LANG`. В командной строке: `--langue fr|en|de|ru|es|it|pt|auto`.

✦ НАЧАТЬ С ЧИСТОГО ЛИСТА

Сбросить настройки внизу окна возвращает исходные параметры после подтверждения (язык, тема, расположение, винтажный экран...). Файлы библиотеки остаются на месте; несохранённые вкладки редактора закрываются. При первом запуске новой версии комплекта задаётся тот же вопрос, потому что `gui.json` сохраняется, когда папку программы стирают, чтобы установить другую.

Для переносного комплекта положите пустой файл с именем `portable` в корень комплекта (рядом с `run.sh`) или запустите программу с `APPLE1_PORTABLE=1`: настройки (`config/gui.json`), библиотека по умолчанию (`programmes`) и кэш тогда находятся в папке комплекта.

Закрыть эмулятор и переименовать `~/.config/apple1-emu/gui.json` — то же, что кнопка, при следующем запуске. Параметры программ остаются в `programs.json` каждой папки.

ГЛАВА 12

Командная строка

Две программы: apple1-emu-gui — графический интерфейс, и apple1-emu — эмулятор в терминале и набор инструментов для преобразования. Всё, что делают кнопки экспорта, можно сделать и из командной строки, для ваших скриптов.

> **! ИНСТРУМЕНТЫ КОМАНДНОЙ СТРОКИ ГОВОРЯТ ПО-ФРАНЦУЗСКИ**
>
> Графический интерфейс доступен на русском (`--langue ru`), но `apple1-emu` выводит сообщения и справку только по-французски, а параметры графического интерфейса, команды F2 и ключевые слова отладчика — французские слова (`--carte-couleur` , `charger` , `pas` ...). В этой главе показан настоящий вывод, с переводом там, где нужно.

12.1 Эмулятор в терминале

`apple1-emu` без команды запускает Apple-1 в терминале (не меньше 42 столбцов × 28 строк): 8 КБ ОЗУ плюс `$E000` , ACI, реальная скорость.

TERMINAL

```
apple1-emu                                # 8 КБ + E000, ACI, реальная скорость
apple1-emu --fast                         # запуск в быстром режиме
apple1-emu --load samples/hello.txt --run 300
apple1-emu --tape samples/hello.wav       # кассета в виртуальной деке
apple1-emu --basic ~/apple1/integer_basic.bin
apple1-emu --ram 32 --no-e000             # 32 КБ, без ОЗУ в E000
```

Параметр	Действие
`--ram KB`	ОЗУ с `$0000`, от 4 до 48 КБ (по умолчанию 8)
`--no-e000`	без ОЗУ в `$E000-$EFFF` (перемычка «W → E» на плате)
`--no-aci`	без кассетной карты
`--basic FILE`	ваш Integer BASIC (двоичный файл 4 КБ или шестнадцатеричный дамп), загружается в `$E000`
`--load FILE[@ADDR]`	программа в память (`.txt`, `.bin@ADDR`, `.wav`, `.aif`); можно повторять
`--tape FILE`	кассета в деке
`--type FILE`	набирает текстовый файл при запуске
`--run ADDR`	набирает `ADDRR` после загрузки
`--fast`, `--no-throttle`	быстрый режим; вывод без ограничения 60 символов/с
`--no-refresh`	без модели регенерации DRAM (1,0227 МГц вместо 0,960)
`--carte-couleur`, `--couleur-50hz`	карта GEN2 (регистры и память; цветное изображение есть только в графическом интерфейсе)
`--cursor-hz HZ`, `--cursor-duty F`	мигание курсора (по умолчанию 2,18 Гц, виден 2/3 времени)
`--monitor-rom`, `--monitor-page`	другой образ Woz Monitor (256 или 512 байт, страница 0 или 1)
`--aci-rom`, `--aci-page`	другой образ ППЗУ ACI

Клавиши терминала. Ctrl+R — RESET, Ctrl+L — CLEAR SCREEN, F1 — справка, F2 — командная строка, F3 — скорость, F4 — ограничение вывода, F5/F6 — воспроизведение/стоп кассеты, F10 или Ctrl+Q — выход. Текст, вставленный в терминал, набирается на клавиатуре Apple-1. Backspace набирает `_` — знак стирания Woz Monitor.

Команды F2 (слова команд французские).

Команда	Действие
charger `FILE[@ADDR]`	*загрузить*: загружает в память `.txt`, `.bin` (нужен `@ADDR`), `.wav` или `.aiff`
sauver `FILE START.END …`	*сохранить*: сохраняет память; формат выбирается по расширению. Диапазоны можно скопировать из строки ACI: `sauver st.aiff 004A.00FFR 0300.0FFFR`
cassette `FILE`	*кассета*: вставляет кассету, которая запускается при первом чтении ACI
`lecture`, `stop`, `rembobiner`, `ejecter`	дека: воспроизведение, стоп, перемотка, извлечение
enregistrer `FILE`	*записать*: взводит запись; затем наберите `C100R` и `0300.03FFW`
`basic FILE`	загружает ваш Integer BASIC в `$E000`
`taper FILE`	*набрать*: набирает текстовый файл на клавиатуре
`reset`, `effacer`, `rapide`, `reel`, `debit`, `aide`, `quitter`	сброс, очистка, быстро, реальная скорость, скорость вывода, справка, выход

12.2 Преобразования

Формат вывода выбирается по **расширению**: `.aif` / `.aiff` даёт AIFF (профиль для смартфона), `.wav` — WAV, `.txt` / `.hex` — дамп Woz, `.bin` — двоичный файл.

export (псевдоним `export-wav`) создаёт кассету. Сигнал создаётся настоящим ПЗУ ACI, работающим в эмулируемом 6502: длительности битов — те же, что у Apple-1, выполняющего `W`.

TERMINAL

```
apple1-emu export samples/hello.txt -o hello.aiff
apple1-emu export prog.bin@300 -o prog.wav --rate 48000 --bits 16
apple1-emu export startrek.txt --range "004A.00FFR 0300.0FFFR" -o startrek.aiff
```

Параметр	WAV (по умолчанию)	AIFF (по умолчанию)
`--rate`	44 100 Гц	48 000 Гц
`--bits 8|16`	16	16
`--channels 1|2`	1	2, одинаковых
`--amplitude` (от 0,05 до 1)	0,8	0,89 (−1 dBFS)
`--silence S`	0,5 с	1,0 с
`--range`	один диапазон или целая строка ACI; можно повторять	
`--no-refresh`, `--no-embed`	биты на 6 % короче; без адресов в файле	

import (псевдоним `import-wav`) декодирует кассету: `-o prog.txt` или `-o prog.bin`, `--addr ADDR` для кассеты без адресов, `--range`, чтобы разместить несколько записей.

info описывает файл (записи, адреса, длина); с `--aci-input stock|mimeo|gen2`, `--volts` и `--series` он моделирует аналоговый вход ACI (раздел 5.8).

convert `INPUT OUTPUT` преобразует между `.txt`, `.bin`, `.wav` и `.aiff`.

12.3 BASIC и ассемблер

```
TERMINAL
apple1-emu basic game.bas -o game.wav          # токенизированный листинг -> кассета
apple1-emu basic game.bas -o game.txt --ram 16 # дамп Woz, для 16 КБ
apple1-emu basic game.bas                      # только проверка
apple1-emu asm game.s -o game.wav              # исходник 6502 -> кассета
apple1-emu asm game.s -o game.bin -l game.lst  # двоичный файл + листинг
apple1-emu asm game.s -I ../lib                # проверка, доп. путь включения
```

`basic` токенизирует листинг так же, как интерпретатор, и записывает блок `$004A-$00FF`, затем программу; `--ram 8|12|16|32` выбирает целевую машину (по умолчанию — самую маленькую из достаточных). `asm` ассемблирует исходник ca65 (или xa); без `-o` только проверяет. Обе команды выводят строку, которую нужно набрать на настоящем Apple-1.

> Вывод apple1-emu asm hello.s -o hello.txt
>
> ```
> 33 octets, $0300-$0320 ; 2 fichier(s), 13 symboles
> Cassette (après C100R) : 0300.0320R   puis 300R
> écrit : hello.txt
> ```

То есть: «33 байта, $0300-$0320; файлов: 2, символов: 13 / Кассета (после C100R): 0300.0320R, затем 300R / записано: hello.txt».

12.4 ASCII-арт

ascii `IMAGE`: `--cols`, `--rows` (23, чтобы сохранить приглашение), `--zoom`, `--cx`, `--cy`, `--brightness`, `--contrast`, `--gamma`, `--invert`, `--dither`, `--no-auto-level`, `--ramp simple|full|slideshow|LIST`, `--format asm|basic`, `--title`, `-o` (`.txt`, `.bas`, `.s`, `.wav`, `.aif`).

12.5 ППЗУ

prom готовит файлы для прошивки настоящих ППЗУ 256 × 4 (82S129, 3601, 6301) в формате дампа по тетрадам:

```
TERMINAL
apple1-emu prom split woz -o proms/     # PROM-A2.bin (D7-D4) и PROM-A1.bin (D3-D0)
apple1-emu prom split aci -o proms/     # PROM-A3.bin (D7-D4) и PROM-A4.bin (D3-D0)
apple1-emu prom join A2.bin A1.bin      # собрать прочитанное и сравнить
```

`--swap-a1-a7` нужен только для платы, разведённой по схеме Apple 1976 года, где адресные линии 1 и 7 ППЗУ A2 перекрещены.

Чтобы прошить EPROM от 2716 до 27C512 или EEPROM 28C из графического интерфейса (программатор TL866II+, T48 или T56 с `minipro`), см. программатор EPROM, раздел 2.6.

12.6 Графический интерфейс

`apple1-emu-gui [OPTIONS] [FILE…]` загружает указанные файлы при запуске. Основные параметры (имена параметров французские):

Параметр	Действие
`--zoom N|ajuster`, `--taille WxH`	масштаб экрана (`ajuster` = по окну), размер окна
`--phosphore vert|blanc|ambre`	люминофор (зелёный, белый, янтарный)
`--screening`, `--sans-screening`, `--screening-options LIST`	винтажный экран Apple-1 и его настройки
`--gen2-screening`	винтажный экран на цветном мониторе GEN2
`--theme clair|sombre|systeme`	светлая, тёмная или системная тема
`--taille-icones auto|petite|normale|grande|tres-grande`	размер значков и текста
`--barre-complete`	никогда не сжимать панель инструментов до одних значков
`--bibliotheque FOLDER`, `--basic FILE`	папка библиотеки, файл BASIC
`--carte-couleur`, `--sans-carte-couleur`, `--couleur-50hz`	карта GEN2
`--langue CODE|auto`	язык интерфейса (`--langue ru` для русского)
`--mode-rapide`, `--mode-cassette`	загрузка из библиотеки: мгновенная (по умолчанию) или через кассету на реальной скорости
`--cassette FILE`	вставляет кассету
`--ecran-separe`, `--debogueur`	экран в отдельном окне, отладчик открыт
`--editeur FILE`, `--ascii FILE`	открывает файл в редакторе или в инструменте ASCII

Для демонстраций и снимков экрана некоторые параметры воспроизводят действия через несколько секунд: `--lancer-biblio S:NAME`, `--charger-biblio S:NAME`, `--parametres S:NAME`, `--preferences S` (и `--onglet-prefs interface|editeur|bibliotheque|cassette|gen2|avance`), `--editeur-lancer S`, `--editeur-recherche TEXT` (строка поиска открыта), `--editeur-menu` и `--ecran-menu` (открыты меню правого щелчка), `--menu-etat` (открыто меню «Состояние»), `--eprom` (открыт программатор EPROM), `--fenetre-fichiers` (открыто окно «Загрузить»), `--editeur-exporter S:FILE`, `--ascii-reglages R`, `--ascii-lancer S`, `--taper TEXT`, `--taper-a S:TEXT`, `--dbg-arret`, `--dbg-surveiller`, `--dbg-commande S:CMD`, `--capture IMAGE.png` (после `--delai S`) и `--quitter`.

ГЛАВА 13

Неполадки и частые вопросы

13.1 Окно не открывается

Нет драйвера OpenGL, или вы работаете в виртуальной машине. `run.sh` автоматически повторяет попытку с программной отрисовкой Mesa; вручную: `LIBGL_ALWAYS_SOFTWARE=1 ./apple1-emu-gui`. Винтажный экран остаётся плавным и при программной отрисовке.

13.2 Клавиши не доходят до Apple-1

Щёлкните по экрану Apple-1. Когда активно текстовое поле или диалоговое окно, клавиатура идёт в это поле; строка состояния показывает, куда идёт клавиатура.

13.3 «Integer BASIC не найден»

BASIC не поставляется. Положите свой `BASIC.wav` (или `.bin` размером 4096 байт) в библиотеку или укажите его кнопкой **BASIC** либо через ⚙ **Настройки → Дополнительно → Integer BASIC**. После этого эмулятор находит его сам.

13.4 Программа на BASIC останавливается с STOPPED AT

Во время её работы была нажата клавиша: на Apple-1 любая клавиша прерывает программу на BASIC. Наберите `RUN`, чтобы начать заново, или `GOTO` с номером строки, чтобы продолжить, не стирая переменные.

13.5 Программа не запускается после загрузки

- Программа на BASIC: нужен `E2B3R` (тёплый вход), а не `E000R`, который стирает программу. BASIC должен быть в памяти **до** программы.
- Машинный код: проверьте адрес запуска в параметрах ⚙; это не всегда `300R` (`280R`, `1000R`, `2000R`…).
- Программа больше 8 КБ: ОЗУ добавляет ▶, но не ручная загрузка. Запустите её через ▶ или используйте `--ram 16`.

13.6 Кассета не загружается на настоящем Apple-1

1. Громкость на максимум, все звуковые эффекты выключены, никаких уведомлений.

2. RETURN — во время заголовка, а не до начала воспроизведения и не после конца заголовка.
3. Точные диапазоны, по четыре цифры: `004A.00FFR`, никогда `4A.FFR`.
4. Попробуйте другой проигрыватель (VLC), затем `--rate 44100`, `--channels 1` или `--no-refresh`.
5. Проверьте уровень командой `apple1-emu info prog.aiff --aci-input stock` (раздел 5.8).
6. Если загрузка не заканчивается: RESET, затем повторите с большей громкостью.

13.7 Экран заполнен инверсными @

Это цветной монитор GEN2: его память начинается с нулей, а код `$00` — это инверсный `@`. Программа ещё не заполнила текстовую страницу. На настоящей карте содержимое было бы случайным.

13.8 Символы выводятся медленно

Это нормально: терминал Apple-1 выводит около 60 символов в секунду. Щёлкните **Вывод: ~60 симв./с** в строке состояния (или **Быстрый** в строке отображения экрана), чтобы вывод стал мгновенным, и переключитесь в быстрый режим (F3), чтобы ускорить всё.

13.9 Звук кассеты не слышен

Он выключен на быстрой скорости, а также при отсутствии ALSA или звуковой карты: осциллограф это сообщает («без звука на максимальной скорости», «нет аудиовыхода»). Громкость задаётся в осциллографе; 🔊 на панели инструментов выключает или включает звук.

13.10 Проект GitHub не собирается

«Проверить последнюю сборку» собирает через ca65/ld65 (пакет `cc65`), если не находит ни Release, ни артефакта. Установите `cc65` или добавьте `apple1.toml`, указывающий нужный файл. Некоторым исходникам нужен свежий ca65 (2.19 или новее); тогда ошибка ca65 показывается как есть.

13.11 Чего эмулятор не делает

- Недокументированные коды операций 6502 обрабатываются как NOP нужной длины, а коды JAM останавливают процессор; их точное действие не воспроизводится (программы для Apple-1 их не используют).
- Компаратор ACI во время эмуляции идеален; только команда `info` моделирует его как аналоговую схему.
- Эмуляция проверена по ПЗУ, листингам и независимым инструментам (c2t, ca65, xa), но автор этого руководства ещё не проигрывал аудиофайлы на настоящий Apple-1.

ГЛАВА 14

Установка ПЗУ или оригинального Integer BASIC

Ничего больше не требуется. Woz Monitor, ПЗУ кассеты ACI и RDK G2 Basic уже в программе. Эта страница для тех, кому нужен Integer BASIC от Apple или другое совместимое ПЗУ из внешнего источника.

Комплект **не распространяет ни одного собственнического файла ПЗУ**. Apple-1, Integer BASIC 1976 года и коммерческие кассеты остаются у правообладателей. Ссылки ниже ведут на уже существующие сайты: свою копию вы берёте там. В комплекте её нет.

14.1 Что уже встроено

- **Woz Monitor**, по адресу `$FF00`.
- **ПЗУ кассеты ACI**, по адресу `$C100`.
- **RDK G2 Basic**, свободный BASIC, совместимый с Integer BASIC; кнопка **BASIC** загружает его в `$E000`.

14.2 Оригинальный Integer BASIC

1. Достаньте свою копию, например кассету `BASIC.wav` из коллекции «Apple1Cassettes» на archive.org: https://archive.org/details/Apple1Cassettes.
2. Положите файл в библиотеку или оставьте его где угодно.
3. Откройте ⚙ **Настройки** → **Дополнительно** → **BASIC**, выберите **оригинальный Integer BASIC**, затем 📂 **Выбрать...** и укажите файл (`.wav`, `.txt` или `.bin` на 4096 байт).
4. Кнопка **BASIC** загрузит этот файл. **Забыть** возвращает встроенный G2 Basic.

Сообщество Apple-1 собирается на https://www.applefritter.com/. Комплект ничего с этих сайтов не скачивает: ссылка — только указатель.

ЧАСТЬ II

Integer BASIC

BASIC Стива Возняка и его свободный клон RDK G2 Basic: все операторы, память, сообщения об ошибках, известные ошибки и способы их обхода, перенос с Applesoft

ГЛАВА 15

Первые шаги в BASIC

Integer BASIC для Apple-1 умещается в 4 КБ. Он знает только целые числа, имена переменных из одной буквы, и в нём нет ни DATA, ни строковых функций: он маленький, быстрый и полон ловушек.

> ▣ **ОТКУДА ВЗЯЛСЯ ЭТОТ BASIC**
>
> Стив Возняк написал его в 1976 году вручную, на языке ассемблера, без компьютера, на котором его можно было бы ассемблировать. Он продавался на кассете (75 $) и загружается в `$E000-$EFFF`. Эмулятор не поставляет оригинал от Apple: если он вам нужен, используйте свой `BASIC.wav` или образ размером 4 КБ (глава 4). Все проверки в этой части выполнены в эмуляторе, с этим оригиналом.

15.1 Два BASIC: RDK G2 Basic и оригинал

Начиная с версии 7.9 у эмулятора есть собственный BASIC — **RDK G2 Basic**, встроенный в программу. Это клон Integer BASIC по методу «чистой комнаты», написанный с нуля на ассемблере 6502 (в нём нет кода интерпретатора Apple) и распространяемый под лицензией MIT. Он посвящён Uncle Bernie.

- **Совместимый**: тот же синтаксис, те же токены, тот же формат программы в памяти, тот же формат кассеты. Программы, листинги и кассеты, написанные для оригинала, работают без изменений. Он сверен с оригиналом на 22 сеансах проверки с одинаковым выводом.
- **Исправленный**: известные ошибки оригинала, описанные в главе 18, исправлены.
- **Больше**: 7 933 байта вместо 4 КБ. Ему нужно ОЗУ от `$E000` до `$FEFF`; эмулятор его даёт, но в окно EPROM размером 4 КБ, как у оригинала, он бы не поместился.
- **Знает GEN2**: при запуске он ищет карту GEN2 и, если находит, запускает драйвер карты (а затем ставит LOMEM в `$2000`, выше графических страниц). Сами графические операторы запланированы для версии 2 G2 Basic.
- **Заставка**: `RDK G2 BASIC 1.0`, затем `DEDICATED TO UNCLE BERNIE`.
- **Расширенный** (версия 7.9.2): `LOMEM=` и `HIMEM=` работают как в оригинале, а новая функция `KEY(n)` читает клавиатуру без ожидания, чтобы игра могла смотреть на клавиши, не блокируясь на `INPUT` и не останавливаясь от нажатия клавиши (глава 15). `LIST a,b` и `DEL a,b` используют диапазоны оригинала, а выводимая строка длиннее экрана переносится так же, как в оригинале.

Кнопка **BASIC** загружает G2 Basic. Чтобы вместо него использовать оригинал Apple, откройте ⚙ **Настройки → Дополнительно → BASIC**, выберите **Оригинальный Integer BASIC (ваш собственный файл, загружаемый в $E000)** и укажите файл; выбор запоминается, и тогда кнопка загружает его. Оба ведут себя одинаково во всём, что описано в этой части; различия отмечены там, где они важны (HIMEM при запуске, ошибки главы 18).

> **ИСХОДНЫЙ КОД И ТЕСТЫ**
> У G2 Basic есть собственный репозиторий, `Old-Wirehead/rdk-gen2basic-apple1`, с его руководством, исходным кодом и регрессионными тестами.

15.2 Запуск BASIC

Щёлкните **BASIC** на панели инструментов: эмулятор загружает выбранный вами BASIC (по умолчанию G2 Basic), а вы запускаете его командой `E000R`. Приглашение BASIC — `>`, приглашение Woz Monitor — `\`.

```
\
E000R
E000: 4C
>
```
WOZ MONITOR

- `E000R`: **холодный запуск**, стирает программу в памяти.
- `E2B3R`: **тёплый запуск**, сохраняет программу. Этот адрес нужен после загрузки программы с кассеты и для возврата в BASIC после захода в монитор (RESET).

E000RE2B3R

15.3 Прямой режим и режим программы

Строка, которая начинается с **числа** (от 0 до 32767), сохраняется в программе; строка без номера выполняется сразу.

```
>PRINT 2*3+4*5
26
>10 PRINT "HELLO"
>20 GOTO 10
>RUN
HELLO
HELLO
…
```
WOZ MONITOR

Нажмите любую клавишу, чтобы остановить программу: BASIC выведет `STOPPED AT 10`. Повторный ввод строки с тем же номером заменяет её; номер без текста удаляет её.

> **! ПРЯМОЙ РЕЖИМ ОГРАНИЧЕН**
> В прямом режиме BASIC отвергает операторы, разделённые `:` (`SYNTAX ERR`), а также `FOR` и `GOSUB` . Чтобы попробовать цикл, запишите его в нумерованной строке.

15.4 Исправление строки

На клавиатуре Apple-1 нет клавиши удаления: **Backspace** эмулятора набирает `_` , что отменяет предыдущий символ (строка на экране не исправляется, но BASIC получает правильную). Esc отменяет всю строку: BASIC выводит `\` и ждёт следующую строку. Для удобной правки используйте редактор (глава 7): он раскрашивает синтаксис, сообщает об ошибках до отправки и набирает программу за вас.

15.5 Первая программа

boucle.bas — INTEGER BASIC

```
10 S=0
20 FOR I=1 TO 10
30 S=S+I
40 NEXT I
50 PRINT "SUM = ";S
60 END
```

WOZ MONITOR

```
>RUN
SUM = 55
>
```

В редакторе ▶ **Запустить** набирает программу и `RUN` ; в отладчике можно следить за ней строка за строкой (рисунок в главе 8).

15.6 Команды

Команда	Действие
LIST `[n[,m]]`	выводит программу, строку `n` или строки с `n` по `m`
RUN `[n]`	очищает переменные и запускает программу, со строки `n` , если она указана
DEL `n[,m]`	удаляет строку `n` или строки с `n` по `m`
SCR	стирает программу и переменные
CLR	очищает переменные, сохраняя программу
AUTO `n[,шаг]`	автоматическая нумерация с `n` (по умолчанию шаг 10); выход — Ctrl + D , затем RETURN , но не Esc
MAN	возвращает ручную нумерацию
HIMEM `=n` , LOMEM `=n`	границы памяти BASIC (глава 16)

`GOTO n` в прямом режиме продолжает остановленную программу, **не** очищая переменные, в отличие от `RUN n`.

> ✦ **LIST И МЕДЛЕННЫЙ ВЫВОД**
>
> Терминал Apple-1 выводит около 60 символов в секунду: вывод длинной программы занимает время. Отключите ограничение вывода (**Быстрый** в строке отображения или в строке состояния), чтобы читать быстрее, или выводите по частям: `LIST 100,200`.

15.7 Три программы для набора

Угадай число

devine.bas — INTEGER BASIC

```
10 REM GUESS A NUMBER FROM 1 TO 100
20 N=RND(100)+1:C=0
30 PRINT "I PICKED A NUMBER FROM 1 TO 100."
40 INPUT "YOUR GUESS",E
50 C=C+1
60 IF E<N THEN PRINT "HIGHER!"
70 IF E>N THEN PRINT "LOWER!"
80 IF E#N THEN 40
90 PRINT "WELL DONE, IN ";C;" TRIES."
100 END
```

WOZ MONITOR

```
>RUN
I PICKED A NUMBER FROM 1 TO 100.
YOUR GUESS?50
HIGHER!
YOUR GUESS?75
LOWER!
YOUR GUESS?62
LOWER!
YOUR GUESS?56
HIGHER!
YOUR GUESS?57
WELL DONE, IN 5 TRIES.
```

`RND(100)` даёт целое от 0 до 99; `#` означает «не равно».

Таблица умножения

table.bas — INTEGER BASIC

```
10 DIM T(10)
20 INPUT "TABLE OF",N
30 FOR I=1 TO 10:T(I)=N*I:NEXT I
40 FOR I=1 TO 10
50 PRINT I;" X ";N;" = ";T(I)
60 NEXT I
70 S=0:FOR I=1 TO 10:S=S+T(I):NEXT I
80 PRINT "TOTAL: ";S
90 END
```

Игры со строками

chaines.bas — INTEGER BASIC

```
10 DIM N$(20),R$(20)
20 INPUT "YOUR NAME",N$
30 L=LEN(N$)
40 PRINT "INITIAL: ";N$(1,1)
50 PRINT "END: ";N$(L-1)
60 R$="":FOR I=L TO 1 STEP -1
70 R$(LEN(R$)+1)=N$(I,I)
80 NEXT I
90 PRINT "BACKWARDS: ";R$
100 IF R$=N$ THEN PRINT "IT'S A PALINDROME!"
110 END
```

WOZ MONITOR

```
>RUN
YOUR NAME?KAYAK
INITIAL: K
END: AK
BACKWARDS: KAYAK
IT'S A PALINDROME!
```

Строки разрезаются через `N$(начало,конец)` и дополняются через `R$(LEN(R$)+1)=…` : нет ни `MID$` , ни `+` для строк (глава 16).

Эти четыре программы лежат в `docs/exemples/en/basic/` : откройте их в редакторе или сделайте кассету командой `apple1-emu basic devine.bas -o devine.aiff` .

ГЛАВА 16

Справочник операторов и функций

Все операторы, которые распознаёт Integer BASIC для Apple-1, взятые из таблицы ключевых слов интерпретатора, с тем, что они делают на самом деле. Каждый пример проверен.

16.1 Операторы

`PRINT список [;]`

PRINT выводит числа и строки. `;` соединяет элементы без пробела (`PRINT A;B` выводит `12` при A=1, B=2); `,` переходит к следующему столбцу с шагом 8 (`PRINT 1,2,3` выводит `1 2 3`). Конечный `;` подавляет перевод строки. `PRINT` без аргументов переходит на следующую строку.

`INPUT ["подсказка",] переменная [,переменная…]`

INPUT читает число или строку. BASIC всегда выводит `?` после подсказки: `INPUT "NAME",N$` выводит `NAME?`. Строка, слишком длинная для своего `DIM`, даёт `STR OVFL ERR`.

`LET переменная = выражение`

LET необязателен: достаточно `A=5`.

`IF выражение THEN оператор | номер`

IF выполняет оператор, когда выражение не равно нулю (допускается `IF A THEN …`). `THEN 100` переходит на строку 100. **Внимание**: от условия зависит только первый оператор (глава 18, ошибка 4).

`GOTO выражение — GOSUB выражение — RETURN`

GOTO и GOSUB принимают **выражение**: `GOTO K*100`, `GOSUB 300+K` («вычисляемый GOTO» заменяет отсутствующий `ON … GOTO`). Несуществующая строка даёт `BAD BRANCH ERR`. Не больше восьми вложенных GOSUB (`>8 GOSUBS ERR` на девятом); `RETURN` без `GOSUB` даёт `BAD RETURN ERR`.

`FOR перем = начало TO конец [STEP шаг] … NEXT перем`

FOR/NEXT: тело выполняется **хотя бы один раз**, даже если начало уже за концом (`FOR I=5 TO 1` проходит один раз). `STEP` может быть отрицательным. Не больше восьми вложенных циклов (`>8 FORS ERR`). `NEXT` должен называть переменную; `NEXT` без `FOR` даёт `BAD NEXT ERR`.

`DIM перем(n) [,перем$(n)…]`

DIM создаёт целочисленный массив из `n` элементов, пронумерованных **от 1 до n** (`A(0)` даёт `RANGE ERR`), или резервирует `n` символов (не больше 255) для строки. Каждую строку нужно объявить до присваивания.

```
REM текст
```

REM: комментарий до конца строки (`:` после REM входит в комментарий).

```
END
```

END завершает программу. Программа, которая заканчивается без `END`, выводит `*** END ERR` : это безвредно, но заканчивайте свои программы оператором `END` .

```
TAB n
```

TAB — это **оператор** (а не функция PRINT, как в Applesoft): `TAB 10:PRINT "*"` выводит звёздочку в столбце 10.

```
POKE адрес,значение
```

POKE записывает байт (от 0 до 255, иначе `>255 ERR`). Адреса выше 32767 записываются отрицательными числами: адрес − 65536.

```
CALL адрес
```

CALL вызывает программу на машинном языке, которая возвращается по `RTS` . Пример: `POKE 768,96 : CALL 768` (96 = `RTS`). `CALL -256` корректно возвращает в Woz Monitor (`$FF00`).

16.2 Функции

Функция	Результат	Проверенный пример
PEEK `(адр)`	байт по адресу	`PEEK(76)+PEEK(77)*256` даёт HIMEM (4096)
RND `(n)`	случайное целое от 0 до n−1; `RND(0)` : `>32767 ERR` ; `RND(-5)` даёт отрицательное число	`RND(6)+1` : игральная кость
SGN `(x)`	−1, 0 или 1 (в том числе для числа 256 и больше)	`SGN(-5);SGN(0)` выводит `-10`
ABS `(x)`	абсолютная величина	`ABS(-9)` = 9
`KEY(n)` (только G2 Basic)	код нажатой клавиши без ожидания (см. ниже)	`K=KEY(0)`
LEN `(a$)`	длина строки	`LEN("ABC")` = 3
USR `(x)`	вызывает `$0000` : без подготовленной программы **машина падает**	—
ASC `(` , SCRN `(`	есть в таблице, но дают `SYNTAX ERR`	—

16.3 Операции

Приоритет	Операции
1	унарные `-` и `+`, `NOT`
2	`*` `/` `MOD`
3	`+` `-`
4	`=` `#` (не равно, также `<>`) `<` `>` `<=` `>=`
5	`AND`
6	`OR`

MOD, AND, OR и NOT:

- Деление **отбрасывает дробную часть в сторону нуля**: `7/2` = 3, `-7/2` = −3. `MOD` берёт знак делимого: `-7 MOD 3` = −1.
- Сравнения дают 1 (истина) или 0 (ложь): `PRINT 5>3;5=3` выводит `10`. `NOT 0` = 1, `NOT 5` = 0.
- `AND` и `OR` логические, а не побитовые: `3 AND 0` = 0, `3 OR 0` = 1.
- `3*-2` принимается и равно −6.
- `^` есть в таблице, но **вешает машину** (глава 18, ошибка 2).
- Строки можно сравнивать только через `=` и `#`; `"ABC"<"ABD"` отвергается (`SYNTAX ERR`).

16.4 Расширения RDK G2 Basic

Они работают только с **RDK G2 Basic** (оригинальный Integer BASIC отвечает `SYNTAX ERR`).

`KEY(n)`

`KEY(0)` возвращает **код клавиши**, которая была нажата (7-битный код ASCII: 65 для `A`, 13 для Return, 32 для пробела), и забирает его, или `0`, если клавиша не нажималась. Она никогда не ждёт, поэтому программа может работать и следить за клавиатурой. `KEY(1)` (любой другой ненулевой аргумент) **ждёт** клавишу.

Ждать клавишу без INPUT

INTEGER BASIC

```
10 K=KEY(0)
20 IF K=0 THEN 10
30 PRINT K
```

В оригинале клавиша, нажатая во время работы программы, **останавливает** её (`STOPPED AT n`), а `INPUT` блокируется до Return. В G2 Basic, как только программа вызвала `KEY`, нажатие клавиши её больше не останавливает: клавиатуру видит только `KEY`, а Ctrl+C (код 3), прочитанный `KEY`, останавливает программу при следующем вызове. Поэтому вызывайте `KEY` в главном цикле программы.

`LOMEM= n` — `HIMEM= n`

Непосредственные команды, принимающие выражение (`LOMEM=2048+512`); внутри программы они дают `SYNTAX ERR`, как в оригинале. Они описаны вместе с памятью в главе 16.

```
LIST [a[,b]] — DEL a[,b]
```

`LIST` выводит всё, `LIST n` — одну строку, `LIST a,b` — строки от `a` до `b`. `DEL n` удаляет одну строку, `DEL a,b` — строки от `a` до `b` (если `b` меньше `a`, только строку `a`). Выводимая строка длиннее экрана переносится как в оригинале (строки продолжения сдвигаются на шесть пробелов).

Срезы строк следуют оригиналу: `A$(I)` при `I` от 256 и больше или отрицательном даёт `>255 ERR`, а `A$(I,J)` при `J` меньше `I-1` даёт `STRING ERR`.

16.5 Ключевые слова, которые ничего не делают

В таблице есть `COLOR=`, `PLOT`, `HLIN`, `VLIN` и `VTAB`, унаследованные от проекта BASIC для Apple II. На Apple-1 `COLOR=` и `PLOT` принимаются и ничего не делают, `VTAB` даёт `SYNTAX ERR`. Не используйте их в своих программах.

ГЛАВА 17

Переменные, строки, массивы и память

17.1 Числа

Integer BASIC знает только **16-битные целые со знаком**, от −32767 до 32767. Вычисление, выходящее за этот диапазон, останавливается с `*** >32767 ERR` (`200*200`, `32767+1`, а также `10/0` и `RND(0)`). Значение −32768 использовать нельзя (глава 18, ошибка 5). Нет ни десятичной точки, ни `INT`: `7/2` равно 3.

17.2 Имена переменных

Это ограничение удивляет больше всего: имя переменной — это **одна буква, за которой может следовать одна цифра**.

Имя	Результат (проверено)
`A`, `Z`, `B1`, `Z9`	принимается
`AB`, `CNT`, `SCORE`, `COUNTER`	`SYNTAX ERR`
`A12`	`SYNTAX ERR`
`A$`, `N$`	строка: только **одна буква**
`B9$`, `Z9$`	`SYNTAX ERR`

Это даёт 26 × 11 = 286 возможных целочисленных переменных и 26 строк. Возьмите за привычку записывать назначение каждой переменной в `REM`.

17.3 Строки

Строку нужно **объявить** до использования, указав её наибольшую длину (от 1 до 255): `DIM N$(20)`. `DIM B$(256)` даёт `>255 ERR`.

Операция	Запись	Пример (N$ = «KAYAK»)
длина	`LEN(N$)`	5
подстрока	`N$(i,j)` : символы с i по j	`N$(1,3)` = «KAY»
хвост	`N$(i)` : с символа i до конца	`N$(4)` = «AK»
сцепление	`A$(LEN(A$)+1)=B$`	«APPLE» + «-1» → «APPLE-1»
сравнение	только `=` и `#`	`IF N$="WOZ" THEN …`

Присваивание строки длиннее её `DIM` даёт `*** STR OVFL ERR`, как и слишком длинный ответ на `INPUT`. Нет ни `MID$`, ни `LEFT$`, ни `CHR$`, ни `STR$`, ни `VAL`, ни `ASC` (последний есть в таблице, но не работает).

> ✦ **ЗАМЕНА CHR$ И VAL**
>
> Чтобы вывести символ по его коду, держите строку-образец и вырезайте из неё: `DIM C$(26):C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"`, затем `PRINT C$(K,K)`. Чтобы прочитать число, используйте `INPUT N`, а не `INPUT N$`.

17.4 Массивы

`DIM T(10)` создаёт массив из 10 целых, **пронумерованных от 1 до 10**. `T(0)` и `T(11)` дают `*** RANGE ERR`. Измерение только одно: для сетки 8 × 8 используйте `G((L-1)*8+C)`. В эмуляторе после `DIM` элементы равны 0; на настоящей машине память не обязательно очищена: инициализируйте массивы. Массив, слишком большой для памяти, даёт `*** MEM FULL ERR` (`DIM A(3000)` в 8 КБ).

17.5 Память BASIC

Адреса	Содержимое	Примечания
FF00–FFFF	**Woz Monitor** (ППЗУ 256 байт)	векторы NMI, RESET, IRQ
E000–EFFF	**Integer BASIC** (4 КБ ОЗУ)	загружается с кассеты
D010–D013	**PIA 6821**: клавиатура, экран	KBD, KBDCR, DSP, DSPCR
C250–C257	**Карта GEN2**: программные переключатели	необязательна, доступ чтением
C100–C1FF	**ППЗУ ACI**(C100R)	C000–C0FF: триггер
1000–BFFF	дополнительное ОЗУ (до 48 КБ)	HIRES 2000–5FFF (GEN2)
0800–0FFF	программы BASIC (LOMEM–HIMEM)	текст/LORES 2: 0800
0300–07FF	ваши программы в машинных кодах	текст/LORES 1: 0400
0200–027F	буфер ввода монитора	не занимать
0100–01FF	стек 6502	
0000–00FF	нулевая страница	монитор 24–2B, BASIC 4A–FF

Рисунок 32 — Память Apple-1: BASIC в $E000, программа ниже HIMEM, переменные выше LOMEM.

BASIC хранит **программу вверху** памяти, растущей вниз от HIMEM, а **переменные внизу**, растущими вверх от LOMEM. Свободное место — между ними.

Указатель	Адрес	Значение в 8 КБ	Роль
LOMEM	`$4A-$4B` (74-75)	`$0800` (2048)	начало переменных
HIMEM	`$4C-$4D` (76-77)	`$1000` (4096)	конец памяти BASIC
PP	`$CA-$CB` (202-203)	опускается вместе с программой	начало программы
PV	`$CC-$CD` (204-205)	поднимается вместе с переменными	конец переменных

`PRINT PEEK(202)+PEEK(203)*256` даёт начало программы, а `PRINT PEEK(204)+PEEK(205)*256` — конец переменных: свободное место — разность между ними. Целочисленная переменная вроде `A` занимает 6 байт.

▣ **ПОЧЕМУ КАССЕТА BASIC НАЧИНАЕТСЯ С $004A**

Эти указатели находятся в нулевой странице, с `$4A` по `$FF`. Поэтому кассета BASIC записывает **два диапазона**: `004A.00FF` (указатели), затем программу от PP до HIMEM, например `0F2C.0FFF`. Команда `apple1-emu basic` вычисляет их за вас (её вывод — на французском: «10 строк, 212 байт… 1836 байт свободно для переменных… диапазоны ACI (после C100R)… затем E2B3R и RUN»):

```
10 lignes, 212 octets ; LOMEM $0800 HIMEM $1000 PP $0F2C ; 1836 octets libres pour
les variables ; RAM 8 Ko
Plages ACI (après C100R) : 004A.00FFR 0F2C.0FFFR   puis E2B3R et RUN
```

Изменение LOMEM и HIMEM

Синтаксис — `LOMEM=n` и `HIMEM=n` (со знаком `=`, **а не** `:`, как на Apple II, что даёт `SYNTAX ERR`).

- `LOMEM=2304` сдвигает начало переменных и **очищает переменные** (программа остаётся). Полезно, чтобы зарезервировать `$0800-$08FF` под программу на машинном языке.
- `HIMEM=` **выше** (при большем ОЗУ, например `HIMEM=8192` в 16 КБ) работает и сохраняет программу.
- `HIMEM=` **ниже молча игнорируется** оригиналом, даже после `SCR` (глава 18, ошибка 11). **RDK G2 Basic его принимает**: программа в памяти сдвигается вниз вместе с границей, или выдаётся `MEM FULL ERR`, если она уже не помещается над переменными (`RANGE ERR` для отрицательного значения; `LOMEM=` даёт `RANGE ERR`, если оно не ниже начала программы, и `>32767 ERR` выше 32767). С оригиналом есть проверенный обход: записать указатель напрямую, затем `SCR`:

Понижение HIMEM до 3072 ($0C00) — INTEGER BASIC

```
POKE 76,0 : POKE 77,12 : SCR
```

При запуске G2 Basic HIMEM — это верх найденного ОЗУ, а LOMEM равен `$0800` (`$2000` с драйвером GEN2). (`76` / `77` = `$4C` / `$4D`, 12 × 256 = 3072.) Наберите эти три оператора в нумерованной строке или по отдельности в прямом режиме, **до** ввода программы: `SCR` её стирает.

17.6 Смешивание BASIC и машинного языка

Область (8 КБ)	Годится для кода 6502?
`$0000-$00FF`	нулевая страница: Woz Monitor использует `$24-$2B`, BASIC — как минимум `$4A-$FF`; избегайте её или сохраняйте и восстанавливайте
`$0100-$01FF`	стек 6502: нет
`$0200-$027F`	буфер строки Woz Monitor; перезаписывается каждой набранной строкой
`$0280-$07FF`	свободна, пока BASIC её не трогает: лучшее место
`$0800-…`	переменные BASIC (если LOMEM не поднят)
`… -$0FFF`	программа на BASIC

Программа на машинном языке загружается с кассеты или через POKE и вызывается через `CALL 768` (для `$0300`). Она должна заканчиваться `RTS` и сохранять нулевую страницу BASIC. Минимальный пример:

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(96 = `$60` = `RTS`: вызов сразу возвращается.)

ГЛАВА 18

Сообщения об ошибках и ограничения

18.1 Сообщения

Ошибка выводит `*** СООБЩЕНИЕ ERR`, а если работает программа — затем `STOPPED AT n`. Строка, отвергнутая при вводе, не сохраняется: наберите её заново.

Сообщение	Причина	Проверенный пример
SYNTAX ERR	неизвестный оператор, недопустимое имя переменной, `:` в прямом режиме	`CNT=7`, `GET A$`, `X=5:PRINT X` в прямом режиме
>32767 ERR	переполнение, деление на ноль, номер строки > 32767	`200*200`, `10/0`, `RND(0)`, `32768 PRINT`
TOO LONG ERR	строка слишком сложная (слишком глубоко вложенное выражение) или слишком длинная	13 вложенных скобок
MEM FULL ERR	между переменными и программой больше нет места	`DIM A(3000)` в 8 КБ
TOO MANY PARENS ERR	скобки (предусмотрено интерпретатором; маскируется ошибкой 3)	—
STRING ERR	неправильное использование строки	—
END ERR	программа заканчивается без `END`	`10 PRINT "X"`, затем `RUN`
BAD BRANCH ERR	`GOTO` / `GOSUB` на несуществующую строку	`GOTO 500`
>8 GOSUBS ERR	больше 8 вложенных `GOSUB`	рекурсия: 9 вызовов
BAD RETURN ERR	`RETURN` без `GOSUB`	`RUN 30` на строке с `RETURN`
>8 FORS ERR	больше 8 вложенных `FOR` или ошибка 1	9 циклов в одной строке
BAD NEXT ERR	`NEXT` без соответствующего `FOR`	один `20 NEXT I`
STOPPED AT	во время работы была нажата клавиша	—
>255 ERR	значение > 255: `POKE`, `DIM` строки	`POKE 768,300`, `DIM B$(256)`
RANGE ERR	индекс массива вне границ	`A(0)`, `A(3)` после `DIM A(2)`
DIM ERR	массив или строка объявлены повторно	—
STR OVFL ERR	строка длиннее своего `DIM`	`A$="ABCDE"` при `DIM A$(3)`
RETYPE LINE	недопустимый ответ на `INPUT` (буквы вместо числа): наберите заново	—

Строки с «—» взяты из таблицы сообщений интерпретатора; в наших проверках они не вызывались.

18.2 Ограничения

Ограничение	Значение
целые	от −32767 до 32767
номера строк	от 0 до 32767
имена переменных	буква + необязательная цифра
имена строк	одна буква + `$`
длина строки	255
измерения массива	1, индексы от 1 до n
вложенные `FOR`	8
вложенные `GOSUB`	8
вложенные скобки	12
длина набранной строки	120 символов проходят, 141 отвергается

18.3 Экономия памяти

В 8 КБ у BASIC около 2 КБ на программу и переменные. Несколько способов выиграть место:

1. **Опустите LOMEM**, если у вас нет машинного кода: `LOMEM=768` (`$0300`) даёт переменным ещё 1280 байт. Проверено: `DIM T(1200)` завершается `MEM FULL` с LOMEM по умолчанию и проходит после `LOMEM=768`. Так и поступают переносы игр из *101 BASIC Computer Games*, сделанные во время разработки.
2. **Пишите кратко**: BASIC хранит пробелы и `REM`. `LIST` сам расставляет отступы, а одна длинная строка с несколькими операторами обходится дешевле, чем несколько строк.
3. **Используйте массивы и строки повторно**, а не объявляйте новые.
4. **Добавьте ОЗУ**: параметры ⚙ программы, `--ram 16` или настоящее расширение. Программе больше 8 КБ нужна машина на 12, 16 или 32 КБ: `apple1-emu basic --ram 16` размещает её в верхней части этой памяти.

ГЛАВА 19

Известные ошибки и способы их обхода

Integer BASIC для Apple-1 был написан в 1976 году без ассемблера и никогда не исправлялся. Вот его известные дефекты, каждый воспроизведён в эмуляторе с настоящим BASIC, и способы их избежать.

ПРОВЕРЕНО: воспроизведено в эмуляторе с настоящим Integer BASIC, есть протокол. НЕ ПРОВЕРЕНО: сообщено источником, но не воспроизведено. Ошибка n соответствует разделу 18.n. Поскольку эмулятор выполняет исходный код инструкция за инструкцией, ошибка, воспроизведённая здесь, существует и на настоящем Apple-1.

19.1 Выход из цикла FOR через GOTO ПРОВЕРЕНО

Цикл, покинутый через `GOTO`, остаётся «открытым». Если программа снова проходит через `FOR`, в стек кладётся новая запись; на девятом проходе: `>8 FORS ERR`.

INTEGER BASIC

```
10 N=0
20 FOR I=1 TO 10
30 IF I=2 THEN GOTO 50
40 NEXT I
50 N=N+1:IF N<20 THEN 20
60 PRINT "DONE ";N
70 END
```

WOZ MONITOR

```
>RUN
*** >8 FORS ERR
STOPPED AT 20
```

✦ ОБХОД

Никогда не выходите из `FOR` через `GOTO`: присвойте переменной конечное значение и пройдите через `NEXT`. Здесь: `30 IF I=2 THEN I=10:GOTO 40`. Тогда программа выводит `DONE 20`. Переносы 101 BASIC *Computer Games* следуют этому правилу.

19.2 Операция ^ вешает машину ПРОВЕРЕНО

`^` (степень) есть в таблице интерпретатора, но `PRINT 2^3` никогда не возвращается: выйти можно только через RESET.

> ✦ ОБХОД
>
> Умножайте: `2*2*2` или цикл `P=1:FOR I=1 TO N:P=P*X:NEXT I`.

19.3 Сообщение «PPED AT» ПРОВЕРЕНО

Когда стек операндов интерпретатора переполняется (слишком глубоко вложенное выражение в прямом режиме), BASIC должен вывести сообщение об ошибке, но указывает в таблицу сообщений **на 3 байта дальше** (дизассемблированный листинг Эрика Смита отмечает это: `ldy #erri_stopped_at+3 ; The "+3" is a bug!`). Вы читаете конец `STOPPED AT`:

WOZ MONITOR

```
>PRINT 1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1+(1))))))))))))
14
*** PPED AT  ERR
STOPPED AT 65535
```

> ✦ ОБХОД
>
> Разбивайте длинные выражения с помощью промежуточных переменных. Свыше 12 уровней скобок строка программы в любом случае отвергается (`TOO LONG ERR`).

19.4 IF … THEN управляет только первым оператором ПРОВЕРЕНО

В `IF A=2 THEN PRINT "X":PRINT "Y"` от условия зависит только `PRINT "X"`: `PRINT "Y"` выполняется **всегда** (при A=1 программа выводит `Y`). Это самая частая ловушка при переносе программы, написанной для другого BASIC.

> ✦ ОБХОД
>
> Обратите условие и перепрыгните через блок:
>
> INTEGER BASIC
>
> ```
> 80 IF A#2 THEN 90
> 85 PRINT "X":PRINT "Y"
> 90 REM CONTINUE
> ```

19.5 −32768 ПРОВЕРЕНО

`X=-32767-1` выводит `-`, затем `*** >32767 ERR`. Наименьшее допустимое значение — −32767.

✦ ОБХОД

Оставайтесь в пределах −32767 … 32767. Чтобы прочитать 16-битное слово через `PEEK`, разделите его: `PEEK(A)+PEEK(A+1)*256` переполняется выше 32767; сначала проверьте старший байт (`IF PEEK(A+1)>127 THEN …`).

19.6 ASC(не работает ПРОВЕРЕНО

`ASC("A")` есть в таблице ключевых слов, но любая строка с ним отвергается: `SYNTAX ERR`. То же самое с `SCRN(`.

✦ ОБХОД (ПРОВЕРЕНО)

Ищите символ в строке-образце. Строка 60 закрывает цикл (`K=26:NEXT K`), чтобы избежать ошибки 1:

INTEGER BASIC

Позиция буквы без ASC

```
10 DIM C$(26),L$(1)
20 C$="ABCDEFGHIJKLMNOPQRSTUVWXYZ"
30 INPUT "LETTER",L$
40 FOR K=1 TO 26:IF C$(K,K)=L$ THEN 60:NEXT K
50 PRINT "NOT A LETTER":END
60 PRINT "POSITION ";K:K=26:NEXT K
70 GOTO 30
```

WOZ MONITOR

```
LETTER?W
POSITION 23
LETTER?E
POSITION 5
LETTER?5
NOT A LETTER
```

19.7 Графические ключевые слова, которые ничего не делают

ПРОВЕРЕНО

`COLOR=`, `PLOT` и `HLIN` принимаются и ничего не делают; `VLIN` и `VTAB` дают `SYNTAX ERR`. Это остатки BASIC, предназначенного для Apple II.

✦ ОБХОД

Чтобы расположить текст, используйте `TAB n` и пустые `PRINT`. Чтобы рисовать в цвете, карта GEN2 управляется через `POKE` (глава 6).

19.8 Ошибка 251 символа Apple II ОТСУТСТВУЕТ

Более поздняя версия Integer BASIC (Apple II) падает на некоторых строках длиной 251 символ и больше. На Apple-1 набираемая строка ограничена задолго до этого (около 128 символов): ошибка возникнуть не может.

19.9 STOPPED AT 65535 ПРОВЕРЕНО

За ошибкой, вызванной строкой, **отвергнутой при вводе** (слишком глубокая вложенность), или ошибкой 3 в прямом режиме следует `STOPPED AT 65535`: BASIC считает, что находится в программе, и выводит бессмысленный номер строки. Последствий нет; программа в памяти не затрагивается.

19.10 USR роняет машину ПРОВЕРЕНО

`X=USR(1)` переходит на `$0000`, где обычно нет кода: машина зависает. RESET, затем `E2B3R` возвращают в BASIC без потери программы.

> ✦ ОБХОД
>
> Вызывайте свои программы на машинном языке через `CALL адрес`; передавайте значения через `POKE` и `PEEK`.

19.11 HIMEM= не может опуститься ПРОВЕРЕНО

`HIMEM=3072` (и любое значение ниже текущего) **молча игнорируется**, даже после `SCR`, потому что интерпретатор сравнивает новое значение с указателем программы, прежде чем его принять.

> ✦ ОБХОД (ПРОВЕРЕНО)
>
> `POKE 76,0 : POKE 77,12 : SCR` устанавливает HIMEM в 3072; следующая программа сохраняется ниже. См. главу 16.

19.12 Двоеточие отвергается в прямом режиме ПРОВЕРЕНО

`X=5:Y=X*2:PRINT Y`, набранное без номера, даёт `SYNTAX ERR`; та же строка с номером работает. `FOR` и `GOSUB` в прямом режиме тоже отвергаются.

> ✦ ОБХОД
>
> Набирайте по одному оператору в строке или напишите временную строку `0` и `RUN 0` ... не забыв её удалить (`DEL 0`).

19.13 INPUT всегда добавляет «?» ПРОВЕРЕНО

`INPUT "AGE ? ",N` выводит `AGE ? ?`: BASIC добавляет свой вопросительный знак. Пишите подсказку без `?`: `INPUT "AGE",N` выводит `AGE?`.

19.14 Цикл FOR выполняется хотя бы раз ПРОВЕРЕНО

`FOR I=5 TO 1 … NEXT I` выполняет тело один раз (при I=5), тогда как другие BASIC его пропускают.

> ✦ ОБХОД
> Проверяйте заранее: `IF N<1 THEN 100` перед `FOR I=1 TO N`.

19.15 Нажатие клавиши останавливает программу ПРОВЕРЕНО

Любая клавиша, нажатая во время работы программы, останавливает её (`STOPPED AT`), кроме как во время `INPUT`. Это не ошибка, а проектное решение: клавиши BREAK нет. Следствие: игра в реальном времени не может ждать клавишу, не останавливаясь. Чтение клавиатуры через `PEEK(-12271)` (`$D011`) и `PEEK(-12272)` (`$D010`) кажется возможным, но интерпретатор тоже проверяет клавиатуру между операторами и может остановиться раньше: мы это не пробовали (НЕ ПРОВЕРЕНО). Для игры в реальном времени переходите на машинный язык (часть III).

19.16 Сводка

№	Ошибка	Статус	Обход
1	выход из FOR через GOTO → `>8 FORS`	**проверено**	завершать цикл через NEXT
2	`^` вешает	**проверено**	умножать
3	«PPED AT» (+3)	**проверено**	промежуточные переменные
4	`IF…THEN a:b` выполняет b	**проверено**	обратное условие + переход
5	−32768	**проверено**	оставаться ≥ −32767
6	`ASC(`, `SCRN(`	**проверено**	строка-образец
7	COLOR=, PLOT, HLIN ничего не делают; VLIN, VTAB отвергаются	**проверено**	TAB, POKE (GEN2)
8	251 символ (Apple II)	**отсутствует**	—
9	STOPPED AT 65535	**проверено**	безвредно
10	USR роняет	**проверено**	CALL
11	HIMEM= не опускается	**проверено**	POKE 76/77 + SCR
12	`:` отвергается в прямом режиме	**проверено**	один оператор в строке
13	INPUT добавляет «?»	**проверено**	подсказка без «?»
14	FOR выполняется один раз	**проверено**	проверять заранее
15	чтение клавиатуры через PEEK во время RUN	**не проверено**	машинный язык

ГЛАВА 20

Перенос программы с Applesoft или Microsoft BASIC

Большинство листингов 1970-х и 1980-х годов написаны на Microsoft BASIC (Applesoft, BASIC из «101 BASIC Computer Games»). Их можно перенести на Apple-1, если знать соответствия. Каждая «отвергнутая» строка ниже была набрана в настоящем BASIC.

20.1 Соответствия

Applesoft / Microsoft	Integer BASIC Apple-1	Примечание
`ON X GOTO 100,200,300`	`GOTO 100*X` (если номера строк позволяют) или серия `IF X=1 THEN 100`	отвергается
`GET A$`	`INPUT A$`	отвергается
`DATA` / `READ` / `RESTORE`	массив, заполненный присваиваниями, или нарезаемая строка	отвергается
`MID$(A$,I,N)`	`A$(I,I+N-1)`	отвергается
`LEFT$(A$,N)` / `RIGHT$(A$,N)`	`A$(1,N)` / `A$(LEN(A$)-N+1)`	—
`A$+B$`	`A$(LEN(A$)+1)=B$`	—
`CHR$(N)` , `ASC(A$)`	строка-образец (глава 18, ошибка 6)	отвергается
`STR$(N)` , `VAL(A$)`	прямо `PRINT N` ; `INPUT N`	отвергается
`INT(X)`	не нужно: всё целое	отвергается
`RND(1)` (вещественное от 0 до 1)	`RND(N)` (целое от 0 до N−1)	—
`X^2`	`X*X`	вешает (ошибка 2)
`HOME`	24 × `PRINT` или CLEAR SCREEN (Ctrl+L) вручную	отвергается
`PRINT TAB(10);"X"`	`TAB 10:PRINT "X"`	—
`STOP`	`END`	отвергается
`DEF FN`	`GOSUB`	—
`A<>B`	`A#B` (`<>` тоже принимается)	—
`IF A THEN X:Y`	обратное условие + переход (ошибка 4)	ловушка
`DIM A(10,10)`	`DIM A(121)` и `A(I*11+J+1)`	—
`A(0)`	индексы начинаются с 1	`RANGE ERR`
`SCORE` , `NAME$`	`S` , `N$` : одна буква (+ цифра)	`SYNTAX ERR`
вещественные числа	целые ×10 или ×100 (фиксированная точка)	—

20.2 Метод

1. **Составьте список переменных** исходной программы и назначьте каждой букву (+ цифру). Запишите таблицу в `REM` в начале.
2. **Замените DATA** присваиваниями в подпрограмме инициализации или строкой, которую вы нарезаете.
3. **Найдите все** `IF … THEN … : …` и перепишите их с переходом (ошибка 4).

4. **Найдите все выходы из `FOR` через `GOTO`** и замените их на «конечное значение + `NEXT`» (ошибка 1).
5. **Сдвиньте индексы массивов** на 1, если программа использует индекс 0.
6. **Проверьте в редакторе**: он помечает слова, неизвестные Integer BASIC, прямо при наборе. ▶ **Запустить** вводит программу: любая отвергнутая строка появляется на экране с `SYNTAX ERR`.
7. **Измерьте память**: `apple1-emu basic prog.bas` показывает размер и нужное ОЗУ. Слишком велика для 8 КБ? `LOMEM=768` (глава 17) или машина на 16 КБ.
8. **Экспортируйте** кассету (📼 **Экспорт WAV**) и запишите диапазоны для настоящей машины.

20.3 Пример переноса

Типичный фрагмент на Microsoft BASIC:

Оригинал (Microsoft BASIC)

```
100 READ N$,S
110 IF S>HI THEN HI=S:BEST$=N$
120 IF N$<>"END" THEN 100
130 PRINT "BEST: ";BEST$;" (";HI;")"
140 DATA "ANNE",12,"BOB",17,"CLAIRE",9,"END",0
```

На Apple-1: нет `READ` / `DATA`, нет длинных имён, а `IF` в строке 110 присваивал бы `BEST$` на каждом проходе (ошибка 4).

Перенос на Integer BASIC (проверено) — INTEGER BASIC

```
10 DIM N$(10),B$(10),S(3)
20 S(1)=12:S(2)=17:S(3)=9
30 H=0
40 FOR I=1 TO 3
50 GOSUB 200
60 IF S(I)<=H THEN 80
70 H=S(I):B$=N$
80 NEXT I
90 PRINT "BEST: ";B$;" (";H;")"
100 END
200 IF I=1 THEN N$="ANNE"
210 IF I=2 THEN N$="BOB"
220 IF I=3 THEN N$="CLAIRE"
230 RETURN
```

WOZ MONITOR

```
>RUN
BEST: BOB (17)
```

20.4 Уроки переносов

Восемь игр из 101 BASIC *Computer Games* (DEC, 1975) были перенесены так во время разработки эмулятора: BAGLES, MUGWMP, HEX, BULCOW, STARS, TRAP, CHOMP и WAR. Каждая была сыграна в эмуляторе, экспортирована на кассету и перезагружена через ПЗУ ACI. Эти переносы не распространяются (исходные листинги несвободны), но они задали правила этой главы: переменные из одной буквы, массивы с 1, никаких `READ` / `DATA`, никаких `GOTO` из `FOR`, только один оператор под `IF` и `LOMEM=768` для самых больших.

ЧАСТЬ III

Ассемблер 6502

Процессор, встроенный ассемблер и его инструменты, ввод-вывод и подпрограммы Woz Monitor, примеры с комментариями и советы

ГЛАВА 21

6502 вкратце

6502 в Apple-1 — 8-битный процессор с частотой 1,023 МГц, тремя регистрами, 56 инструкциями и адресным пространством 64 КБ. Эта глава даёт самое необходимое, чтобы читать и писать примеры этой части.

21.1 Регистры

Регистр	Размер	Роль
A	8 бит	аккумулятор: арифметика, сравнения, чтение и запись
X, **Y**	8 бит	индексные регистры: счётчики циклов, смещение в таблице
S	8 бит	указатель стека; стек занимает страницу `$0100-$01FF`
PC	16 бит	счётчик команд: адрес следующей инструкции
P	8 бит	флаги N V – B D I Z C

Флаги обновляются почти каждой инструкцией:

`N (отрицательный)`
: копия бита 7 результата; проверяется `BMI` / `BPL`.

`V (переполнение)`
: знаковое переполнение при сложении или вычитании; `BVS` / `BVC`.

`Z (ноль)`
: результат равен нулю; `BEQ` / `BNE`.

`C (перенос)`
: перенос при сложении, отсутствие заёма при вычитании, выдвинутый бит; `BCS` / `BCC`.

`D (десятичный)`
: `ADC` и `SBC` работают в BCD. Woz Monitor выполняет `CLD` при запуске.

`I (прерывания)`
: маскирует IRQ. У стандартного Apple-1 их нет.

21.2 Режимы адресации

Режим	Синтаксис	Байты	Пример	Действие
неявный	`INX`	1	`inx`	X ← X + 1
аккумулятор	`ASL A`	1	`asl a`	сдвигает A
непосредственный	`#знач`	2	`lda #$8D`	A ← $8D
нулевая страница	`zp`	2	`lda $06`	A ← память $0006
нулевая страница, X	`zp,X`	2	`lda $06,x`	
абсолютный	`адр`	3	`lda $D010`	
абсолютный, X / Y	`адр,X`	3	`lda message,x`	индексированная таблица
косвенный индексный	`(zp),Y`	2	`lda ($06),y`	16-битный указатель в нулевой странице плюс Y
индексный косвенный	`(zp,X)`	2	`lda ($06,x)`	таблица указателей
относительный	`метка`	2	`bne loop`	переход на −128…+127 байт
косвенный	`(адр)`	3	`jmp ($FFFC)`	только `JMP`

Нулевая страница (`$0000-$00FF`) — самая ценная: её инструкции короче и быстрее, и только этот режим позволяет указатели `(zp),Y` . Woz Monitor использует только `$24-$2B` ; Integer BASIC — гораздо больше (глава 16).

21.3 Инструкции по группам

Группа	Инструкции
загрузка, сохранение	`LDA LDX LDY STA STX STY`
пересылки	`TAX TAY TXA TYA TSX TXS`
стек	`PHA PLA PHP PLP`
арифметика	`ADC SBC` (с переносом: `CLC` перед `ADC` , `SEC` перед `SBC`)
инкременты	`INC INX INY DEC DEX DEY`
логика	`AND ORA EOR BIT`
сдвиги	`ASL LSR ROL ROR`
сравнения	`CMP CPX CPY` (C = 1, если регистр ≥ значения, Z = 1 при равенстве)
ветвления	`BEQ BNE BCC BCS BMI BPL BVC BVS`
переходы	`JMP JSR RTS RTI BRK`
флаги	`CLC SEC CLD SED CLI SEI CLV`
ничего	`NOP`

✦ БЕЗЗНАКОВОЕ СРАВНЕНИЕ

После `CMP #n` `BCC` переходит, если A < n, а `BCS` — если A ≥ n. Для «A > n» сравнивайте с n + 1: так и делают примеры с `cmp #'9' | $80 + 1`.

21.4 NMOS 6502, а не 65C02

В Apple-1 стоит исходный 6502 (NMOS), и в эмуляторе тоже. Недокументированные коды операций выполняются как `NOP` нужной длины, а коды `JAM` останавливают процессор. Ассемблер принимает инструкции 65C02 после `.setcpu "65C02"` (`BRA`, `STZ`, `PHX`, `PLX`, `PHY`, `PLY`, `INC A`...) для платы с 65C02, но в эмуляторе они **не работают**:

c02.s (тест) 6502

```
        .setcpu "65C02"
        .org $0300
        lda #$C1        ; "A"
        bra next        ; on an NMOS 6502: 2-byte NOP
        lda #$D8        ; "X"
next:   jsr $FFEF
        jmp $FF1F
```

Эта программа выводит `A` на 65C02 и `X` в эмуляторе: там `BRA` ($80) — это `NOP`, пропускающий свой операнд, и выполняется `LDA #$D8`.

21.5 Карта памяти с первого взгляда

Адреса	Содержимое
`$0000-$00FF`	нулевая страница (Woz Monitor: `$24-$2B`; BASIC: гораздо больше)
`$0100-$01FF`	стек
`$0200-$027F`	буфер строки Woz Monitor
`$0280-$1FFF`	свободно: ваши программы, часто с `$0280` или `$0300` (по умолчанию в эмуляторе 8 КБ ОЗУ; 4 КБ, до `$0FFF`, на стандартном Apple-1)
`$C000-$C0FF`	карта ACI (кассета); `$C100-$C1FF`: её программа
`$C250-$C257`	переключатели карты GEN2
`$D010-$D013`	PIA: клавиатура и дисплей
`$E000-$EFFF`	Integer BASIC (если загружен)
`$FF00-$FFFF`	Woz Monitor

Полная карта приведена в приложении A.

ГЛАВА 22

Ассемблер devkit

Эмулятор содержит полноценный двухпроходный ассемблер 6502, совместимый с ca65. Он используется в редакторе, который ассемблирует при каждом нажатии клавиши, в командной строке и для проектов GitHub. Устанавливать ничего не нужно.

22.1 Рабочий цикл



Рисунок 33 — Рабочий цикл на ассемблере: написать, ассемблировать, запустить, отладить, затем экспортировать на настоящий Apple-1.

1. **Написать**: 📝 **Редактор**, затем **Новый** или **Открыть** файл `.s`, `.asm` или `.a65`. Подсветка синтаксиса различает инструкции, директивы, метки, числа и комментарии.
2. **Ассемблировать**: делать ничего не нужно, ассемблирование следует за каждым изменением. Внизу панели показаны размер, адреса и строка для настоящей машины; ошибки показаны со строкой и столбцом.
3. **Запустить**: ▶ **Запустить** помещает байты в память по адресу `.org` и набирает команду Woz Monitor (`300R`).
4. **Отладить**: ● **Точка останова** на строке с курсором, затем пошаговое выполнение в отладчике (глава 8), с вашими метками в дизассемблере.
5. **Экспортировать**: 📼 **Экспорт WAV** записывает кассету; на настоящем Apple-1 наберите `C100R`, показанный диапазон (`0300.0320R`), затем `300R`.

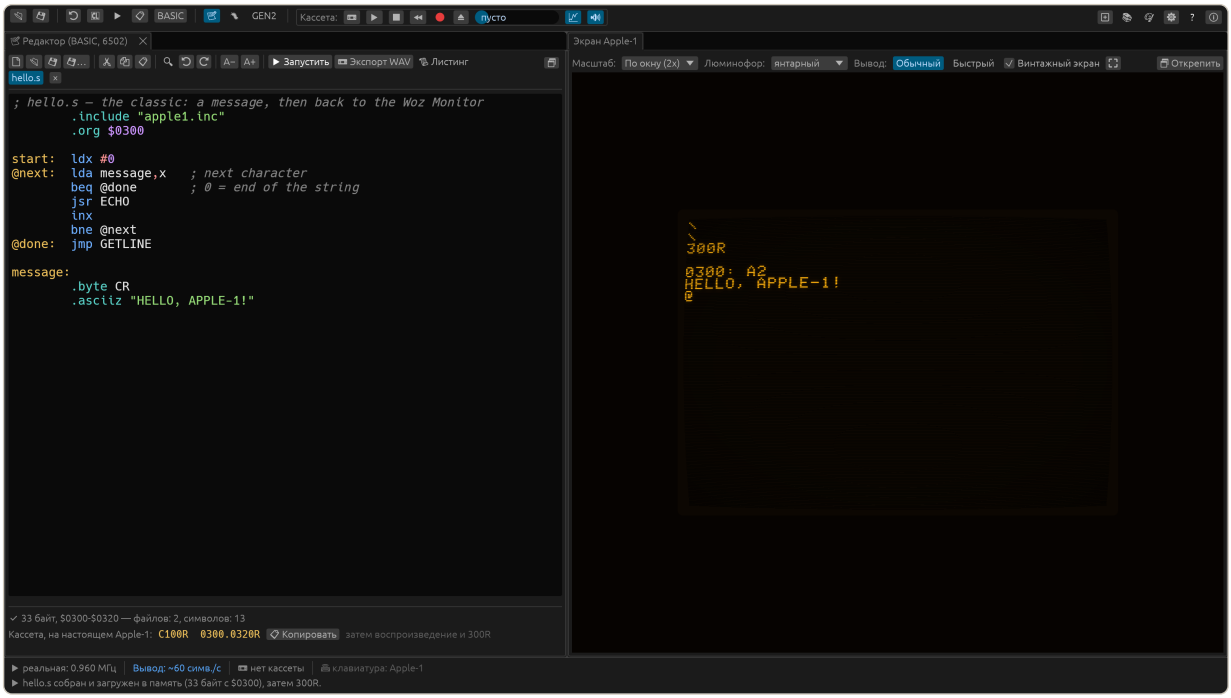


Рисунок 34 — Исходник в редакторе: слева раскрашенный текст, внизу результат ассемблирования; справа Apple-1 выполнил программу.

22.2 Листинг и символы

📃 **Листинг** показывает для каждой строки адрес, полученные байты и исходный текст; строки, порождённые макросом, помечены `+`, включаемые файлы развёрнуты, а за ними следует таблица символов. Щелчок по строке возвращает к исходнику; щелчок на полях ставит точку останова. Вот листинг `hello.s` (глава 23), полученный из командной строки:

hello.lst

```
0300  A2 00       start:  ldx #0
0302  BD 10 03    @next:  lda message,x   ; next character
0305  F0 06               beq @done       ; 0 = end of the string
0307  20 EF FF            jsr ECHO
030A  E8                  inx
030B  D0 F5               bne @next
030D  4C 1F FF    @done:  jmp GETLINE

                  message:
0310  8D                  .byte CR
0311  48 45 4C 4C         .asciiz "HELLO, APPLE-1!"
0315  4F 2C 20 41
…
; ---- Symboles (13)
; ECHO        $FFEF
; GETLINE     $FF1F
; message     $0310
; start       $0300
; start@done  $030D
; start@next  $0302
```

Локальные метки `@next` и `@done` принадлежат `start` : таблица называет их `start@next` и `start@done` , и отладчик принимает эти имена как адреса точек останова. (Листинг, записанный командной строкой, сохраняет французский заголовок *Symboles*.)

22.3 Командная строка

`apple1-emu asm` делает то же без окна:

TERMINAL

```
apple1-emu asm hello.s                  # только проверка
apple1-emu asm hello.s -l hello.lst     # с листингом
apple1-emu asm hello.s -o hello.wav     # кассета (.wav или .aif)
apple1-emu asm hello.s -o hello.bin     # сырой двоичный файл
apple1-emu asm hello.s -o hello.hex     # дамп Woz для вставки в монитор
apple1-emu asm jeu.s -I ../common       # другая папка для .include
```

Каждая команда подытоживает ассемблирование и даёт строку для настоящего Apple-1. Итог выводится по-французски («33 байта … файлов: 2, символов: 13 / Кассета (после C100R): 0300.0320R, затем 300R / записано: hello.wav»):

```
33 octets, $0300-$0320 ; 2 fichier(s), 13 symboles
Cassette (après C100R) : 0300.0320R   puis 300R
écrit : hello.wav
```

Без `.org` программа размещается с `$0300`. Дамп `.hex` имеет вид, который Woz Monitor принимает с клавиатуры, и заканчивается командой запуска:

t1.hex (фрагмент)

```
0280: 34 12 34 12 10 05 FF 34
0288: 12 FF FF A8 02 EA EA EA
…
0280R
```

22.4 Синтаксис

Строка содержит, в таком порядке и всё необязательно: метку, инструкцию или директиву и комментарий после `;`. Мнемоники и директивы можно писать строчными или заглавными буквами; символы чувствительны к регистру.

6502

```
PTR     = $06           ; a constant (no byte produced)
start:  lda #0          ; a global label, followed by a colon
@loop:  inx             ; a local label: visible up to the next global one
        bne @loop
:       dex             ; an anonymous label
        bne :-          ; :- the previous one, :+ the next one, :-- the one before
```

Числа: `255` в десятичной системе, `$FF` в шестнадцатеричной, `%1010` в двоичной, `'A'` — символ ($41). Клавиатура возвращает коды с **установленным битом 7**: чтобы сравнить клавишу, пишите `'A' | $80`; `apple1.inc` так же определяет `CR = $8D`. `*` обозначает текущий адрес.

Операции, от низшего приоритета к высшему:

Приоритет	Операции	Значение
1	`||`	логическое ИЛИ (результат 0 или 1)
2	`&&`	логическое И
3	`=` `<>` `<` `>` `<=` `>=`	сравнения (0 или 1)
4	`+` `-` `|`	сложение, вычитание, побитовое ИЛИ
5	`*` `/` `&` `^` `<<` `>>`	умножение, деление, И, исключающее ИЛИ, сдвиги
6 (унарные)	`<` `>` `-` `~` `!`	младший байт, старший байт, смена знака, дополнение, логическое НЕ

Так, `lda #<message` загружает младший байт адреса, а `lda #>message` — старший; `#>S+1` — это `(>S)+1`. Операции взятия остатка нет (`%` и `.mod` дают синтаксическую ошибку).

Нулевая страница или абсолютный адрес. Адрес ниже `$100`, известный во время ассемблирования, кодируется как нулевая страница (`lda $0006` даёт `A5 06`, как в ca65). Чтобы принудительно получить абсолютную форму, пишите `a:` (`lda a:$06` даёт `AD 06 00`); чтобы принудительно получить нулевую страницу — `z:`.

> **! ОПРЕДЕЛЯЙТЕ КОНСТАНТЫ НУЛЕВОЙ СТРАНИЦЫ ДО ИХ ИСПОЛЬЗОВАНИЯ**
>
> Если `PTR = $06` записано **после** `lda PTR`, ассемблер ещё не знает значения и кодирует адрес двумя байтами. Он отмечает это жёлтым: `предупреждение: адрес нулевой страницы закодирован 2 байтами (символ определён ниже; определите его р`
> Программа работает, но она длиннее, а режимы `(zp),Y` становятся невозможными.

22.5 Директивы

Директива	Действие	Пример
`.org адр`	адрес ассемблирования (и загрузки)	`.org $0300`
`.byte` (`.byt`)	байты, включая строки	`.byte CR, "OK", 0`
`.word` (`.addr`)	16-битные слова, младший байт первым	`.word 10000, end`
`.asciiz`	строка с нулём на конце	`.asciiz "HELLO"`
`.res n[, знач]`	резервирует n байт (0 или знач)	`.res 3, $EA`
`.include "f"`	вставляет файл	`.include "apple1.inc"`
`.macro имя арг` … `.endmacro`	определяет макрос (`.mac`, `.endmac`)	см. `macros.s`
`.local a, b`	метки, свои для каждого вызова макроса	
`.repeat n[, перем]` … `.endrep`	повторяет блок, перем = 0 … n−1	`.repeat 16, I`
`.if` `.elseif` `.else` `.endif`	условное ассемблирование	`.if WIDTH > 30`
`.ifdef` `.ifndef`	в зависимости от того, существует ли символ	`.ifdef APPLE1`
`.ifblank` `.ifnblank`	аргумент макроса пуст или нет	
`.defined(симв)`	1, если символ существует (в выражении)	`.if .defined(d)`
`.define имя значение`	текстовая подстановка (`.undefine`)	`.define CHARACTER '*'`
`.setcpu "65C02"`	принимает инструкции 65C02 (`.pc02`, `.p02`)	
`.segment` `.code` `.data` `.rodata` `.bss`	принимаются для совместимости	
`.feature string_escapes`	`\n`, `\"` … в строках	

> **! СЕГМЕНТЫ НИЧЕГО НЕ ПЕРЕМЕЩАЮТ**
>
> В ca65 и ld65 `.code`, `.data` и `.bss` направляют код в области, заданные конфигурацией компоновщика. Встроенный ассемблер создаёт **один блок** в порядке исходника: директивы сегментов принимаются, но не действуют, а `.bss` даёт нули. Для проекта, которому нужен настоящий файл `.cfg`, библиотека использует ca65 и ld65 (глава 4).

22.6 Макросы, .repeat и условное ассемблирование

Макрос — это шаблон строк, копируемый при каждом вызове со своими аргументами. `.local` даёт каждому вызову собственные метки без конфликтов. `.repeat` разворачивает блок во время ассемблирования, что удобно и для **вычисления таблиц**:

```6502
squares:
.repeat 16, I
        .byte I * I     ; 0, 1, 4, 9 … 225
.endrep
```

Глава 23 представляет `macros.s`, где используется всё это, и `gen2-lores.s`, который вычисляет адреса 24 строк экрана с помощью `.repeat`.

22.7 Сообщения об ошибках

Каждая ошибка указывает файл, строку, столбец и проблемный текст; пока остаётся ошибка, не записывается ни одного байта. Редактор показывает сообщения на языке интерфейса (ниже — по-русски); командная строка выводит их по-французски.

Сообщение	Обычная причина
`синтаксическая ошибка`	неизвестная операция, пропущенная скобка
`неизвестная инструкция, директива или макрос`	опечатка; инструкция 65C02 без `.setcpu` ; метка без двоеточия
`режим адресации невозможен для этой инструкции`	`sta #5` , `ldx ($06),y` …
`неопределённый символ`	`jsr ECO` вместо `ECHO` ; забытый `.include`
`символ уже определён`	две метки с одним именем
`значение вне допустимого диапазона`	`lda #300` , адрес за `$FFFF`
`переход слишком далеко (от -128 до +127 байт)`	замените `bne far` на `beq :+` / `jmp far` / `:`
`файл .include не найден`	имя или папка: добавьте `-I`
`неправильно закрытый блок (.macro/.endmacro, .if/.endif)`	забытый `.endif` или `.endmacro`
`деление на ноль`	в выражении
`строка или символ без закрывающей кавычки`	`"HELLO`
`значение здесь ещё неизвестно (.org, .res, .if, .repeat)`	этим директивам нужно уже определённое значение

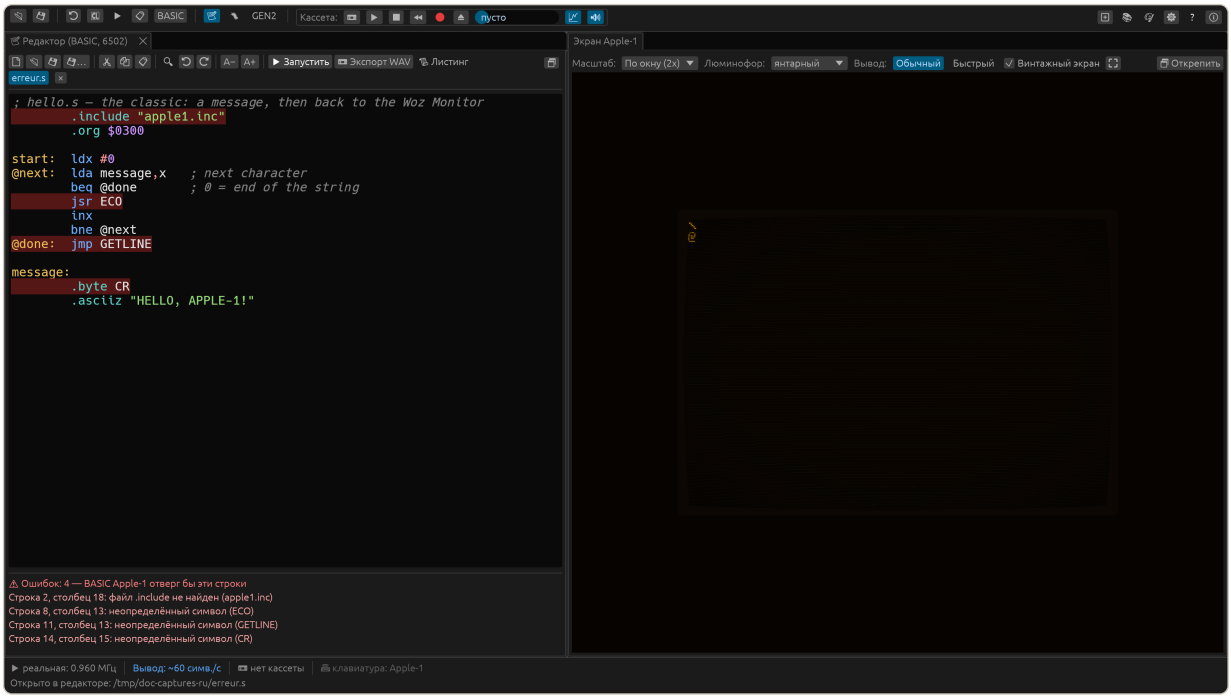


Рисунок 35 — Ошибка в редакторе: ECO вместо ECHO. Щелчок по сообщению переводит к строке.

22.8 Совместимость с ca65 и xa

Встроенный ассемблер следует синтаксису **ca65** (пакет cc65) во всём, что нужно программе из одного блока. Восемь примеров главы 23 были ассемблированы обоими инструментами и сравнены байт в байт:

Исходник	Байты	Встроенный = ca65
`hello.s`	33	**идентично**
`clavier.s`	32	**идентично**
`hexa.s`	64	**идентично**
`decimal.s`	117	**идентично**
`jeu.s`	222	**идентично**
`macros.s`	135	**идентично**
`gen2-lores.s`	110	**идентично**
`gen2-hires.s`	94	**идентично**

Исходники, написанные для **xa** (xa65), распознаются по строкам `#include` или `#define` и переводятся построчно до ассемблирования (номера строк в ошибках остаются верными): `#define`, `#include`, `*=`, `.BYT`, `.ASC`, комментарии `/* */` и метки в столбце 0 без двоеточия. Так ассемблируется слайд-шоу Дэйва Шменка (глава 9) — в те же 3456 байт, что и у xa 2.4.1.

Для **проектов GitHub** библиотека предпочитает ca65 и ld65, если они установлены (или если проект поставляет конфигурацию ld65), по умолчанию с `-D APPLE1`; иначе она использует встроенный ассемблер. Файл `apple1.toml` репозитория задаёт адреса и символы (глава 4).

ГЛАВА 23

Клавиатура, дисплей и Woz Monitor

У Apple-1 всего четыре регистра ввода-вывода в PIA 6820 (или 6821) и монитор размером 256 байт. Это немного, но достаточно: в этой главе показано, как прочитать клавишу, вывести символ и использовать подпрограммы Стива Возняка.

23.1 Четыре регистра PIA

Адрес	Имя	Чтение	Запись
`$D010`	`KBD`	код клавиши с установленным битом 7; чтение **подтверждает** клавишу	—
`$D011`	`KBDCR`	бит 7 = 1: клавиша ждёт	настройка (делает монитор)
`$D012`	`DSP`	бит 7 = 1: дисплей занят	выводимый символ (7 бит)
`$D013`	`DSPCR`	—	настройка (делает монитор)

Эти имена определены в `apple1.inc`, который включают все примеры:

apple1.inc

6502

```
; apple1.inc — I/O addresses and Woz Monitor routines
; (checked in the ROM: FFDC PRBYTE, FFE5 PRHEX, FFEF ECHO, FF1F GETLINE)
KBD     = $D010         ; keyboard data (bit 7 = 1)
KBDCR   = $D011         ; bit 7 = 1: a key is waiting
DSP     = $D012         ; write: character to the display (bit 7 read = busy)
DSPCR   = $D013
PRBYTE  = $FFDC         ; prints A as 2 hex digits (A destroyed)
PRHEX   = $FFE5         ; prints the low nibble of A (A destroyed)
ECHO    = $FFEF         ; prints the character in A (A, X, Y preserved)
GETLINE = $FF1F         ; back to the monitor: new line, then prompt
CR      = $8D           ; carriage return (bit 7 set, as the monitor uses it)
```

23.2 Чтение клавиши

Дождитесь, пока бит 7 `KBDCR` станет 1, затем прочитайте `KBD` :

```6502
read:   lda KBDCR       ; a key?
        bpl read        ; bit 7 = 0: not yet
        lda KBD         ; ASCII code + $80
```

Клавиатура выдаёт только заглавные буквы (эмулятор преобразует строчные), с установленным битом 7: `A` даёт `$C1` , RETURN — `$8D` , ESC — `$9B` . Чтобы узнать, нажата ли клавиша, **не дожидаясь**, проверяйте только `KBDCR` :

```6502
        lda KBDCR
        bpl nokey       ; no key: carry on working
        lda KBD         ; a key: read it (and acknowledge it)
```

`jeu.s` (глава 23) использует это, чтобы выбрать случайное число: он считает в цикле, пока игрок не нажмёт клавишу.

23.3 Вывод символа

Терминал Apple-1 принимает по одному символу и только когда готов: бит 7 `DSP` остаётся равным 1, пока предыдущий символ не выведен. Подпрограмма монитора `ECHO` ждёт за вас. Вот её четыре инструкции, прочитанные в мониторе эмулятора:

```WOZ MONITOR
\
FFEF.FFFFR

FFEF: 2C
FFF0: 12 D0 30 FB 8D 12 D0 60
FFF8: 00 00 00 0F 00 FF 00 00
```

Это `BIT DSP` / `BMI ECHO` / `STA DSP` / `RTS` . `ECHO` сохраняет A, X и Y. Поскольку оригинальный терминал выводит только **один символ за кадр**, около 60 в секунду, программу, которая много выводит, ограничивает дисплей, а не процессор; эмулятор по умолчанию соблюдает это ограничение (F4 снимает его, глава 2).

Что дисплей делает с каждым кодом:

- **RETURN** (`$8D` или `$0D`) начинает новую строку; внизу экрана всё прокручивается на строку вверх.
- **Остальные управляющие символы** (`$00-$1F` , с битом 7 или без) игнорируются: ни очистки, ни возврата на символ, ни звонка.
- **Строчные** `$60-$7F` выводятся как заглавные `$40-$5F` : терминал сохраняет только 6 бит.
- После 40-го столбца курсор сам переходит на следующую строку.
- Кода для очистки экрана нет: это кнопка на машине (CL **Очистить экран**, Ctrl + L).

23.4 Подпрограммы Woz Monitor

Адрес	Имя	Вход	Действие	Регистры
`$FFEF`	`ECHO`	A = символ	выводит его	A, X, Y сохраняются
`$FFDC`	`PRBYTE`	A = байт	выводит его двумя шестнадцатеричными цифрами	A портится; X, Y сохраняются
`$FFE5`	`PRHEX`	A (младшая тетрада)	выводит одну шестнадцатеричную цифру	A портится; X, Y сохраняются
`$FF1F`	`GETLINE`	—	возврат в монитор: новая строка, затем ожидание команды	—
`$FF1A`	`ESCAPE`	—	как `GETLINE`, но сначала выводит `\`	—
`$FF00`	`RESET`	—	инициализирует PIA и монитор	—

Монитор использует нулевую страницу `$24-$2B` и буфер строки `$0200-$027F`; всё остальное — ваше (если только не нужно сохранить Integer BASIC, глава 16). Его команды (`300R`, `0300.031F`, `300: A9 8D` ...) сведены в приложении B.

23.5 Запуск и завершение программы

Программу запускает команда монитора `R` (`300R`), которая делает `JMP` по адресу, а не `JSR`. Поэтому в стеке **нет адреса возврата**: завершайте программу через `jmp GETLINE` (`$FF1F`), а не через `RTS`.

> **! RTS И BRK НЕ ВОЗВРАЩАЮТ УПРАВЛЕНИЕ**
>
> `RTS` в конце программы, запущенной через `R`, снимает со стека что попало. `BRK` переходит по вектору прерывания ПЗУ, который равен `$0000` (последние три слова монитора выше: NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000`): тогда процессор выполняет нулевую страницу. В обоих случаях нажмите RESET (Ctrl+R).

Зато программа, вызванная из BASIC через `CALL 768`, вызывается командой `JSR`: она должна заканчиваться `RTS`, чтобы вернуться в BASIC. В главе 16 показано, как разместить машинный код ниже программы на BASIC.

WOZ MONITOR

```
>POKE 768,96
>CALL 768
>
```

(`96` = `$60`, код операции `RTS`: самая маленькая возможная подпрограмма.)

23.6 PIA, кассета и цветная карта

Карта ACI занимает `$C000-$C0FF` (каждое обращение переключает выход), а её программа — `$C100-$C1FF`; в главе 5 объясняется, как она работает, а в приложении C — её временные параметры. Карта GEN2 управляется простым **чтением** `$C250-$C257` (`bit SS_GR`); её адреса — в `gen2.inc`:

gen2.inc

```6502
; gen2.inc — soft switches of Uncle Bernie's GEN2 color card
; (the Apple II $C050-$C057, moved to $C25x; a READ flips the switch)
SS_GR    = $C250        ; graphics (leaves text mode)
SS_TEXT  = $C251        ; text
SS_FULL  = $C252        ; full screen
SS_MIX   = $C253        ; 4 lines of text at the bottom
SS_PAGE1 = $C254
SS_PAGE2 = $C255
SS_LORES = $C256
SS_HIRES = $C257
```

ГЛАВА 24

Примеры с комментариями

Восемь программ, от самой простой до самой полной. Они лежат в папке `docs/exemples/en/asm/` дистрибутива; каждая ассемблирована встроенным ассемблером и ca65 (байты совпадают), затем запущена в эмуляторе. Приведённый здесь вывод — вывод эмулятора.

Чтобы попробовать: откройте файл в редакторе и нажмите ▶ **Запустить** или в командной строке выполните `apple1-emu asm hello.s -o hello.wav`, затем проиграйте кассету (глава 5). Держите `apple1.inc` (и `gen2.inc`) в той же папке, что и исходники.

Комментарии и выводимые тексты программ — на английском: Apple-1 не умеет выводить кириллицу.

Файл	Байты	Адрес	Что показывает
`hello.s`	33	`$0300`	строка, `ECHO`, локальные метки, возврат в монитор
`clavier.s`	32	`$0300`	чтение клавиатуры, сравнение символов
`hexa.s`	64	`$0300`	указатель в нулевой странице, `(zp),Y`, 16-битное сложение, `PRBYTE`
`decimal.s`	117	`$0300`	16-битные числа, вычитания, таблицы слов
`jeu.s`	222	`$0300`	полная игра: случайность, подпрограммы, сообщения
`macros.s`	135	`$0300`	макросы, `.local`, `.repeat`, `.if`, вычисляемые таблицы
`gen2-lores.s`	110	`$0C00`	карта GEN2 в режиме LORES
`gen2-hires.s`	94	`$0C00`	карта GEN2 в режиме HIRES

24.1 `hello.s`: вывод сообщения

hello.s 6502

```
; hello.s — the classic: a message, then back to the Woz Monitor
        .include "apple1.inc"
        .org $0300

start:  ldx #0
@next:  lda message,x   ; next character
        beq @done       ; 0 = end of the string
        jsr ECHO
        inx
        bne @next
@done:  jmp GETLINE

message:
        .byte CR
        .asciiz "HELLO, APPLE-1!"
```

X проходит по строке до нулевого байта, поставленного `.asciiz`. `bne @next` после `inx` никогда не ложен для строки короче 256 символов: это короткий и надёжный переход. Программа заканчивается `jmp GETLINE`, а не `RTS` (глава 22).

WOZ MONITOR

```
\
300R
0300: A2
HELLO, APPLE-1!
```

Монитор сначала выводит `0300: A2` — байт по начальному адресу, затем управление берёт программа.

24.2 clavier.s: эхо клавиш

clavier.s 6502

```
; clavier.s — reads the keyboard and echoes each key to the screen.
; Digits are printed twice; ESC returns to the monitor.
        .include "apple1.inc"
        .org $0300

read:   lda KBDCR       ; a key?
        bpl read        ; bit 7 = 0: not yet
        lda KBD         ; ASCII code + $80; reading clears the key strobe
        cmp #$9B        ; ESC
        beq done
        jsr ECHO
        cmp #'0' | $80
        bcc read        ; < '0'
        cmp #'9' | $80 + 1
        bcs read        ; > '9'
        jsr ECHO        ; a digit: print it a second time
        jmp read
done:   jmp GETLINE
```

`cmp #'0' | $80` сравнивает с кодом цифры 0 с установленным битом 7, как это делает клавиатура. `bcc` («меньше») и `bcs` («больше или равно») ограничивают цифры с двух сторон. При нажатии `A`, `B`, `1`, `2`, затем ESC на экране видно:

```
0300: ADAB1122
\
```

`AD` выводит монитор (`lda KBDCR` начинается с кода операции `$AD`); `AB`, затем `11` и `22` — программа; ESC возвращает управление монитору, который выводит `\`.

24.3 hexa.s: дамп в стиле монитора

hexa.s 6502

```
; hexa.s — monitor-style memory dump with PRBYTE:
; 2 lines of 8 bytes starting at $FF00
        .include "apple1.inc"
        .org $0300
PTR     = $06           ; zero-page pointer (not used by the monitor)

start:  lda #<$FF00
        sta PTR
        lda #>$FF00
        sta PTR+1
        ldx #2          ; number of lines
line:   lda #CR
```

```
        jsr ECHO
        lda PTR+1       ; address: high byte, then low byte
        jsr PRBYTE
        lda PTR
        jsr PRBYTE
        lda #':' | $80
        jsr ECHO
        ldy #0
byte:   lda #' ' | $80
        jsr ECHO
        lda (PTR),y
        jsr PRBYTE      ; PRBYTE destroys A but keeps X and Y
        iny
        cpy #8
        bne byte
        clc             ; PTR += 8
        lda PTR
        adc #8
        sta PTR
        bcc :+
        inc PTR+1
:       dex
        bne line
        jmp GETLINE
```

`PTR` и `PTR+1` образуют **16-битный указатель** в нулевой странице, младший байт первым; `lda (PTR),y` читает байт по адресу PTR + Y. Сложение `PTR += 8` показывает перенос в действии: `clc`, `adc #8`, и если возник перенос (`bcc` не сработал), `inc PTR+1`. Результат — начало Woz Monitor:

```
FF00: D8 58 A0 7F 8C 12 D0 A9
FF08: A7 8D 11 D0 8D 13 D0 C9
```

24.4 `decimal.s`: вывод числа в десятичном виде

6502

decimal.s

```
; decimal.s — prints an unsigned 16-bit integer in decimal
; (repeated subtraction of powers of ten, no leading zeros)
        .include "apple1.inc"
        .org $0300
NUM     = $06           ; number to print (2 bytes)
FLAG    = $08           ; 1 once a non-zero digit has been printed

start:  lda #CR
        jsr ECHO
        ldx #0
@list:  lda numbers,x   ; print each number of the list
        sta NUM
```

```
        lda numbers+1,x
        sta NUM+1
        jsr prdec
        lda #' ' | $80
        jsr ECHO
        inx
        inx
        cpx #end_numbers - numbers
        bne @list
        jmp GETLINE

; prdec: prints NUM in decimal. Preserves X.
prdec:  txa
        pha
        lda #0
        sta FLAG
        ldx #0          ; index into the table of powers
@digit: ldy #'0' | $80  ; Y = current digit
@sub:   lda NUM         ; NUM - power
        sec
        sbc powers,x
        pha
        lda NUM+1
        sbc powers+1,x
        bcc @found      ; negative: digit found
        sta NUM+1
        pla
        sta NUM
        iny
        bne @sub
@found: pla
        tya
        cmp #'0' | $80
        bne @print
        ldy FLAG        ; leading zero? skip it
        beq @next
@print: jsr ECHO
        inc FLAG
@next:  inx
        inx
        cpx #8          ; 4 powers: 10000 down to 10
        bne @digit
        lda NUM         ; remainder: the units, always printed
        ora #'0' | $80
        jsr ECHO
        pla
        tax
        rts

powers: .word 10000, 1000, 100, 10
```

```
numbers:
        .word 0, 7, 42, 1976, 6502, 65535
end_numbers:
```

6502 не умеет делить: `prdec` вычитает 10000 столько раз, сколько возможно (счётчик даёт цифру десятков тысяч), затем 1000, 100 и 10. 16-битное вычитание делается в два шага: `sbc` по младшему байту, затем по старшему, а перенос связывает их. `FLAG` подавляет ведущие нули. Вывод:

```
0 7 42 1976 6502 65535
```

✦ **СОХРАНЕНИЕ X В ПОДПРОГРАММЕ**

`prdec` использует X для себя, а главному циклу он тоже нужен: подпрограмма сохраняет его через `txa` / `pha` в начале и восстанавливает через `pla` / `tax` в конце. У NMOS 6502 нет `PHX`.

24.5 jeu.s: угадай цифру

jeu.s 6502

```
; jeu.s — guess the digit (0 to 9): a simple game loop.
; The randomness comes from how long the player takes to press a key.
        .include "apple1.inc"
        .org $0300
SECRET  = $06
COUNT   = $07

start:  ldx #<title
        ldy #>title
        jsr prmsg
        ldx #0          ; X spins while we wait: the seed
@wait:  inx
        lda KBDCR
        bpl @wait
        lda KBD
        txa
@mod:   cmp #10         ; X modulo 10
        bcc @ok
        sbc #10         ; C = 1 here
        bcs @mod
@ok:    sta SECRET
        lda #0
        sta COUNT

turn:   ldx #<question
        ldy #>question
        jsr prmsg
        jsr getdig      ; A = digit 0..9 (other keys are ignored)
        inc COUNT
```

```
        cmp SECRET
        beq won
        bcc @higher
        ldx #<lower
        ldy #>lower
        jsr prmsg
        jmp turn
@higher:
        ldx #<higher
        ldy #>higher
        jsr prmsg
        jmp turn

won:    ldx #<bravo
        ldy #>bravo
        jsr prmsg
        lda COUNT
        ora #'0' | $80  ; < 10 tries: a single digit
        cmp #'9' | $80 + 1
        bcc :+
        lda #'+' | $80
:       jsr ECHO
        ldx #<tries
        ldy #>tries
        jsr prmsg
        jmp GETLINE

; getdig: waits for a digit, echoes it, returns it in binary in A
getdig: lda KBDCR
        bpl getdig
        lda KBD
        cmp #'0' | $80
        bcc getdig
        cmp #'9' | $80 + 1
        bcs getdig
        jsr ECHO
        and #$0F
        rts

; prmsg: prints the zero-terminated string whose address is in Y:X
MSG     = $08
prmsg:  stx MSG
        sty MSG+1
        ldy #0
:       lda (MSG),y
        beq :+
        jsr ECHO
        iny
        bne :-
:       rts
```

```
title:    .byte CR
          .asciiz "I PICKED A DIGIT. PRESS ANY KEY."
question: .byte CR
          .asciiz "YOUR DIGIT? "
higher:   .asciiz " HIGHER."
lower:    .asciiz " LOWER."
bravo:    .byte " WON IN ", 0
tries:    .asciiz " TRIES!"
```

Случайность даёт игрок: X непрерывно считает (`@wait`) до первой клавиши, затем остаток от деления на 10 даёт загаданную цифру. `prmsg` получает адрес сообщения в **Y:X** (старший байт в Y), сохраняет его в указателе `MSG` и выводит его через `(MSG),y` . Одна партия, где набраны `x` , затем цифры по порядку:

```
I PICKED A DIGIT. PRESS ANY KEY.
YOUR DIGIT? 0 HIGHER.
YOUR DIGIT? 1 HIGHER.
…
YOUR DIGIT? 7 HIGHER.
YOUR DIGIT? 8 WON IN 9 TRIES!
```

24.6 `macros.s`: пусть работает ассемблер

6502

macros.s

```
; macros.s — macros, .repeat and conditional assembly
        .include "apple1.inc"
        .org $0300

WIDTH = 20              ; try 10 or 40
.define CHARACTER '*'

; print a constant character (the macro expands to lda/jsr)
.macro putc c
        lda #(c) | $80
        jsr ECHO
.endmacro

; print a string: local labels for each call
.macro print text
        .local loop, done, string
        ldx #0
loop:   lda string,x
        beq done
        jsr ECHO
        inx
        bne loop
        beq done
```

```
string: .asciiz text
done:
.endmacro

start:  putc 13
        print "MACROS AND .REPEAT"
        putc 13
.repeat 3               ; three rows of stars, unrolled at assembly time
        ldx #WIDTH
:       putc CHARACTER
        dex
        bne :-
        putc 13
.endrep
.if WIDTH > 30
        print "LONG LINES"
.else
        print "SHORT LINES"
.endif
        jmp GETLINE

; a table computed by the assembler: the squares of 0 to 15
squares:
.repeat 16, I
        .byte I * I
.endrep
```

`putc` и `print` порождают код при каждом вызове; `.local` не даёт двум вызовам `print` дважды определить `loop`. `.repeat 3` трижды разворачивает строку звёздочек (каждую со своей анонимной меткой), `.if` выбирает сообщение в зависимости от `WIDTH`, а таблица `squares` вычисляется во время ассемблирования. Вывод:

```
MACROS AND .REPEAT
**********
**********
**********
SHORT LINES
```

Замените `WIDTH = 20` на `40`: редактор сразу переассемблирует, строки станут длиннее, а сообщение сменится на `LONG LINES`.

24.7 `gen2-lores.s`: цвет в низком разрешении

Этим двум программам нужна карта GEN2 (🎨 **GEN2** на панели инструментов или ⚙ **Настройки ▸ GEN2**, глава 6), и они начинаются с `$0C00`, выше страницы LORES.

gen2-lores.s 6502

```
; gen2-lores.s — color bars in LORES (40 x 48, 16 colors)
; Needs the GEN2 card (Settings ▸ Color card). Any key: back to text.
        .include "apple1.inc"
        .include "gen2.inc"
        .org $0C00      ; above LORES page 1 ($0400-$07FF)
PTR     = $06

start:  ldx #0          ; X = text row (0..23)
row:    lda rows_lo,x
        sta PTR
        lda rows_hi,x
        sta PTR+1
        ldy #39
column:
        tya             ; color = column / 2 (0..15, then again)
        lsr a
        and #$0F
        sta $00         ; one byte = 2 blocks: low nibble on top, high below
        asl a
        asl a
        asl a
        asl a
        ora $00
        sta (PTR),y
        dey
        bpl column
        inx
        cpx #24
        bne row

        bit SS_LORES    ; LORES, full screen, page 1, then graphics
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
wait:
        lda KBDCR
        bpl wait
        lda KBD
        bit SS_TEXT
        jmp GETLINE

; addresses of the 24 rows: $400 + (r & 7) * $80 + (r >> 3) * $28
rows_lo:
.repeat 24, R
        .byte <($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
rows_hi:
.repeat 24, R
        .byte >($0400 + (R & 7) * $80 + (R >> 3) * $28)
.endrep
```

Две таблицы `rows_lo` и `rows_hi` вычисляются через `.repeat` по формуле адресов строк из главы 6. Каждый байт несёт два блока, отсюда тетрада, скопированная в верхнюю и нижнюю половины. Инструкции `bit SS_…` — это **чтения**: так переключаются режимы карты.

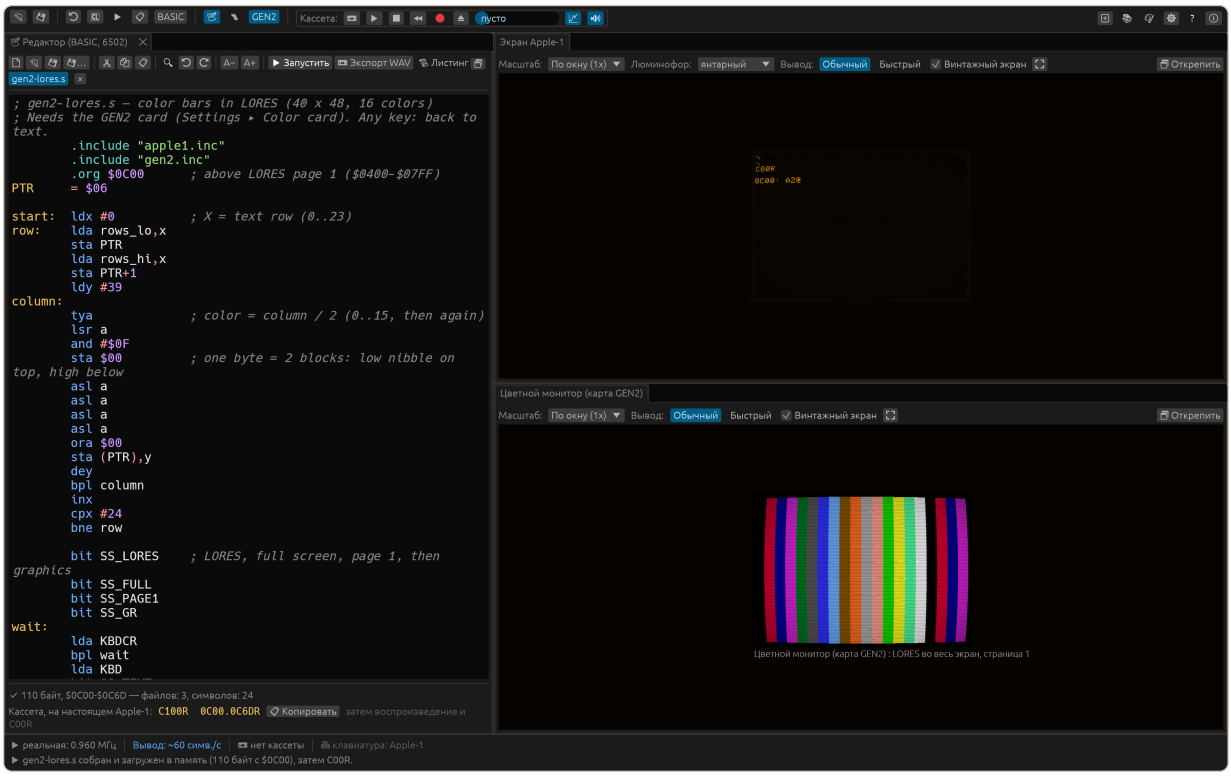


Рисунок 36 — gen2-lores.s: шестнадцать цветов LORES, по два столбца на цвет, на цветном мониторе.

24.8 `gen2-hires.s`: высокое разрешение

gen2-hires.s

6502

```
; gen2-hires.s — three color bands in HIRES (280 x 192)
; In each 128-byte block of the $2000 page, bytes 0-39 go to the top
; third of the screen, 40-79 to the middle, 80-119 to the bottom (120-127: holes).
        .include "apple1.inc"
        .include "gen2.inc"
        .org $0C00
PTR     = $06

start:  lda #<$2000
        sta PTR
        lda #>$2000
        sta PTR+1
block:  ldy #0
        ldx #0          ; X = pattern: 0 top, 2 middle, 4 bottom
third:  lda #40
        sta $08
byte:   tya             ; even or odd column?
        and #1
        ora pattern_idx,x
```

```
        sty $09
        tay
        lda patterns,y
        ldy $09
        sta (PTR),y
        iny
        dec $08
        bne byte
        inx
        cpx #3
        bne third
        clc             ; next block (+128)
        lda PTR
        adc #$80
        sta PTR
        bcc block
        inc PTR+1
        lda PTR+1
        cmp #$40        ; end of page 1 ($2000-$3FFF)
        bne block

        bit SS_HIRES
        bit SS_FULL
        bit SS_PAGE1
        bit SS_GR
wait:
        lda KBDCR
        bpl wait
        lda KBD
        bit SS_TEXT
        jmp GETLINE

pattern_idx: .byte 0, 2, 4
; two bytes per pattern (even column, odd column):
; violet, green, then blue and orange (bit 7 = 1: the other palette)
patterns: .byte $55, $2A        ; violet
        .byte $2A, $55          ; green
        .byte $D5, $AA          ; blue
```

В HIRES страница `$2000-$3FFF` разделена на блоки по 128 байт, первые 40 байт которых идут в верхнюю треть экрана, следующие 40 — в среднюю, а ещё 40 — в нижнюю. Программа заполняет каждую треть своим узором, чередуя два байта в зависимости от чётности столбца: получаются три цветные полосы.

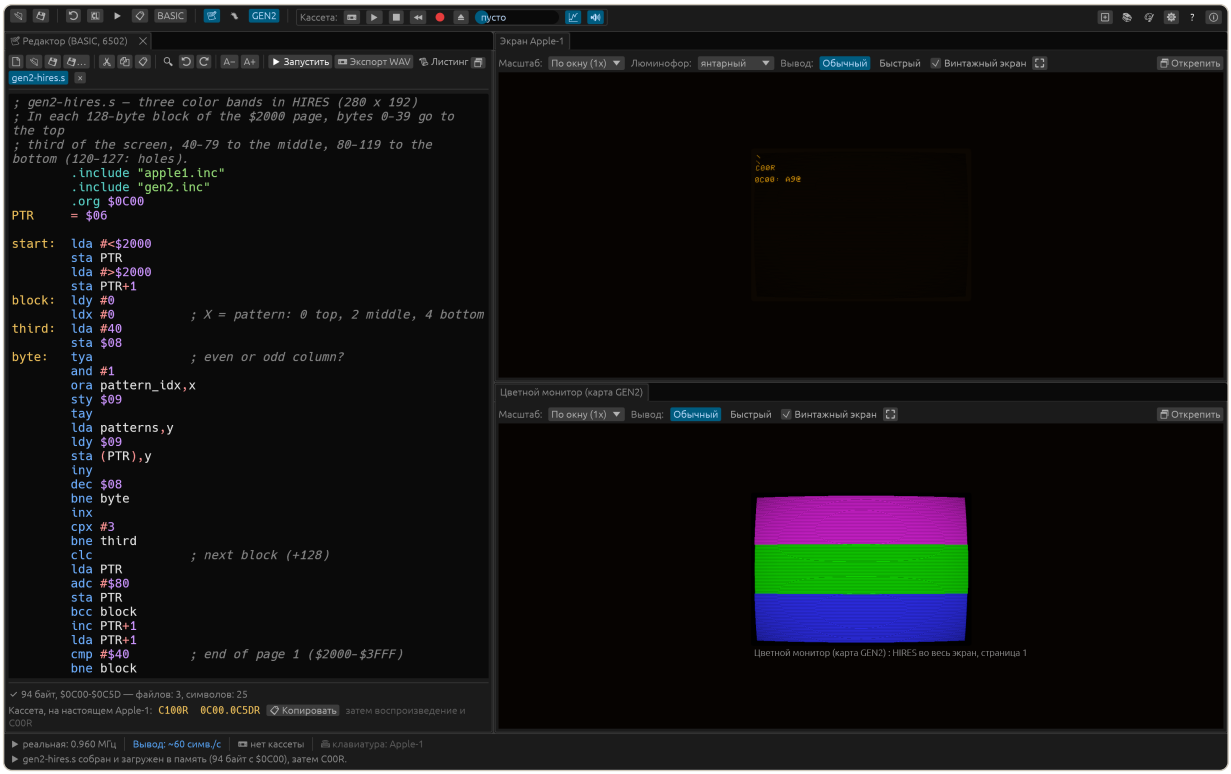


Рисунок 37 — gen2-hires.s: три цветные полосы в HIRES: фиолетовая, зелёная и синяя.

ГЛАВА 25

Советы, ловушки и ограничения

Что узнаёшь, когда пишешь программы для Apple-1 — сначала в эмуляторе, затем на настоящей машине.

25.1 Где разместить программу

Адрес	Для чего
`$0280`	как можно ниже: сразу после буфера монитора
`$0300`	обычный выбор (и адрес ассемблера по умолчанию)
`$0C00`	с картой GEN2 в режиме LORES или текста (страницы `$0400-$0BFF` оставлены свободными для изображения)
ниже HIMEM	с Integer BASIC: зарезервируйте место (глава 16)

В нулевой странице примеры используют `$06-$09`, которые монитор оставляет свободными. Если программа должна сосуществовать с BASIC, выбирайте адреса, которые BASIC не использует, или сохраняйте и восстанавливайте их.

25.2 Жизнь в пределах дальности перехода

Ветвление достаёт только на 127 байт вперёд или на 128 назад. Когда ассемблер отвечает `переход слишком далеко`, обратите условие и перепрыгните через `JMP`:

```6502
        beq :+          ; instead of: bne far
        jmp far
:
```

25.3 Строки и бит 7

Клавиатура возвращает коды с установленным битом 7, и монитор сравнивает именно с такими кодами. Дисплей же бит 7 игнорирует: `.asciiz "HELLO"` выводится прекрасно. Чтобы **сравнить** клавишу, добавьте бит 7 в константу: `cmp #'Q' | $80`. Строчные буквы в строке выводятся как заглавные.

25.4 Небольшая пауза

На 1,023 МГц (0,96 МГц фактически) цикл `dex` / `bne` стоит 5 тактов на итерацию:

```6502
delay:
        ldy #78         ; 78 x 256 iterations of 5 cycles: about 0.1 s
:       ldx #0
:       dex
        bne :-
        dey
        bne :--
        rts
```

Эмулятор на реальной скорости соблюдает эти длительности; на быстрой скорости (F3) они исчезают. Циклы задержки ACI считаются в тактах точно так же (приложение C).

25.5 Эффективная отладка

- Ставьте точки останова на **метку** (`start` , `prdec` , `start@done`), а не на адрес: она остаётся верной, когда код сдвигается.
- Проходите `jsr ECHO` с помощью ↪ **Обойти**: иначе вы будете следовать за циклом ожидания дисплея инструкция за инструкцией.
- Если Apple-1 перестал отвечать после вашей программы, посмотрите на PC: если он в нулевой странице, это почти всегда случайный `BRK` или `RTS` (глава 22).
- **Дамп** показывает нулевую страницу: там видны ваши указатели, младший байт первым.

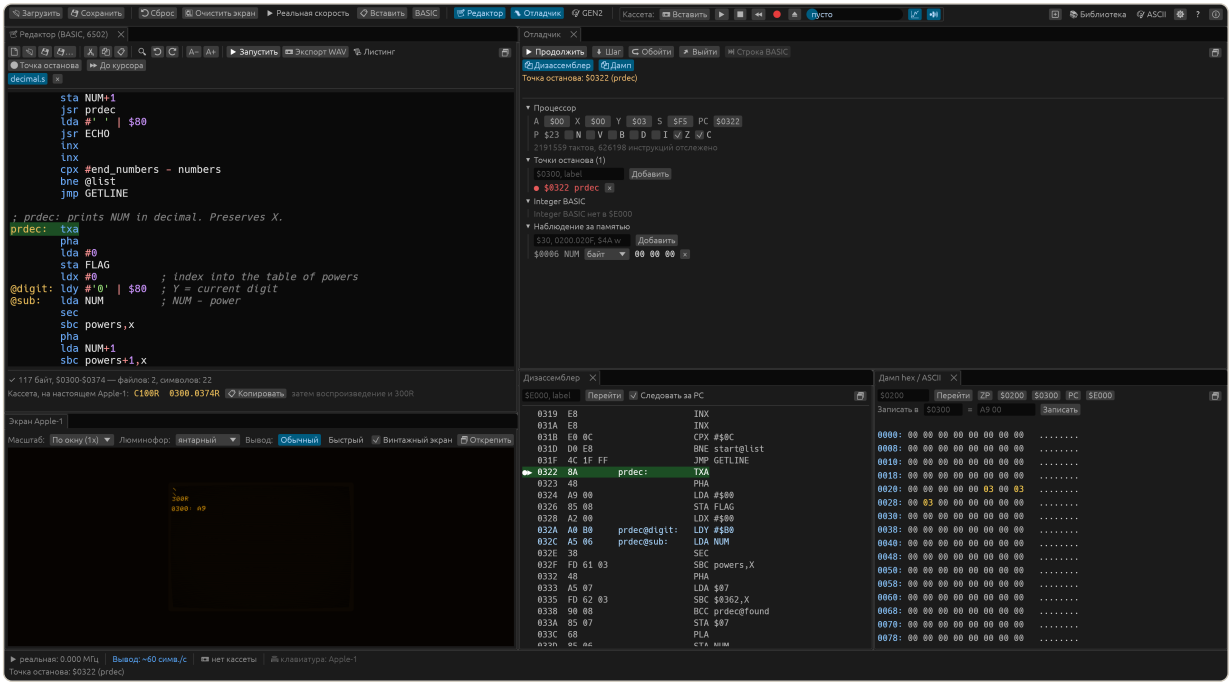


Рисунок 38 — Отладчик, остановленный в исходнике 6502: регистры, флаги, дизассемблер с метками и память.

25.6 Переход на настоящий Apple-1

1. Ассемблируйте и проверьте в эмуляторе, **на реальной скорости** и с выводом, ограниченным 60 символами в секунду.
2. Экспортируйте кассету (📼 **Экспорт WAV** или `-o prog.aif`): AIFF — самый надёжный формат при воспроизведении со смартфона.
3. На Apple-1: `C100R`, затем показанный диапазон (`0300.0320R`), включите воспроизведение, затем `300R`.
4. Без ACI: дамп `.hex` можно набрать (или вставить в последовательный терминал) прямо в Woz Monitor; его последняя строка запускает программу.

25.7 Ограничения встроенного ассемблера

- Один выходной блок в порядке исходника: сегменты не действуют, компоновщика нет. Для кода, разнесённого по нескольким областям, используйте ca65 и ld65.
- Нет операции взятия остатка, нет `.proc`, `.scope`, `.struct` и `.enum`: это директивы ca65, которых встроенный ассемблер не знает.
- Инструкции 65C02 ассемблируются, но эмулируемый процессор — NMOS 6502: в эмуляторе они не работают (глава 20).
- В **листинге** строка, порождённая макросом, иногда теряет пробел между инструкцией и операндом (`ldastring, x` вместо `lda string,x`). Это дефект отображения только в листинге: байты верные.

ПРИЛОЖЕНИЯ

Приложения

Карта памяти, команды Woz Monitor, временные параметры ACI, набор символов и сочетания клавиш

ПРИЛОЖЕНИЕ А

Карта памяти

Адреса	Содержимое	Примечания
FF00–FFFF	**Woz Monitor** (ППЗУ 256 байт)	векторы NMI, RESET, IRQ
E000–EFFF	**Integer BASIC** (4 КБ ОЗУ)	загружается с кассеты
D010–D013	**PIA 6821**: клавиатура, экран	KBD, KBDCR, DSP, DSPCR
C250–C257	**Карта GEN2**: программные переключатели	необязательна, доступ чтением
C100–C1FF	**ППЗУ ACI**(C100R)	C000–C0FF: триггер
1000–BFFF	дополнительное ОЗУ (до 48 КБ)	HIRES 2000–5FFF (GEN2)
0800–0FFF	программы BASIC (LOMEM–HIMEM)	текст/LORES 2: 0800
0300–07FF	ваши программы в машинных кодах	текст/LORES 1: 0400
0200–027F	буфер ввода монитора	не занимать
0100–01FF	стек 6502	
0000–00FF	нулевая страница	монитор 24–2B, BASIC 4A–FF

Рисунок 39 — Память Apple-1 в эмуляторе, от векторов монитора до нулевой страницы.

Адреса	Содержимое	Подробности
`$0000-$00FF`	нулевая страница	Woz Monitor `$24-$2B`; Integer BASIC `$4A-$FF` (указатели LOMEM `$4A`, HIMEM `$4C`, PP `$CA`, PV `$CC`)
`$0100-$01FF`	стек 6502	
`$0200-$027F`	буфер строки монитора	128 символов
`$0280-$0FFF`	базовое ОЗУ (4 КБ)	программы на машинном языке; BASIC между LOMEM (`$0800`) и HIMEM (`$1000`)
`$1000-$BFFF`	ОЗУ расширения	по умолчанию в эмуляторе 8 КБ (до `$1FFF`), до 48 КБ с `--ram`
`$C000-$C0FF`	ACI: переключение выхода, вход кассеты	любое обращение переключает выход
`$C100-$C1FF`	ACI: ППЗУ (программа ACI)	запускается через `C100R`
`$4000-$7FFF`	CodeTank карты P-LAB	EPROM 16 КБ, только чтение, когда ПЗУ подключено (поверх ОЗУ)
`$C250-$C257`	карта GEN2: переключатели	чтение; страницы текста/LORES `$0400`, `$0800`, HIRES `$2000`, `$4000`
`$CC00-$CC01`	карта P-LAB: TMS9918A	`$CC00` данные, `$CC01` управление и состояние; повторяется на `$CC00-$CFFF`
`$D010-$D013`	PIA: `KBD`, `KBDCR`, `DSP`, `DSPCR`	глава 22
`$E000-$EFFF`	BASIC	RDK G2 Basic (7 933 байта, нужно ОЗУ до `$FEFF`) или Integer BASIC от Apple (4 КБ ОЗУ, загружается с кассеты или из образа)
`$FF00-$FFFF`	Woz Monitor	векторы: NMI `$0F00`, RESET `$FF00`, IRQ/BRK `$0000`

ПРИЛОЖЕНИЕ B

Команды Woz Monitor

Монитор ждёт строку, завершённую RETURN. Адреса и байты записываются в шестнадцатеричном виде, без `$`; строка может содержать несколько команд через пробел.

Команда	Действие	Пример
`адр`	выводит байт по этому адресу	`FFEF`
`адр.адр`	выводит диапазон, по 8 байт в строке	`FF00.FF1F`
`.адр`	продолжает вывод до этого адреса	`.FF3F`
`адр: байт байт …`	записывает байты начиная с адреса	`300: A9 C1 20 EF FF`
`: байт …`	продолжает запись	`: 4C 1F FF`
`адрR`	запускает программу по этому адресу (`JMP`)	`300R`
Esc	отменяет строку (выводит `\`)	
`_` (backspace)	отменяет последний символ	

Строка длиннее 127 символов отменяется, как по Esc. После `R` монитор сначала выводит байт по адресу (`0300: A2`), затем управление берёт программа.

Адрес	Команда	Роль
`E000R`	холодный запуск BASIC	стирает программу
`E2B3R`	тёплый запуск BASIC	сохраняет программу
`C100R`	программа ACI	затем `начало.конецR` (чтение) или `начало.конецW` (запись)
`FF00R`	программный RESET монитора	

ПРИЛОЖЕНИЕ C

Временные параметры ACI

Длительности создаёт программа ACI, отсчитывая такты между двумя переключениями выхода. Значения измерены в эмуляторе, который выполняет эту программу на своём 6502 (тактовая частота 1,022727 МГц, с задержкой в 4 такта каждые 65 тактов на регенерацию памяти):

Элемент	Такты	Длительность	Частота
полупериод заголовка	592	618 мкс	≈ 809 Гц
первая половина стартового бита	180	188 мкс	
полупериод «0»	238	248 мкс	≈ 2,0 кГц
полупериод «1»	473	494 мкс	≈ 1,0 кГц
полный заголовок	16 390 полупериодов	≈ 10,1 с	

Формат записи:

1. **заголовок**: около 10 секунд полупериодов по 618 мкс;
2. **стартовый бит**: один короткий полупериод (188 мкс), затем полупериод «0»;
3. **данные**: каждый байт старшим битом вперёд, один полный период на бит;
4. больше ничего: ни длины, ни адреса, ни контрольной суммы. Диапазон задаёт команда чтения.

При чтении программа ACI измеряет полупериоды циклом в 12 тактов. Стартовым битом она считает любой полупериод короче 31 итерации (≈ 372 такта, 388 мкс), а «0» от «1» отделяет примерно на 0,74 мс полного периода. Декодер файлов эмулятора принимает заголовки от 0,40 до 0,90 мс на полупериод, что покрывает инструмент c2t (1 кГц) и колебания скорости ленты.

ПРИЛОЖЕНИЕ D

Набор символов 2513



Рисунок 40 — 64 символа Signetics 2513 так, как их рисует эмулятор: коды от $20 до $5F (с битом 7 — от $A0 до $DF).

Генератор символов знает только 64 знака: пробел, знаки препинания, цифры и заглавные латинские буквы. Терминал сохраняет только 6 бит полученного кода:

Посланный код	Отображение
`$20-$5F` (или `$A0-$DF`)	символ из таблицы
`$60-$7F` (строчные, или `$E0-$FF`)	соответствующая заглавная буква (`$40-$5F`)
`$0D` (или `$8D`)	новая строка
прочие `$00-$1F`	ничего: игнорируются

Каждый символ — матрица 5 × 7 точек в ячейке 7 × 8; экран — 40 × 24 символа, то есть 280 × 192 точки.

ПРИЛОЖЕНИЕ E

Сочетания клавиш

Клавиша	Действие
Ctrl+R	RESET Apple-1
Ctrl+L	CLEAR SCREEN
Ctrl+V	вставка: набирает содержимое буфера обмена на клавиатуре Apple-1
F1	режим справки (подробные подсказки)
F3	реальная или быстрая скорость
F11	полный экран
Ctrl++ / Ctrl+− / Ctrl+0	масштаб экрана (или текста редактора, когда фокус клавиатуры в редакторе)
Ctrl+S	сохранить файл редактора
Ctrl+F / Ctrl+H / Ctrl+G	редактор: найти, заменить, перейти к строке
Ctrl+/	редактор: закомментировать / раскомментировать (6502), REM / убрать REM (BASIC)
F5 / F9 / F12	редактор: запустить, точка останова на строке, перейти к определению (6502)
Ctrl+буква	управляющий символ (`Ctrl+D` = `$84`)
правый щелчок по экрану	Копировать, Вставить (напечатать текст), Очистить экран, Сброс
Esc	ESC (`$9B`): отменяет строку в мониторе и в BASIC
backspace	набирает `_` — знак стирания монитора и BASIC

В **терминале** (`apple1-emu` без окна, глава 12):

Клавиша	Действие
F2	командная строка эмулятора
F5 / F6	кассета: воспроизведение, стоп
F10 или Ctrl+Q	выход

ПРИЛОЖЕНИЕ F

Ловушки и особенности поведения

Мелочи, которые удивляют в первый раз. Ни одна из них не ошибка; у каждой есть причина.

- **BASIC и RUN.** ▶ **Запустить** в редакторе запускает BASIC, загружает листинг и набирает за вас `RUN`. Программа, запрашивающая ввод, ждёт ваших клавиш на экране Apple-1: сначала щёлкните по экрану.
- **RND.** Первый `RND` запуска всегда был бы одним и тем же на только что сброшенной машине; ▶ **Запустить** ждёт случайное время перед `RUN`, чтобы игра не начиналась всегда одинаково.
- **Команды кассеты.** Строка «Кассета, на настоящем Apple-1» под редактором — это команда Woz Monitor, которую нужно набрать на настоящей машине (`C100R`, затем команды `R` для диапазонов) перед воспроизведением ленты. Это подсказка, а не то, что нужно набирать в эмуляторе.
- **Клавиатура идёт только в одно место.** Строка внизу окна говорит, куда: в Apple-1, в редактор, в поле. Щелчок по экрану Apple-1 направляет клавиши в машину; щелчок в редакторе возвращает их. После щелчка по ошибке в списке редактора клавиши возвращаются в редактор.
- **Отладчик на паузе.** Клавиши, набранные, пока машина на паузе, не теряются, но сохраняется только **последняя** (защёлка настоящей клавиатуры хранит один символ); она передаётся, когда вы продолжаете.
- **Точки останова привязаны к программе.** Загрузка другой программы (файл, библиотека, ассемблер) стирает точки останова и метки предыдущей. Ваши точки останова сохраняются от сеанса к сеансу, только пока вы не загрузили что-то другое.
- **Вставка средней кнопкой.** В редакторе щелчок средней кнопкой переносит туда курсор и вставляет **первичное выделение** X11/Wayland (текст, который вы только что выделили в любом приложении). Редактор пока не *записывает* своё выделение в первичное: чтобы скопировать из редактора, используйте Ctrl+C.
- **Маленькие окна.** На мониторе шириной меньше 1400 пикселей расположение первого запуска — редактор и экран; в меню ⊞ **Расположение** ▾ можно выбрать другое. На экране 1024 × 768 панель инструментов прокручивается по горизонтали: кнопка **»** открывает группу «Кассета», вместо того чтобы обрезать её справа. Фиксированный масштаб (Ctrl+ +) больше панели экрана сохраняется, и экран прокручивается в своей панели; строка состояния сообщает об этом.
- **Сохранение безымянной вкладки. Сохранить как** (значок, Ctrl+S или меню правого щелчка по вкладке) никогда не оставляет две вкладки для одного файла: уже открытая вкладка этого файла закрывается.

- **Первый запуск и сеть.** При первом запуске программа обращается к GitHub за демонстрационным проектом (глава 4); удалите проект из списка «Проекты GitHub» в ⚙ **Настройки**, чтобы прекратить это навсегда.
- **Чтение клавиатуры в G2 Basic.** Клавиша, нажатая во время работы программы, останавливает её, как в оригинале, а `INPUT` ждёт. Игра, которой нужно следить за клавишами, вызывает `KEY(0)` в главном цикле (глава 15); после этого клавиша программу больше не останавливает.
- **Метки и точки останова.** Точка останова на метке, которой ещё нет (набранная до ассемблирования или запуска), сохраняется и ставится, когда метка появляется.
- **Расположения и окна.** Сохранённое расположение помнит закреплённые панели, откреплённые окна (место и размер) и размер главного окна. Оно не помнит, какое окно владеет клавиатурой.

ПРИЛОЖЕНИЕ G

Лицензия и благодарности

Retro Dev Kit Apple 1 — свободное программное обеспечение, выпущенное под **Стандартной общественной лицензией GNU версии 3** (GPL-3.0), без каких-либо гарантий. Полный текст находится в файле `LICENSE` архива и репозитория.

Некоторые части взяты из других проектов и сохраняют собственную лицензию **MIT**, совместимую с GPL; они перечислены в файле `NOTICE`:

- **RDK G2 Basic**, встроенный BASIC, написанный для этого проекта (репозиторий `Old-Wirehead/rdk-gen2basic-apple1`, MIT);
- **тесты GEN2 от Uncle Bernie** (`gen2-test1` — `gen2-test4`, полосы) и **глифы GEN2**, предоставленные Uncle Bernie.

Игры библиотеки, написанные **Claude IA** (Виселица, Спички, Морской бой, Туннель и Крестики-нолики), распространяются вместе с проектом.

Чего в комплекте **нет**: Integer BASIC от Apple, коммерческих кассет и демонстраций карты P-LAB. Они остаются у владельцев; свои копии вы приносите сами.

Карта P-LAB — создание P-LAB; TMS9918A — микросхема Texas Instruments. Проект не связан с Apple, Texas Instruments и P-LAB.

Указатель

#

Загрузить 13
Сохранить 13
Обойти 47
Выйти 47
перенос 92
адр.адр 130
двойной щелчок 25
▶ Запустить 45
локальные метки 102
анонимные метки 102
Проверить последнюю сборку 28
📼 Экспорт WAV 45
Шаг (отладчик) 47
регистры (6502) 96
инструкции (6502) 97
>255 ERR 83
>32767 ERR 83
>8 FORS ERR 83
>8 GOSUBS ERR 83

A

ABS 75
AND 76
ASC 75
ascii 63
.asciiz 103
asm (командная строка) 63, 101
AUTO 71

B

BAD BRANCH ERR 83
BAD NEXT ERR 83
BAD RETURN ERR 83
BASIC 13
basic (командная строка) 63
.byte 103

C

C100R 32
$C250-$C257 36
CALL 75
cassette 62
charger 62
CLR 71
convert 63

D

DEL 71
DIM 75
DIM ERR 83
DSP 107

E

E000R 70
E2B3R 70
ECHO 109
END 75
END ERR 83
enregistrer 62
export 62

F

FOR 74

G

GETLINE 109
GOSUB 74
GOTO 74

H

HIMEM 71, 80

I

IF 74
.if 103
import 62
.include 103
info 63
INPUT 74

K

KBD 107
KBDCR 107

L

LEN 75
LET 74
LIST 71
LOMEM 71, 80

M

.macro 103
MAN 71
MEM FULL ERR 83
MOD 76

N

NEXT 74
NOT 76

O

OR 76
.org 103

P

PEEK 75
POKE 75
PP (указатель программы) 80
PRBYTE 109
PRHEX 109
PRINT 74
prom 63

R

адрR 130
R (чтение) 32
RANGE ERR 83
REM 75
.repeat 103
.res 103
RETYPE LINE 83
RND 75
RUN 71

S

sauver 62
SCR 71
SCRN 75
.setcpu 103
SGN 75
STOPPED AT 83
STR OVFL ERR 83
STRING ERR 83
SYNTAX ERR 83

T

TAB 75
TOO LONG ERR 83
TOO MANY PARENS ERR 83

U

USR 75

W

W (запись) 32
.word 103